

ĐẠI HỌC BÁCH KHOA HÀ NỘI



ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống đặt vé trực tuyến các sự kiện

Mai Quốc Đạt

Dat.MQ225277@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Nguyễn Thị Thanh Nga

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống đặt vé trực tuyến các sự kiện

Mai Quốc Đạt

Dat.MQ225277@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Nguyễn Thị Thanh Nga

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Lời đầu tiên, em xin trân trọng gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn đồ án tốt nghiệp, TS. Nguyễn Thị Thanh Nga, vì sự tận tình và chu đáo trong suốt quá trình em thực hiện đồ án. Em vô cùng biết ơn những ý kiến đóng góp quý báu và sâu sắc của cô, những điều đó đã giúp em nhận ra những thiếu sót của bản thân và hoàn thiện sản phẩm một cách tốt nhất, đáp ứng đầy đủ các yêu cầu của đồ án. Sự hỗ trợ và hướng dẫn của cô là nguồn động viên lớn lao, giúp em tự tin hơn trong việc phát triển và hoàn thành đồ án tốt nghiệp này.

Em xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong những năm gần đây, nhu cầu tổ chức và tham gia các sự kiện giải trí tại Việt Nam ngày càng tăng cao, trong khi các hệ thống đặt vé trực tuyến hiện có cũng đã tồn tại nhưng vẫn còn chưa đa dạng và đáp ứng được nhu cầu người dùng. Điều này tạo ra cơ hội phát triển một hệ thống đặt vé hiệu quả, dễ sử dụng và có khả năng mở rộng để đáp ứng nhu cầu ngày càng tăng của người dùng.

Đồ án tập trung xây dựng hệ thống đặt vé cho các sự kiện giải trí với ba mục tiêu chính: đảm bảo tính chính xác tuyệt đối về số lượng vé trong môi trường đồng thời cao, kiểm soát lưu lượng truy cập đột biến qua hàng chờ ảo theo cơ chế FIFO, và cá nhân hóa trải nghiệm người dùng thông qua hệ thống gợi ý sự kiện dựa trên AI.

Hệ thống được xây dựng theo kiến trúc Client-Server với frontend React 18, backend NestJS, cơ sở dữ liệu PostgreSQL và Redis quản lý kho vé thời gian thực. Để giải quyết bài toán bán vượt số lượng, đồ án áp dụng cơ chế kiểm tra và giữ chỗ nguyên tử bằng tập lệnh Lua trên Redis đảm bảo tính nguyên tử trong kiểm tra và giữ chỗ vé. Khi lưu lượng tăng đột biến, hàng chờ ảo (Virtual Waiting Room) sử dụng kiến trúc kết hợp giữa Redis và RabbitMQ để điều phối hàng đợi người dùng trước khi cho phép đặt vé. Hệ thống gợi ý sự kiện được triển khai qua Vietnamese-SBERT kết hợp pgvector. Thanh toán tích hợp SePay qua webhook xác thực API key và Socket.IO thông báo kết quả thời gian thực. Hệ thống được triển khai trên Internet thông qua nền tảng Vercel cho frontend và VPS cho backend.

Kết quả kiểm thử k6 với 1500 người dùng ảo (Virtual User — VU) đồng thời trên ba kịch bản xác nhận hệ thống không xảy ra bán vượt số lượng, hàng chờ ảo điều phối đúng thứ tự FIFO, và toàn bộ luồng từ đặt vé đến nhận vé điện tử hoạt động chính xác.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In recent years, the demand for organizing and attending entertainment events in Vietnam has grown considerably, yet existing online ticketing platforms, while present, remain limited in variety and fail to fully meet user needs. This gap presents a clear opportunity to develop an effective, user-friendly, and scalable ticketing system capable of meeting the increasingly diverse needs of users.

This thesis aims to build an online ticketing system for entertainment events around three core objectives: guaranteeing absolute ticket inventory accuracy under high concurrency, regulating traffic spikes through a FIFO-based virtual waiting room, and personalizing the user experience via an AI-driven event recommendation engine.

The system follows a Client-Server architecture comprising a React 18 frontend, a NestJS backend, a PostgreSQL database, and Redis for real-time inventory management. To eliminate overselling, an atomic ticket validation and reservation mechanism is implemented using Lua Scripts on Redis to enforce atomicity during ticket checking and seat holding. Under sudden traffic surges, the virtual waiting room — built on a hybrid Redis and RabbitMQ architecture — queues users and grants booking access in strict order. The event recommendation system leverages Vietnamese-SBERT embeddings stored and queried via pgvector. Payment is handled through SePay with API-key-authenticated webhooks, while Socket.IO delivers real-time booking notifications to clients. The system is deployed via Vercel for the frontend and a VPS (Virtual Private Server) for the backend.

Load testing with k6 at 1500 concurrent virtual users (VU) across three scenarios confirmed zero overselling, correct FIFO ordering in the virtual waiting room, and accurate end-to-end execution of the full booking and e-ticket delivery workflow.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	viii
DANH MỤC THUẬT NGỮ	ix
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã — Tìm kiếm sự kiện và đặt vé	7
2.2.3 Biểu đồ use case phân rã — Tổ chức và vận hành sự kiện	8
2.2.4 Biểu đồ use case phân rã — Phê duyệt và quản lý hệ thống	9
2.2.5 Quy trình nghiệp vụ	9
2.2.6 Vòng đời của vé và sự kiện.....	15
2.3 Đặc tả chức năng	17
2.3.1 Đặc tả use case: Đăng nhập hệ thống	17
2.3.2 Đặc tả use case: Đăng ký tài khoản.....	18
2.3.3 Đặc tả use case: Tìm kiếm sự kiện	19
2.3.4 Đặc tả use case: Đặt vé và thanh toán	19
2.3.5 Đặc tả use case: Tạo và quản lý sự kiện	21
2.3.6 Đặc tả use case: Check-in vé bằng mã QR	22
2.3.7 Đặc tả use case: Phê duyệt nhà tổ chức (KYC/KYB)	23

2.3.8 Đặc tả use case: Phê duyệt sự kiện	24
2.4 Yêu cầu phi chức năng	25
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	26
3.1 Tổng quan các lựa chọn công nghệ	26
3.2 Nhóm xử lý nghiệp vụ.....	27
3.2.1 NestJS và TypeScript.....	27
3.3 Nhóm dữ liệu	28
3.3.1 PostgreSQL và TypeORM.....	28
3.3.2 Redis	28
3.3.3 Hệ gợi ý ngữ nghĩa: Vietnamese-SBERT và pgvector	29
3.4 Giao tiếp thời gian thực và Xác thực	29
3.4.1 Hàng đợi tin nhắn: RabbitMQ	29
3.4.2 Giao tiếp hai chiều: Socket.IO	30
3.4.3 Xác thực và phân quyền: JWT và Passport.js	30
3.5 Nhóm giao diện người dùng.....	31
3.5.1 React 18 và Vite.....	31
3.5.2 Quản lý trạng thái và dữ liệu	31
3.5.3 Sơ đồ chỗ ngồi tương tác: Konva.js	31
3.6 Tích hợp dịch vụ và Triển khai.....	32
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	33
4.1 Thiết kế kiến trúc.....	33
4.1.1 Lựa chọn kiến trúc phần mềm	33
4.1.2 Thiết kế tổng quan.....	34
4.2 Thiết kế chi tiết.....	34
4.2.1 Thiết kế giao diện	34

4.2.2 Thiết kế hành vi	38
4.2.3 Thiết kế cơ sở dữ liệu	43
4.3 Xây dựng ứng dụng.....	49
4.3.1 Thư viện và công cụ sử dụng.....	49
4.3.2 Kết quả đạt được	50
4.3.3 Minh họa các chức năng chính	50
4.4 Kiểm thử.....	54
4.4.1 Kịch bản 1: Tính nguyên tử của thao tác gia nhập hàng chờ.....	54
4.4.2 Kịch bản 2: Tính đúng đắn của cơ chế đẩy hàng FIFO	55
4.4.3 Kịch bản 3: Tính chính xác tồn kho vé dưới tải cao.....	56
4.5 Triển khai	57
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	58
5.1 Kiểm soát tồn kho vé và hàng chờ ảo trong điều kiện lưu lượng đột biến....	58
5.1.1 Bài toán	58
5.1.2 Giải pháp	58
5.1.3 Kết quả	62
5.2 Bảo vệ hệ thống chống bot và lạm dụng tự động	62
5.2.1 Bài toán	62
5.2.2 Giải pháp	63
5.2.3 Kết quả	63
5.3 Hệ gợi ý sự kiện cá nhân hóa theo ngữ nghĩa tiếng Việt.....	64
5.3.1 Bài toán	64
5.3.2 Giải pháp	64
5.3.3 Kết quả	65
5.4 Giao diện thiết kế sơ đồ chỗ ngồi tương tác theo khu vực.....	66
5.4.1 Bài toán	66

5.4.2 Giải pháp	66
5.4.3 Kết quả	67
5.5 Quy trình xác minh nhà tổ chức và phê duyệt sự kiện	67
5.5.1 Bài toán	67
5.5.2 Giải pháp	67
5.5.3 Kết quả	68
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	69
6.1 Kết luận	69
6.2 Hướng phát triển	70
TÀI LIỆU THAM KHẢO	73

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	6
Hình 2.2	Biểu đồ use case phân rã nhóm chức năng tìm kiếm sự kiện và đặt vé	7
Hình 2.3	Biểu đồ use case phân rã nhóm chức năng tổ chức và vận hành sự kiện	8
Hình 2.4	Biểu đồ use case phân rã nhóm chức năng phê duyệt và quản lý hệ thống	9
Hình 2.5	Biểu đồ hoạt động quy trình đặt vé — Giai đoạn chọn vé và giữ chỗ	10
Hình 2.6	Biểu đồ hoạt động quy trình đặt vé — Giai đoạn thanh toán và xác nhận	11
Hình 2.7	Biểu đồ hoạt động quy trình xác minh danh tính người tổ chức (KYC/KYB)	12
Hình 2.8	Biểu đồ hoạt động quy trình tạo sự kiện và phê duyệt	13
Hình 2.9	Biểu đồ hoạt động quy trình vận hành sự kiện và quyết toán	14
Hình 2.10	Sơ đồ trạng thái vé (Ticket)	15
Hình 2.11	Sơ đồ trạng thái sự kiện (Event)	16
Hình 4.1	Biểu đồ phụ thuộc module của hệ thống	34
Hình 4.2	Giao diện trang chủ — danh sách và phân loại sự kiện	36
Hình 4.3	Giao diện trang chi tiết sự kiện — thông tin và lựa chọn vé	36
Hình 4.4	Giao diện tạo sự kiện mới — bước nhập thông tin cơ bản	37
Hình 4.5	Giao diện thanh toán — đếm ngược thời gian giữ vé và mã QR chuyển khoản	37
Hình 4.6	Giao diện trang vé của tôi — quản lý vé chờ dùng và vé đã dùng	38
Hình 4.7	Biểu đồ trình tự luồng đặt vé — Bước 1: Giữ chỗ vé (POST /bookings/reserve)	39
Hình 4.8	Biểu đồ trình tự luồng đặt vé — Bước 2: Tạo đơn hàng và lấy mã QR (POST /bookings/initiate-payment)	40
Hình 4.9	Biểu đồ trình tự luồng xác nhận thanh toán qua webhook SePay — Bước 1: Xác thực webhook	41
Hình 4.10	Biểu đồ trình tự luồng xác nhận thanh toán qua webhook SePay — Bước 2: Ghi nhận thanh toán và thông báo thời gian thực	42
Hình 4.11	Sơ đồ cơ sở dữ liệu vật lý của hệ thống	43

Hình 4.12	Giao diện trang chủ thực tế — tìm kiếm và gợi ý sự kiện cá nhân hóa	51
Hình 4.13	Giao diện trang chi tiết sự kiện thực tế — thông tin và lựa chọn hạng vé	51
Hình 4.14	Giao diện tạo sự kiện thực tế — quy trình nhiều bước nhập thông tin và cấu hình vé	52
Hình 4.15	Giao diện thiết kế sơ đồ chỗ ngồi thực tế — kéo thả phân vùng trên canvas	52
Hình 4.16	Giao diện hàng chờ ảo thực tế — hiển thị vị trí và thời gian chờ theo thời gian thực	53
Hình 4.17	Giao diện quản lý sự kiện thực tế — danh sách, trạng thái và thao tác của nhà tổ chức	53
Hình 4.18	Giao diện trang vé thực tế — quản lý vé điện tử và mã QR check-in	54
Hình 4.19	Kết quả kiểm thử tải kịch bản 1 — gia nhập hàng chờ đồng thời với 1500 VU	55
Hình 4.20	Kết quả kiểm thử tải kịch bản 2 — xác minh thứ tự FIFO khi đẩy hàng chờ	56
Hình 4.21	Kết quả kiểm thử tải kịch bản 3 — kiểm tra tồn kho vé với 1500 VU đồng thời	57
Hình 5.1	Kiến trúc ba luồng của hệ thống gợi ý sự kiện	65

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case UC-01 — Đăng nhập hệ thống	17
Bảng 2.2	Đặc tả use case UC-02 — Đăng ký tài khoản	18
Bảng 2.3	Đặc tả use case UC-03 — Tìm kiếm sự kiện	19
Bảng 2.4	Đặc tả use case UC-04 — Đặt vé và thanh toán	20
Bảng 2.5	Đặc tả use case UC-05 — Tạo và quản lý sự kiện	21
Bảng 2.6	Đặc tả use case UC-06 — Check-in vé bằng mã QR	22
Bảng 2.7	Đặc tả use case UC-07 — Phê duyệt nhà tổ chức (KYC/KYB)	23
Bảng 2.8	Đặc tả use case UC-08 — Phê duyệt sự kiện	24
Bảng 3.1	Tổng quan lựa chọn công nghệ theo nhóm chức năng	27
Bảng 4.1	Hệ thống điểm ngắt giao diện	34
Bảng 4.2	Bảng màu ngữ nghĩa của hệ thống	35
Bảng 4.3	Danh sách trang của ứng dụng frontend phân theo vai trò	35
Bảng 4.4	Nhóm bảng tài khoản và người dùng	43
Bảng 4.5	Nhóm bảng giao dịch tài chính	44
Bảng 4.6	Nhóm bảng sự kiện và cấu hình	44
Bảng 4.7	Đặc tả bảng <code>users</code>	45
Bảng 4.8	Đặc tả bảng <code>organizers</code>	46
Bảng 4.9	Đặc tả bảng <code>event_sessions</code>	46
Bảng 4.10	Đặc tả bảng <code>seat_maps</code>	47
Bảng 4.11	Đặc tả bảng <code>orders</code>	47
Bảng 4.12	Đặc tả bảng <code>tickets</code>	48
Bảng 4.13	Đặc tả bảng <code>ticket_types</code>	48
Bảng 4.14	Đặc tả bảng <code>events</code>	49
Bảng 4.15	Danh sách công cụ và thư viện sử dụng	50

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, nhất quán, cô lập và bền vững
AI	Artificial Intelligence	Trí tuệ nhân tạo
AMQP	Advanced Message Queuing Protocol	Giao thức hàng đợi tin nhắn nâng cao
API	Application Programming Interface	Giao diện lập trình ứng dụng
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart	Mã xác thực phân biệt người và máy
CDN	Content Delivery Network	Mạng phân phối nội dung
CNTT		Công nghệ thông tin
ĐATN		Đồ án tốt nghiệp
FIFO	First In, First Out	Vào trước ra trước
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm
HNSW	Hierarchical Navigable Small World	Đồ thị dẫn hướng thứ bậc
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
JWT	JSON Web Token	Thẻ xác thực JSON
KYB	Know Your Business	Xác minh danh tính doanh nghiệp
KYC	Know Your Customer	Xác minh danh tính khách hàng
ORM	Object-Relational Mapping	Ánh xạ đối tượng quan hệ
QR	Quick Response	Mã phản hồi nhanh
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
REST	Representational State Transfer	Kiểu kiến trúc truyền trạng thái đại diện
SLA	Service Level Agreement	Thỏa thuận mức dịch vụ
SMTP	Simple Mail Transfer Protocol	Giao thức gửi thư điện tử đơn giản
SV		Sinh viên
VPS	Virtual Private Server	Máy chủ riêng ảo
VU	Virtual User	Người dùng ảo
XSS	Cross-Site Scripting	Tấn công mã chèn trang web

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích
Bộ nhớ đệm (Cache)	Vùng lưu trữ tạm thời tốc độ cao, giảm thời gian truy xuất dữ liệu lặp lại
Check-in	Xác nhận sự hiện diện của người tham dự bằng cách quét mã QR trên vé điện tử
Escrow (Ký quỹ)	Cơ chế giữ tiền thanh toán đến khi các điều kiện hợp đồng được thỏa mãn, bảo vệ quyền lợi người mua
Hàng chờ ảo (Virtual Waiting Room)	Cơ chế điều phối người dùng truy cập theo thứ tự FIFO khi lưu lượng vượt ngưỡng cho phép
Kéo thả (Drag-and-drop)	Tương tác người dùng cho phép di chuyển và sắp xếp đối tượng trên giao diện bằng thao tác giữ và kéo chuột
Nonce (Mã dùng một lần)	Chuỗi ký tự ngẫu nhiên chỉ hợp lệ cho một lần sử dụng, chống tấn công phát lại
Oversell (Bán vượt số lượng)	Tình trạng vé bán ra vượt tồn kho thực tế do nhiều yêu cầu đồng thời không được kiểm soát
Phương án dự phòng (Fallback)	Cơ chế thay thế được kích hoạt tự động khi phương án chính không khả dụng hoặc không đủ điều kiện áp dụng
Rate limiting (Giới hạn tần suất)	Giới hạn số yêu cầu mỗi người dùng hoặc địa chỉ IP được gửi trong một khoảng thời gian nhất định
Sơ đồ chỗ ngồi (Seat map)	Biểu đồ tương tác thể hiện bố cục chỗ ngồi của sự kiện, cho phép người dùng chọn ghế trực tiếp
Tính bất biến (Idempotency)	Tính chất một thao tác cho kết quả như nhau dù thực thi nhiều lần, tránh xử lý trùng lặp
Véc-tơ nhúng (Embedding)	Biểu diễn văn bản dưới dạng véc-tơ số học mã hóa ngữ nghĩa, phục vụ tìm kiếm và gợi ý
Webhook	Cơ chế máy chủ gửi thông báo HTTP tự động tới URL định trước khi có sự kiện phát sinh
Xung đột tương tranh (Race condition)	Tình huống nhiều luồng đồng thời đọc và ghi tài nguyên chung thiếu đồng bộ, gây kết quả không nhất quán

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, ngành tổ chức sự kiện giải trí tại Việt Nam ghi nhận mức tăng trưởng đáng kể. Các sự kiện âm nhạc, thể thao, festival nghệ thuật ngày càng thu hút đông đảo người tham gia, với quy mô lên đến hàng chục nghìn khán giả mỗi sự kiện. Sự bùng nổ này kéo theo nhu cầu ngày càng cao về một nền tảng đặt vé trực tuyến vừa đáng tin cậy, vừa đủ khả năng xử lý lưu lượng truy cập lớn.

Về phía người dùng, trải nghiệm hiện tại còn rời rạc và thiếu cá nhân hóa. Người mua vé thường phải tự tìm kiếm sự kiện phù hợp qua nhiều kênh khác nhau mà không có sự hỗ trợ cá nhân hóa. Quá trình từ tìm kiếm, chọn vé, thanh toán đến nhận vé điện tử chưa được tích hợp liền mạch trên một nền tảng duy nhất, dẫn đến trải nghiệm phân mảnh và thiếu nhất quán. Nếu giải quyết được vấn đề này, người dùng có thể khám phá và tham gia sự kiện phù hợp với sở thích một cách nhanh chóng, thuận tiện, từ đó thúc đẩy sự phát triển của toàn ngành.

Về tính chính xác của vé, khi các sự kiện lớn mở bán, lượng người truy cập đồng thời có thể tăng vọt trong vài giây đầu tiên. Hệ thống không được thiết kế cho tải đồng thời cao dễ xảy ra tình trạng bán vượt số lượng, hoặc ghi nhận thanh toán thành công nhưng không phát hành được vé, gây thiệt hại cho cả người mua lẫn người tổ chức. Một cơ chế kiểm soát đồng thời chặt chẽ sẽ đảm bảo sự công bằng và minh bạch trong phân phối vé, bảo vệ uy tín của nền tảng.

Về phía người tổ chức, để vận hành một sự kiện cần đồng thời thiết lập thông tin sự kiện, cấu hình nhiều loại vé, theo dõi doanh thu và kiểm soát check-in tại cổng vào. Việc thiếu một bảng điều khiển tập trung khiến quy trình vận hành trở nên phức tạp và dễ xảy ra sai sót. Một hệ thống tích hợp đầy đủ các công cụ này sẽ giúp người tổ chức tiết kiệm thời gian, giảm thiểu rủi ro vận hành và nâng cao chất lượng tổ chức sự kiện.

Từ ba nhóm vấn đề trên, có thể thấy nhu cầu xây dựng một hệ thống đặt vé sự kiện trực tuyến đáp ứng đồng thời ba yêu cầu: trải nghiệm người dùng liền mạch, tính chính xác tuyệt đối về số lượng vé trong môi trường tải cao, và bộ công cụ quản lý toàn diện cho người tổ chức.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay tại Việt Nam đã có một số nền tảng đặt vé sự kiện trực tuyến phục vụ nhu cầu mua vé cho các sự kiện giải trí. Nhìn chung, các nền tảng này đáp ứng được luồng nghiệp vụ cơ bản: tạo sự kiện, bán vé và xử lý thanh toán. Tuy nhiên,

qua khảo sát thực tế, một số hạn chế vẫn còn tồn tại.

Về phía người dùng, các nền tảng hiện có chưa cung cấp khả năng gợi ý sự kiện cá nhân hóa dựa trên sở thích và lịch sử của từng người dùng. Việc khám phá sự kiện chủ yếu dựa vào tìm kiếm thủ công hoặc duyệt danh sách, khiến người dùng dễ bỏ lỡ các sự kiện phù hợp với mình. Bên cạnh đó, trải nghiệm chọn chỗ ngồi trên sơ đồ tương tác còn chưa phổ biến, đặc biệt với các sự kiện có phân khu chỗ ngồi phức tạp.

Về phía người tổ chức, bộ công cụ quản lý trên nhiều nền tảng nội địa chưa tích hợp đầy đủ các nghiệp vụ trong một giao diện duy nhất, từ cấu hình sự kiện, theo dõi doanh thu thời gian thực đến kiểm soát check-in tại cổng vào.

Về mặt kỹ thuật, bài toán đảm bảo tính chính xác tuyệt đối về số lượng vé khi lưu lượng tăng đột biến đòi hỏi các cơ chế xử lý đồng thời chặt chẽ, đây là thách thức kỹ thuật quan trọng mà không phải nền tảng nào cũng giải quyết triệt để.

Trên cơ sở đó, đề án hướng đến xây dựng một hệ thống đặt vé sự kiện trực tuyến tập trung vào ba mục tiêu: (i) đảm bảo tính chính xác tuyệt đối về số lượng vé trong môi trường đồng thời cao, (ii) cá nhân hóa trải nghiệm khám phá sự kiện bằng hệ thống gợi ý dựa trên mô hình ngôn ngữ tiếng Việt, và (iii) cung cấp bộ công cụ quản lý tích hợp cho người tổ chức gồm sơ đồ chỗ ngồi tương tác, theo dõi doanh thu và check-in bằng mã QR.

1.3 Định hướng giải pháp

Hệ thống được xây dựng theo kiến trúc Client-Server, với frontend sử dụng React 18 và backend sử dụng NestJS — hai framework hiện đại, có hệ sinh thái trưởng thành và phù hợp với yêu cầu phát triển ứng dụng web quy mô vừa đến lớn. Dữ liệu nghiệp vụ được lưu trữ trong PostgreSQL và mở rộng với pgvector để hỗ trợ tìm kiếm vector embedding cho hệ thống gợi ý. Redis được chọn làm lớp bộ nhớ đệm và quản lý trạng thái thời gian thực nhờ khả năng xử lý thao tác nguyên tử, trong khi RabbitMQ đảm nhận các tác vụ bất đồng bộ như nhả vé hết hạn và xử lý hoàn tiền. Thanh toán tích hợp SePay thông qua webhook, kết quả giao dịch được thông báo thời gian thực đến máy khách qua Socket.IO.

Từ định hướng công nghệ trên, đề án triển khai ba giải pháp chính. Để chống bán vượt số lượng, hệ thống áp dụng tập lệnh Lua nguyên tử trên Redis nhằm đảm bảo việc kiểm tra và giữ chỗ vé diễn ra như một thao tác không thể bị gián đoạn, ngay cả khi có hàng nghìn yêu cầu đồng thời. Khi lưu lượng tăng đột biến, hàng chờ ảo điều phối người dùng theo cơ chế FIFO thông qua kiến trúc kết hợp Redis và RabbitMQ, đảm bảo hệ thống không bị quá tải trong khi vẫn duy trì thứ tự công

bằng cho người dùng. Để cá nhân hóa trải nghiệm, hệ thống gợi ý sự kiện sử dụng mô hình Vietnamese-SBERT tạo véc-tơ nhúng ngữ nghĩa tiếng Việt, kết hợp với pgvector để truy vấn độ tương đồng hiệu quả.

Kết quả kiểm thử tải với k6 ở mức 1500 người dùng ảo đồng thời xác nhận hệ thống không xảy ra tình trạng bán vượt số lượng, toàn bộ luồng từ đặt vé đến nhận vé điện tử hoạt động chính xác. Đây là những đóng góp kỹ thuật chính của đề án, được trình bày chi tiết trong Chương 5.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày kết quả khảo sát và phân tích yêu cầu hệ thống. Chương này mô tả chi tiết các nhóm người dùng và vai trò của họ trong hệ thống, bao gồm người mua vé, người tổ chức sự kiện, cộng tác viên check-in và quản trị viên. Trên cơ sở đó, chương đặc tả đầy đủ các yêu cầu chức năng theo từng nhóm người dùng và các yêu cầu phi chức năng về hiệu năng, bảo mật và tính khả dụng, làm nền tảng cho quá trình thiết kế ở các chương tiếp theo.

Chương 3 giới thiệu các nền tảng lý thuyết và công nghệ được áp dụng trong đề án. Chương tập trung vào ba nhóm kiến thức cốt lõi: các cơ chế xử lý đồng thời và đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, kiến trúc hàng đợi tin nhắn với RabbitMQ và mô hình truyền thông bất đồng bộ, và kỹ thuật biểu diễn ngữ nghĩa văn bản tiếng Việt bằng mô hình Sentence-BERT kết hợp với tìm kiếm vector trên pgvector. Ngoài ra, chương cũng trình bày cơ chế tích hợp cổng thanh toán thông qua webhook.

Chương 4 trình bày quá trình phân tích thiết kế, triển khai và đánh giá hệ thống. Nội dung bao gồm thiết kế kiến trúc tổng thể theo mô hình Client-Server, thiết kế cơ sở dữ liệu quan hệ và lược đồ vector embedding, cùng các luồng nghiệp vụ chính như đặt vé, thanh toán, check-in và quản lý sự kiện. Phần cuối chương trình bày kết quả kiểm thử hiệu năng với công cụ k6, bao gồm kịch bản 1500 người dùng ảo đồng thời và đánh giá các chỉ số độ trễ, tính chính xác của hệ thống.

Chương 5 trình bày năm giải pháp kỹ thuật nổi bật của đề án. Giải pháp thứ nhất là cơ chế kiểm soát tồn kho nguyên tử bằng tập lệnh Lua trên Redis kết hợp hàng chờ ảo với kiến trúc kết hợp Redis và RabbitMQ, đảm bảo không xảy ra bán vượt số lượng trong điều kiện lưu lượng đột biến. Giải pháp thứ hai là bảo vệ đa lớp chống bot gồm xác thực CAPTCHA qua Cloudflare Turnstile, mã đặt vé một lần và giới hạn tần suất nhiều tầng. Giải pháp thứ ba là hệ thống gợi ý sự kiện cá nhân hóa sử dụng Vietnamese-SBERT để tạo véc-tơ nhúng ngữ nghĩa tiếng Việt

kết hợp với pgvector, triển khai theo ba chiến lược: item-to-item, user-to-item và theo độ phổ biến. Giải pháp thứ tư là giao diện thiết kế sơ đồ chỗ ngồi tương tác bằng Konva.js với cập nhật trạng thái thời gian thực qua Socket.IO. Giải pháp thứ năm là quy trình xác minh danh tính nhà tổ chức và cơ chế ký quỹ bảo vệ quyền lợi người mua.

Chương 6 tổng kết các kết quả đạt được so với mục tiêu đề ra ban đầu, chỉ ra các hạn chế còn tồn tại của hệ thống và đề xuất các hướng phát triển tiếp theo trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày kết quả khảo sát hiện trạng và phân tích yêu cầu cho hệ thống đặt vé sự kiện trực tuyến. Trên cơ sở các vấn đề đã xác định ở Chương 1, chương này làm rõ các chức năng cần xây dựng thông qua biểu đồ use case tổng quan, phân rã từng nhóm chức năng, mô tả quy trình nghiệp vụ chính, và đặc tả chi tiết các use case quan trọng nhất. Phần cuối chương tổng hợp các yêu cầu phi chức năng làm cơ sở cho quá trình thiết kế ở Chương 4.

2.1 Khảo sát hiện trạng

Yêu cầu của hệ thống được xác định từ hai nguồn chính. Nguồn thứ nhất là phân tích các hạn chế của các nền tảng đặt vé sự kiện hiện có tại thị trường Việt Nam. Nguồn thứ hai là khảo sát trực tiếp nhu cầu của người mua vé và người tổ chức sự kiện, qua đó xác định các kỳ vọng cụ thể về trải nghiệm đặt vé, quản lý sự kiện và kiểm soát check-in.

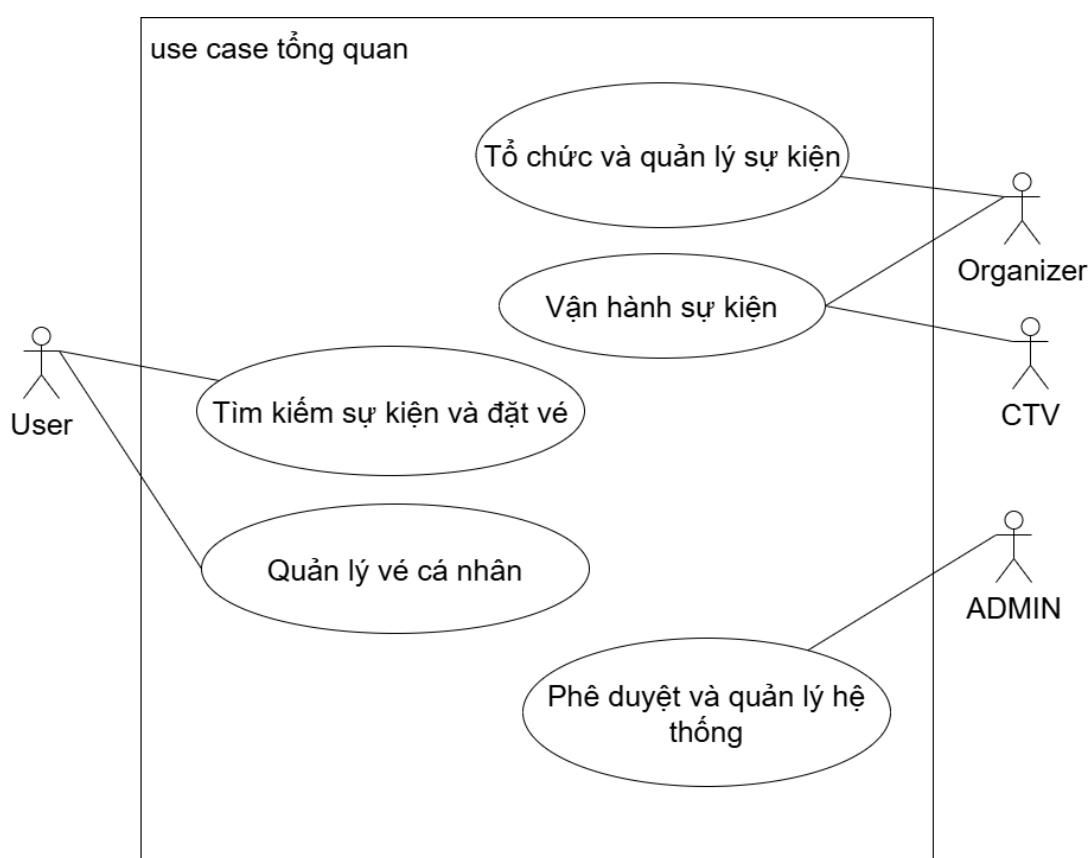
Về phía các nền tảng hiện có, các hệ thống đặt vé trực tuyến đang hoạt động tại Việt Nam đã đáp ứng được luồng nghiệp vụ cơ bản gồm tạo sự kiện, bán vé và xử lý thanh toán. Tuy nhiên, qua khảo sát thực tế, một số hạn chế đáng chú ý vẫn còn tồn tại. Về phía người dùng, các nền tảng này chưa cung cấp khả năng gợi ý sự kiện cá nhân hóa dựa trên sở thích và lịch sử tương tác của từng người; việc khám phá sự kiện chủ yếu dựa vào tìm kiếm thủ công hoặc duyệt danh sách. Trải nghiệm chọn chỗ ngồi theo sơ đồ tương tác cũng chưa phổ biến, đặc biệt với các sự kiện có phân khu chỗ ngồi phức tạp. Về phía người tổ chức, bộ công cụ quản lý trên nhiều nền tảng nội địa chưa tích hợp đầy đủ các nghiệp vụ trong một giao diện duy nhất, từ cấu hình sự kiện, theo dõi doanh thu thời gian thực đến kiểm soát check-in tại cổng vào. Về mặt kỹ thuật, bài toán đảm bảo tính chính xác tuyệt đối về số lượng vé khi lưu lượng tăng đột biến — đặc biệt trong các đợt mở bán sự kiện lớn — đòi hỏi các cơ chế xử lý đồng thời chặt chẽ mà không phải nền tảng nào cũng giải quyết triệt để.

Từ kết quả khảo sát, hệ thống cần xây dựng ba nhóm chức năng chính. Nhóm thứ nhất phục vụ người mua vé, bao gồm tìm kiếm và khám phá sự kiện có hỗ trợ gợi ý cá nhân hóa, đặt vé và thanh toán trực tuyến, cùng quản lý vé điện tử cá nhân. Nhóm thứ hai phục vụ người tổ chức sự kiện, bao gồm tạo và quản lý sự kiện qua quy trình phê duyệt, cấu hình nhiều loại vé linh hoạt, sơ đồ chỗ ngồi tương tác, theo dõi doanh thu thời gian thực và check-in bằng mã QR. Nhóm thứ ba phục vụ quản trị viên hệ thống, bao gồm phê duyệt nhà tổ chức và sự kiện, quản lý tài khoản người dùng và đối soát tài chính.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát

Hệ thống có bốn tác nhân chính tham gia tương tác. *Người dùng* (User) là tác nhân cơ bản đại diện cho người mua vé, thực hiện tìm kiếm và đặt vé sự kiện cùng quản lý vé cá nhân. *Người tổ chức* (Organizer) kế thừa toàn bộ quyền của User và bổ sung thêm nhóm chức năng tổ chức, quản lý sự kiện cùng vận hành trong ngày diễn ra. *Cộng tác viên* (CTV) là tác nhân chuyên biệt do Organizer tạo và cấp quyền, chỉ tham gia vào nhóm chức năng vận hành sự kiện với quyền duy nhất là quét mã QR check-in tại cổng vào sự kiện được chỉ định. *Quản trị viên* (Admin) chịu trách nhiệm phê duyệt và quản lý toàn bộ hệ thống ở cấp độ vận hành.

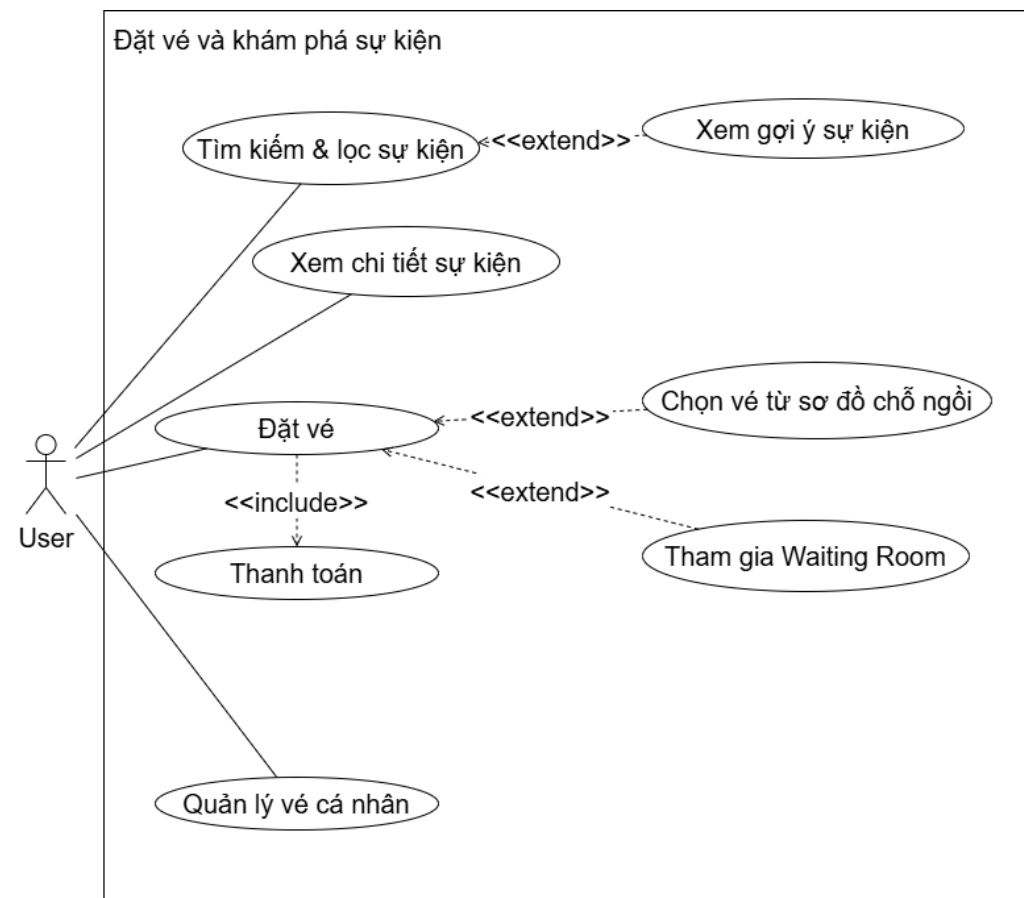


Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Hình 2.1 trình bày biểu đồ use case tổng quát với năm use case cốt lõi. *Tìm kiếm sự kiện và đặt vé* cùng *Quản lý vé cá nhân* là hai use case dành cho User, đồng thời được Organizer kế thừa thông qua quan hệ tổng quát hóa. *Tổ chức và quản lý sự kiện* bao quát toàn bộ vòng đời của một sự kiện từ góc độ Organizer, từ tạo sự kiện, cấu hình vé và thiết lập sơ đồ chỗ ngồi đến gửi lên để Admin xét duyệt. *Vận hành sự kiện* là use case dùng chung giữa Organizer và CTV, tập trung vào các tác vụ trong ngày diễn ra gồm check-in bằng mã QR và theo dõi doanh thu thời gian thực. *Phê duyệt và quản lý hệ thống* là use case dành riêng cho Admin, bao gồm

xét duyệt hồ sơ xác minh danh tính nhà tổ chức (Know Your Customer/Business — KYC/KYB), phê duyệt sự kiện, quản lý tài khoản người dùng và giám sát quyết toán doanh thu.

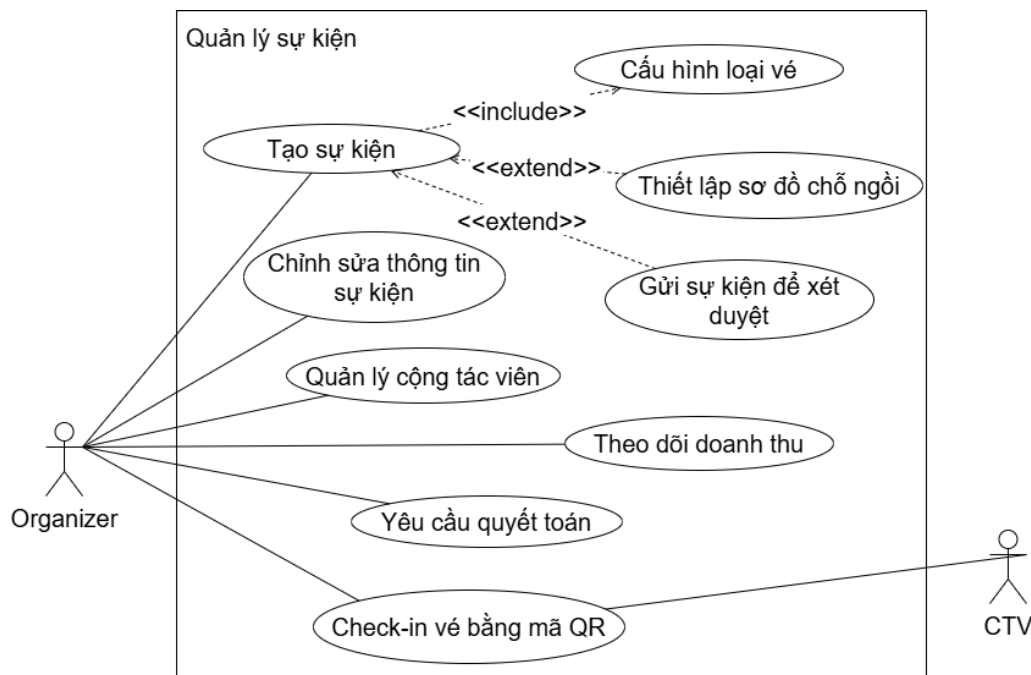
2.2.2 Biểu đồ use case phân rã — Tìm kiếm sự kiện và đặt vé



Hình 2.2: Biểu đồ use case phân rã nhóm chức năng tìm kiếm sự kiện và đặt vé

Hình 2.2 mô tả chi tiết nhóm chức năng phục vụ người mua vé. Use case *Tìm kiếm sự kiện* bao gồm các chức năng phân rã: lọc theo thể loại, địa điểm và thời gian; sắp xếp theo tiêu chí; và xem gợi ý cá nhân hóa từ hệ thống AI. Use case *Đặt vé và thanh toán* phân rã thành: xem chi tiết sự kiện và sơ đồ chỗ ngồi, chọn loại vé và số lượng, vào hàng chờ ảo khi lưu lượng cao, xác nhận đặt và thanh toán qua mã QR, nhận vé điện tử sau khi thanh toán thành công. Use case *Quản lý vé cá nhân* cho phép người dùng xem lịch sử mua vé và xem chi tiết từng vé điện tử.

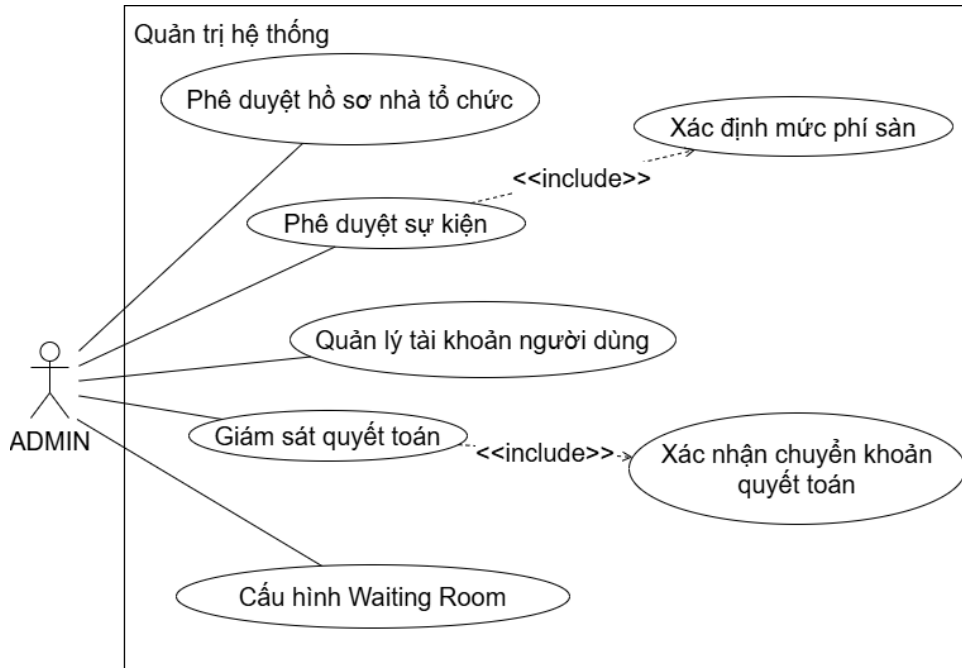
2.2.3 Biểu đồ use case phân rã — Tổ chức và vận hành sự kiện



Hình 2.3: Biểu đồ use case phân rã nhóm chức năng tổ chức và vận hành sự kiện

Hình 2.3 mô tả nhóm chức năng dành cho Organizer và CTV. Use case *Tạo và quản lý sự kiện* phân rã thành: nhập thông tin sự kiện (tên, mô tả, địa điểm, thời gian, hình ảnh), thiết lập sơ đồ chỗ ngồi bằng công cụ kéo thả, đính kèm văn bản pháp lý và gửi lên để Admin xét duyệt, và chỉnh sửa thông tin sau khi được phê duyệt. Use case *Cấu hình loại vé* cho phép tạo nhiều loại vé với giá, số lượng. Use case *Thiết lập sơ đồ chỗ ngồi* cho phép tạo và chỉnh sửa sơ đồ chỗ ngồi theo khu vực bằng công cụ kéo thả tương tác, sau đó gắn mỗi khu vực với loại vé tương ứng; đây là bước tùy chọn chỉ áp dụng với các sự kiện có phân khu chỗ ngồi cố định. Use case *Theo dõi doanh thu* hiển thị số liệu bán vé và doanh thu thời gian thực theo từng sự kiện, bao gồm số lượng vé đã bán theo từng loại và tổng doanh thu tích lũy. Use case *Check-in vé* — dùng chung cho cả Organizer và CTV — cho phép quét mã QR của vé tại cổng vào để xác thực và đánh dấu đã sử dụng.

2.2.4 Biểu đồ use case phân rã — Phê duyệt và quản lý hệ thống



Hình 2.4: Biểu đồ use case phân rã nhóm chức năng phê duyệt và quản lý hệ thống

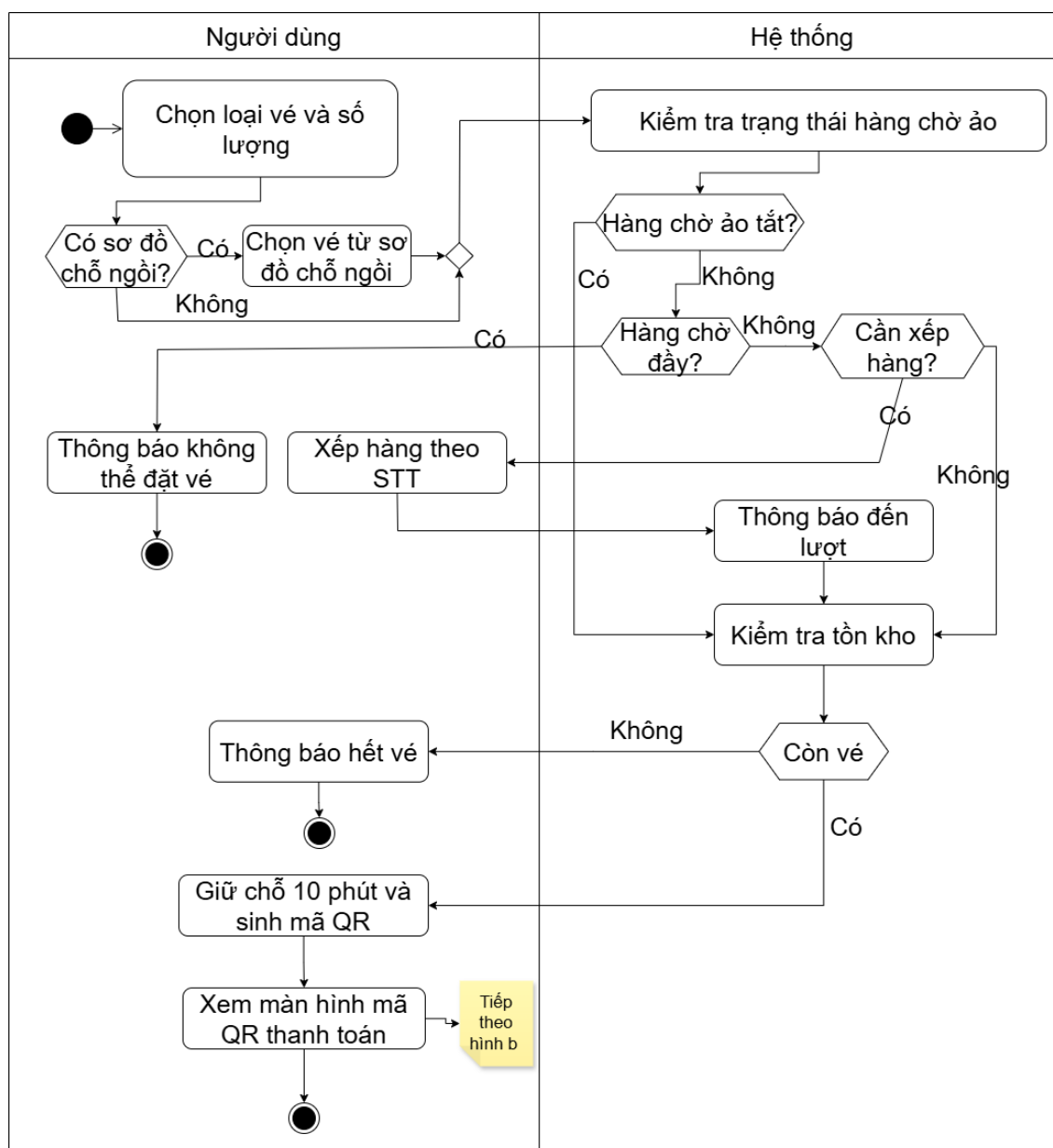
Hình 2.4 mô tả nhóm chức năng của Admin. Use case *Phê duyệt nhà tổ chức* cho phép Admin xem xét hồ sơ KYC/KYB, thực hiện checklist kiểm tra tài liệu pháp lý và ra quyết định chấp thuận hoặc từ chối trong thời hạn thỏa thuận mức dịch vụ (Service Level Agreement — SLA) hai ngày. Use case *Phê duyệt sự kiện* cho phép Admin xét duyệt sự kiện theo ba cấp độ quy mô với checklist và SLA tương ứng, đồng thời xác định mức phí sàn. Use case *Quản lý tài khoản người dùng* cho phép Admin xem danh sách toàn bộ tài khoản, khóa hoặc mở khóa tài khoản vi phạm. Use case *Giám sát quyết toán* cho phép Admin xem thống kê doanh thu toàn hệ thống và xác nhận chuyển khoản quyết toán cho từng Organizer sau khi sự kiện kết thúc.

2.2.5 Quy trình nghiệp vụ

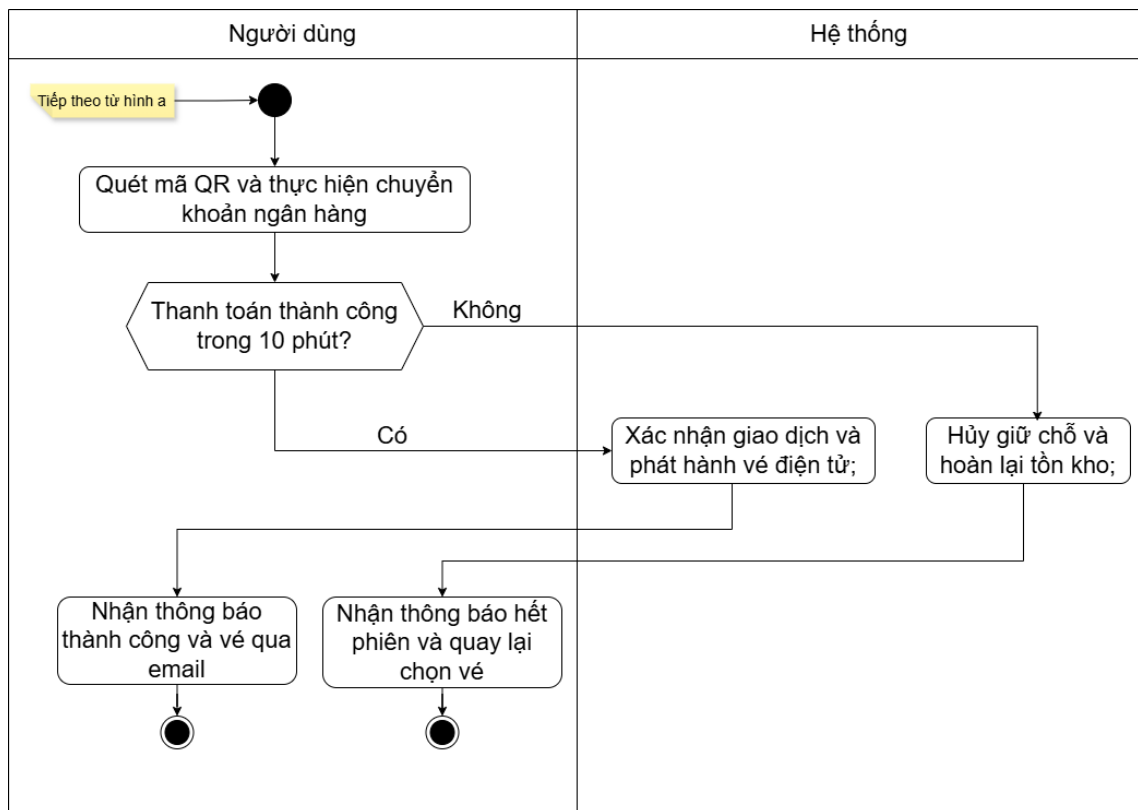
Hệ thống có hai quy trình nghiệp vụ cốt lõi cần mô tả chi tiết.

Quy trình thứ nhất là đặt vé và thanh toán — quy trình trung tâm của hệ thống, kết hợp nhiều use case từ tìm kiếm đến nhận vé điện tử. Quy trình bắt đầu khi người dùng tìm kiếm và chọn sự kiện muốn tham dự. Nếu sự kiện đang trong giai đoạn lưu lượng cao và người tổ chức bật tính năng hàng chờ ảo, người dùng được đưa vào phòng chờ và nhận số thứ tự theo nguyên tắc FIFO. Khi đến lượt, hệ thống cho phép người dùng chọn loại vé và số lượng; lúc này một thao tác nguyên tử trên Redis được thực hiện để kiểm tra tồn kho và giữ chỗ vé trong vòng mười phút. Người dùng tiến hành thanh toán bằng cách quét mã QR được sinh ra tương ứng với

đơn hàng. Sau khi cổng thanh toán xác nhận giao dịch thành công qua webhook, hệ thống xác thực API key của cổng thanh toán, tạo vé điện tử trong cơ sở dữ liệu và thông báo kết quả thời gian thực đến máy khách qua Socket.IO. Nếu hết mười phút mà người dùng chưa hoàn tất thanh toán, RabbitMQ sẽ kích hoạt tác vụ trả vé về kho tự động. Hình 2.5 thể hiện giai đoạn chọn vé và giữ chỗ: nếu sự kiện có sơ đồ chỗ ngồi, người dùng chọn trực tiếp từ khu vực tương ứng trên sơ đồ; hệ thống kiểm tra trạng thái hàng chờ ảo và điều phối theo thứ tự FIFO; sau khi xác nhận còn vé, hệ thống giữ chỗ trong mười phút và sinh mã QR thanh toán.



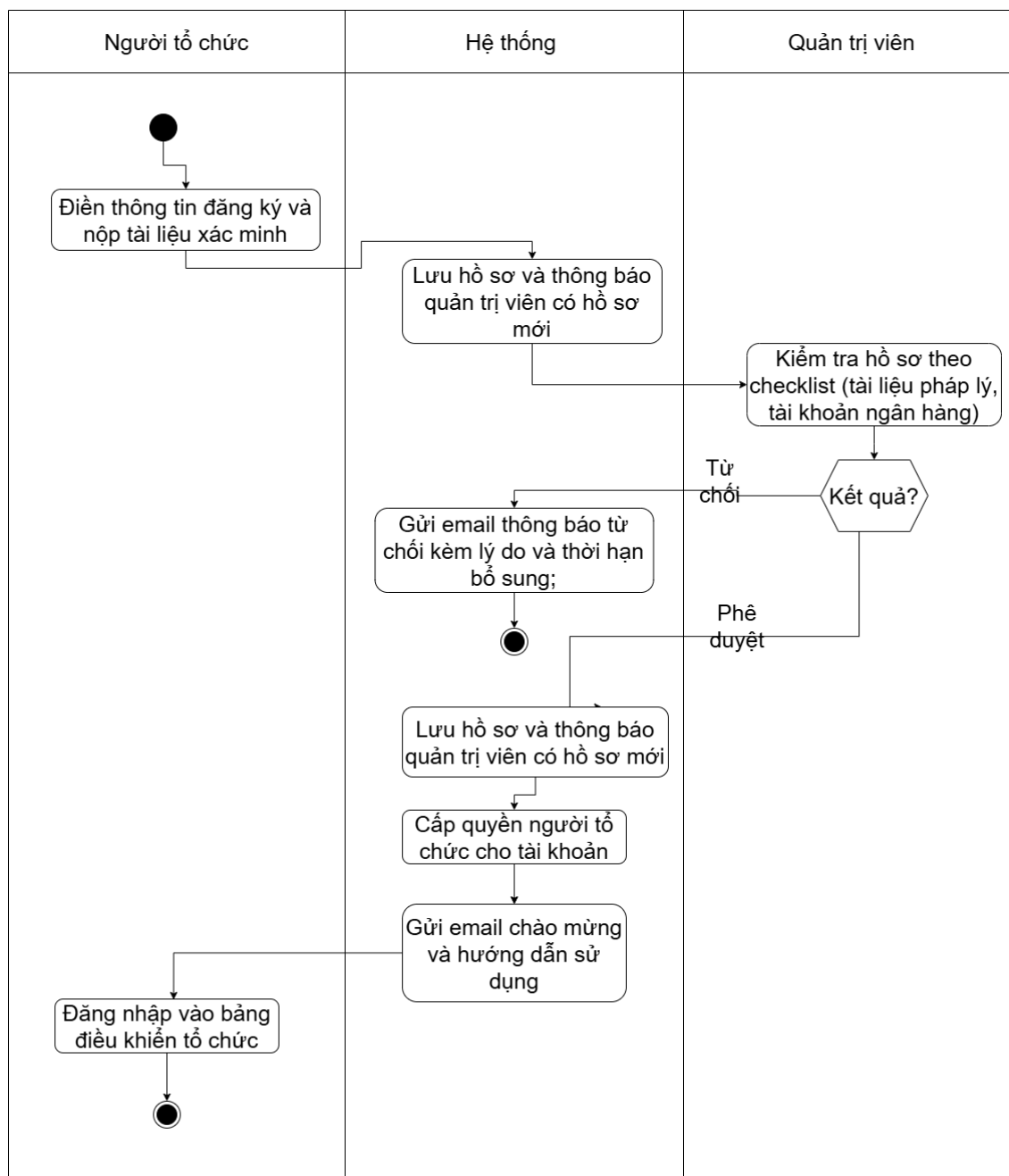
Hình 2.5: Biểu đồ hoạt động quy trình đặt vé — Giai đoạn chọn vé và giữ chỗ



Hình 2.6: Biểu đồ hoạt động quy trình đặt vé — Giai đoạn thanh toán và xác nhận

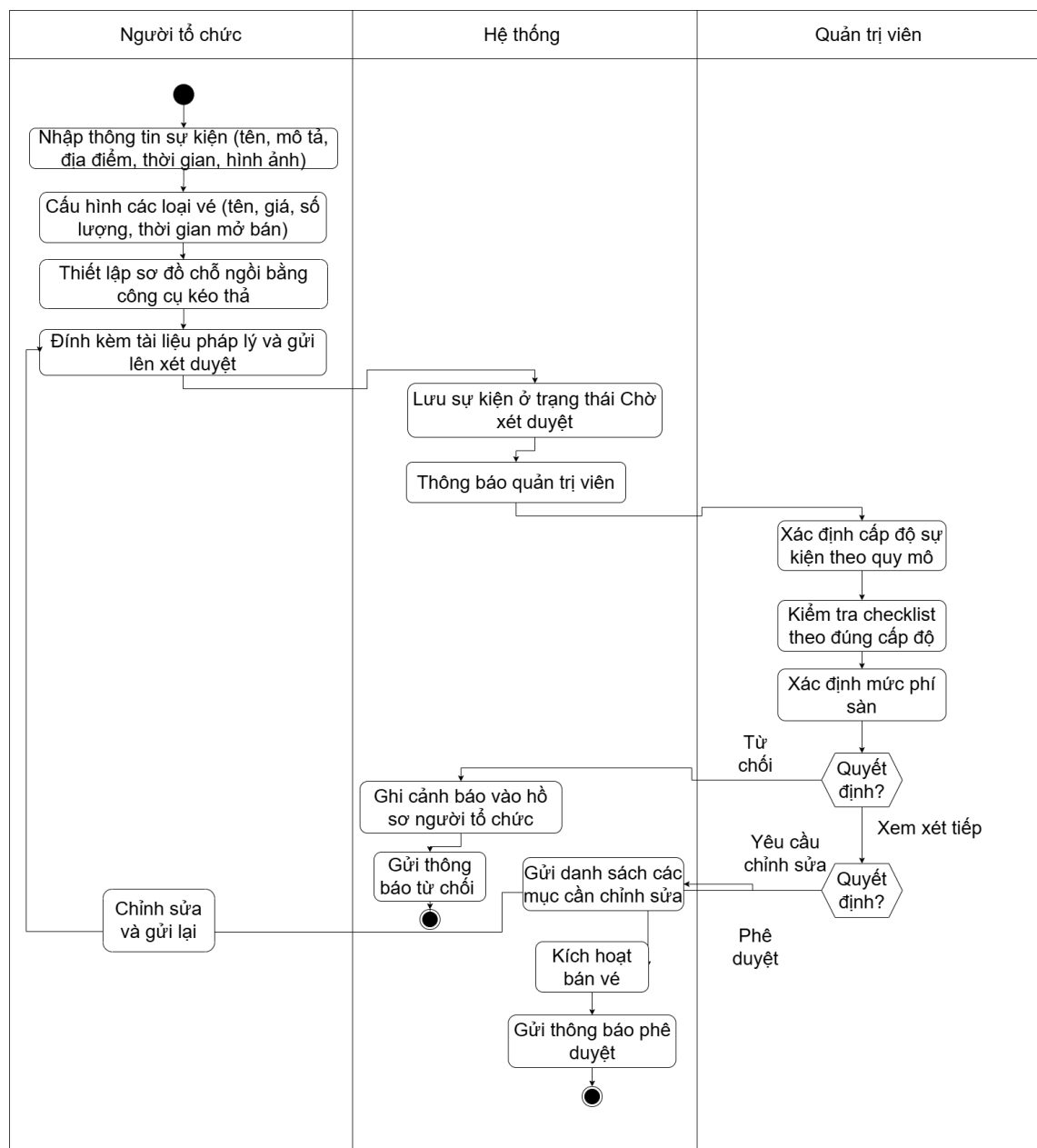
Hình 2.6 thể hiện giai đoạn tiếp theo về thanh toán: nếu người dùng hoàn tất chuyển khoản trong thời hạn, vé điện tử được phát hành và gửi qua email; nếu hết thời gian, hệ thống tự động hủy giữ chỗ và hoàn lại tiền kho.

Quy trình thứ hai là tổ chức và vận hành sự kiện, mô tả vòng đời đầy đủ của một sự kiện từ góc độ người tổ chức. Trước khi tạo sự kiện, người tổ chức phải hoàn tất quy trình xác minh danh tính và được Admin phê duyệt hồ sơ. Sau khi có đủ quyền, người tổ chức tạo sự kiện với thông tin mô tả, hình ảnh và địa điểm, thiết lập sơ đồ chỗ ngồi bằng công cụ kéo thả tương tác, rồi cấu hình các loại vé với mức giá, số lượng và thời gian mở bán tương ứng. Sự kiện sau khi tạo ở trạng thái nháp phải được người tổ chức gửi lên để Admin xem xét; chỉ khi Admin phê duyệt, sự kiện mới chuyển sang trạng thái sắp diễn ra và bắt đầu nhận đặt vé; đến ngày tổ chức, trạng thái tự động chuyển thành đang diễn ra. Trong quá trình bán vé, hệ thống cập nhật số liệu doanh thu thời gian thực trên bảng điều khiển của người tổ chức. Trong ngày diễn ra sự kiện, người tổ chức và cộng tác viên sử dụng chức năng check-in để quét mã QR của từng vé tại cổng vào, hệ thống xác thực tính hợp lệ và đánh dấu vé đã sử dụng ngay lập tức. Sau khi sự kiện kết thúc, hệ thống thực hiện quy trình quyết toán để chuyển doanh thu về tài khoản người tổ chức sau khi trừ phí nền tảng.



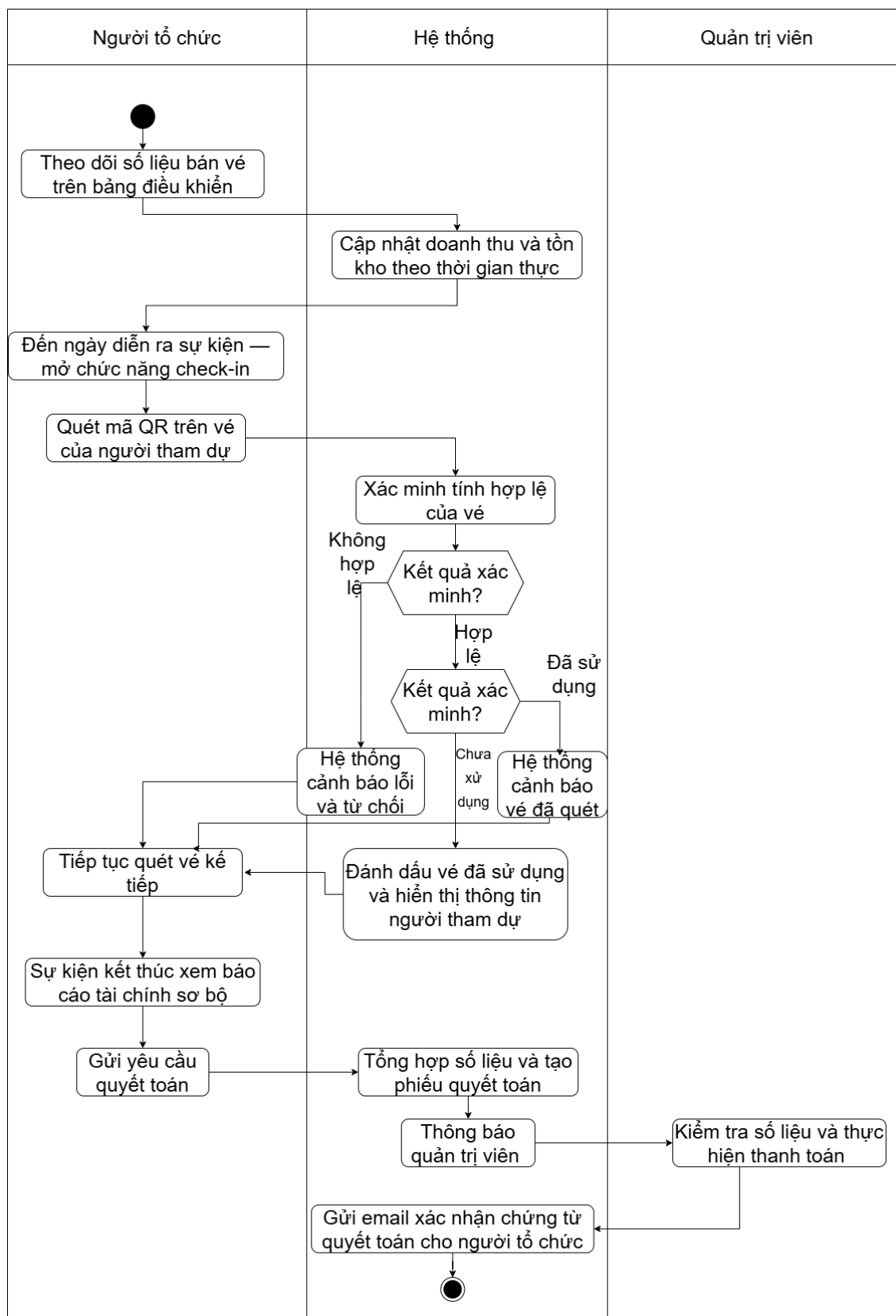
Hình 2.7: Biểu đồ hoạt động quy trình xác minh danh tính người tổ chức (KYC/KYB)

Hình 2.7 mô tả quy trình xác minh danh tính: người tổ chức nộp hồ sơ kèm tài liệu pháp lý; quản trị viên kiểm tra theo các đề mục và ra quyết định phê duyệt hoặc từ chối kèm lý do; khi được phê duyệt, hệ thống cấp quyền và thông báo cho người tổ chức để bắt đầu sử dụng bảng điều khiển.



Hình 2.8: Biểu đồ hoạt động quy trình tạo sự kiện và phê duyệt

Hình 2.8 mô tả quy trình tạo và xét duyệt sự kiện: người tổ chức nhập thông tin sự kiện, thiết lập sơ đồ chỗ ngồi, cấu hình các loại vé và đính kèm tài liệu pháp lý trước khi gửi lên; quản trị viên xác định cấp độ quy mô, thực hiện kiểm tra các đề mục tương ứng và xác định mức phí sàn, sau đó ra một trong ba quyết định: phê duyệt để kích hoạt bán vé, yêu cầu chỉnh sửa với thời hạn phản hồi năm ngày, hoặc từ chối vĩnh viễn khi sự kiện vi phạm quy định.



Hình 2.9: Biểu đồ hoạt động quy trình vận hành sự kiện và quyết toán

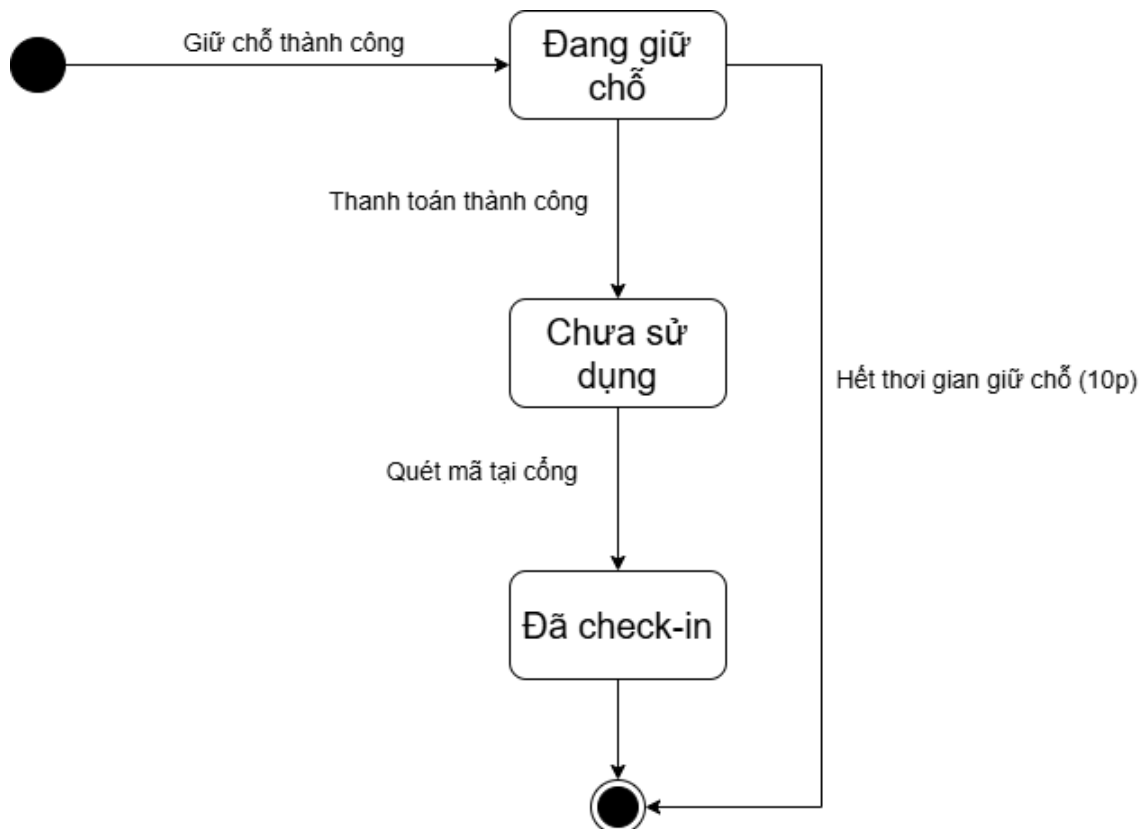
Hình 2.9 mô tả giai đoạn vận hành: trong thời gian mở bán, người tổ chức theo dõi số liệu doanh thu được cập nhật theo thời gian thực; đến ngày diễn ra sự kiện, người tổ chức và cộng tác viên sử dụng chức năng quét mã QR để check-in từng

người tham dự tại cổng vào — hệ thống xác minh tính hợp lệ của vé và đánh dấu đã sử dụng ngay lập tức; sau khi sự kiện kết thúc, người tổ chức xem báo cáo tài chính sơ bộ và gửi yêu cầu quyết toán, quản trị viên xác nhận chuyển khoản và hệ thống gửi chứng từ quyết toán cho người tổ chức.

2.2.6 Vòng đời của vé và sự kiện

Để làm rõ hành vi của hệ thống trong các tình huống bất thường, phần này mô tả vòng đời của hai thực thể trung tâm: vé và sự kiện.

Vòng đời của vé phản ánh trực tiếp luồng nghiệp vụ đặt vé. Cần phân biệt hai giai đoạn: trước và sau thanh toán. Trước thanh toán, hệ thống không tạo bản ghi vé trong cơ sở dữ liệu — trạng thái giữ chỗ chỉ tồn tại dưới dạng khóa tạm thời trong Redis với thời gian sống mười phút, kèm một tin nhắn có độ trễ trên RabbitMQ để tự động hoàn lại tồn kho nếu hết hạn. Khi người dùng hoàn tất thanh toán trong thời hạn, vé điện tử mới được tạo trong cơ sở dữ liệu với trạng thái chưa sử dụng. Khi vé được quét tại cổng vào sự kiện, trạng thái chuyển thành đã check-in — đây là trạng thái cuối không thể thay đổi.

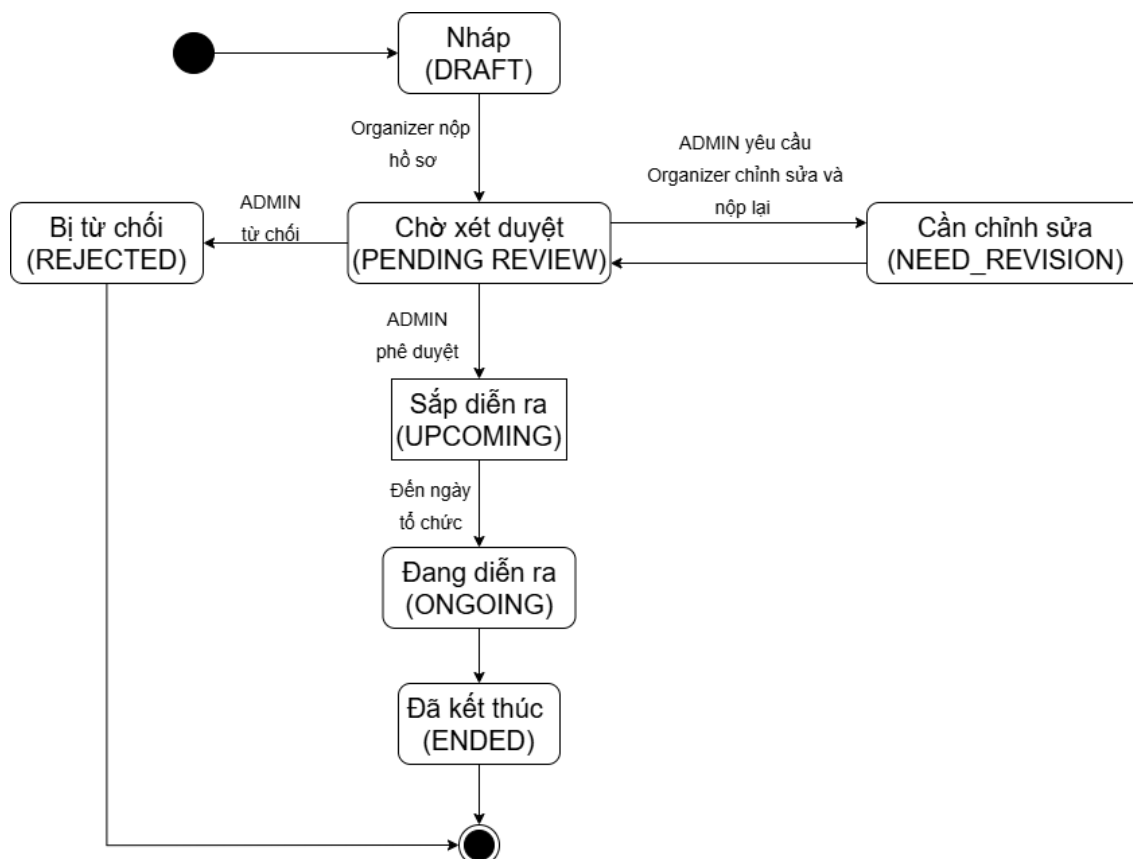


Hình 2.10: Sơ đồ trạng thái vé (Ticket)

Hình 2.10 minh họa toàn bộ các trạng thái của vé và điều kiện chuyển đổi giữa chúng.

Vòng đời của sự kiện phản ánh quy trình phê duyệt và vận hành. Sự kiện mới

được tạo ở trạng thái nháp. Sau khi người tổ chức hoàn thiện thông tin và gửi lên, sự kiện chuyển sang chờ xét duyệt. Admin xem xét và có thể ra một trong ba quyết định: phê duyệt, yêu cầu chỉnh sửa, hoặc từ chối vĩnh viễn. Khi bị từ chối vĩnh viễn, sự kiện chuyển sang trạng thái bị từ chối. Khi yêu cầu chỉnh sửa, người tổ chức có năm ngày để phản hồi; quá thời hạn mà không có phản hồi, sự kiện tự động bị hủy. Khi được phê duyệt, sự kiện chuyển sang trạng thái sắp diễn ra và bắt đầu nhận đặt vé; đến ngày tổ chức, trạng thái tự động chuyển thành đang diễn ra. Khi hết thời gian tổ chức, sự kiện chuyển sang trạng thái kết thúc và kích hoạt quy trình quyết toán.



Hình 2.11: Sơ đồ trạng thái sự kiện (Event)

Hình 2.11 minh họa toàn bộ các trạng thái của sự kiện và điều kiện chuyển đổi tương ứng.

2.3 Đặc tả chức năng

Phần này đặc tả chi tiết bảy use case quan trọng nhất của hệ thống, được lựa chọn dựa trên mức độ phức tạp và tầm ảnh hưởng đến trải nghiệm người dùng.

2.3.1 Đặc tả use case: Đăng nhập hệ thống

Bảng 2.1 đặc tả chi tiết use case đăng nhập hệ thống, bao gồm hai phương thức xác thực được hỗ trợ là đăng nhập bằng email/mật khẩu và đăng nhập qua Google OAuth2.

Tên use case	Đăng nhập hệ thống
Mã use case	UC-01
Tác nhân	User, Organizer, Admin, CTV
Mô tả	Người dùng xác thực danh tính để truy cập hệ thống thông qua email/mật khẩu hoặc tài khoản Google.
Tiền điều kiện	Người dùng đã có tài khoản trong hệ thống.
Hậu điều kiện	Phiên làm việc được khởi tạo; người dùng có quyền truy cập các chức năng tương ứng với vai trò của mình.
Luồng sự kiện chính	1. Người dùng truy cập trang đăng nhập và nhập email cùng mật khẩu. 2. Hệ thống kiểm tra trạng thái khóa tài khoản; nếu đang bị khóa, trả về thông báo kèm thời gian còn lại và dừng xử lý. 3. Hệ thống xác thực thông tin đăng nhập. 4. Hệ thống khởi tạo phiên làm việc và cấp quyền truy cập. 5. Người dùng được chuyển đến trang chủ tương ứng với vai trò của mình.
Luồng thay thế	<i>1a. Đăng nhập qua Google:</i> Tại bước 1, người dùng chọn đăng nhập bằng Google; hệ thống chuyển hướng đến màn hình xác nhận của Google. Sau khi người dùng xác nhận, hệ thống tiếp tục từ bước 4.
Luồng ngoại lệ	<i>3a. Sai thông tin đăng nhập:</i> Hệ thống ghi nhận lần đăng nhập thất bại; sau năm lần sai liên tiếp trong vòng năm phút, tài khoản bị khóa tự động trong mười lăm phút. Hệ thống không tiết lộ trường nào sai để tránh tấn công dò tìm tài khoản.

Bảng 2.1: Đặc tả use case UC-01 — Đăng nhập hệ thống

2.3.2 Đặc tả use case: Đăng ký tài khoản

Bảng 2.2 đặc tả chi tiết use case đăng ký tài khoản người dùng mới, với quy trình xác thực email bắt buộc trước khi tài khoản được kích hoạt.

Tên use case	Đăng ký tài khoản
Mã use case	UC-02
Tác nhân	Người dùng chưa có tài khoản
Mô tả	Người dùng tạo tài khoản mới bằng email/mật khẩu và xác thực địa chỉ email để kích hoạt tài khoản.
Tiền điều kiện	Địa chỉ email chưa được đăng ký trong hệ thống.
Hậu điều kiện	Tài khoản được tạo và kích hoạt với vai trò User; người dùng có thể đăng nhập vào hệ thống.
Luồng sự kiện chính	1. Người dùng truy cập trang đăng ký và nhập email cùng mật khẩu (tối thiểu tám ký tự, bao gồm chữ hoa và chữ số). 2. Hệ thống kiểm tra giới hạn đăng ký; nếu vượt quá số lần cho phép trong khoảng thời gian quy định, yêu cầu bị từ chối. 3. Hệ thống kiểm tra email chưa tồn tại và tạo tài khoản ở trạng thái chờ xác thực. 4. Hệ thống gửi email chứa liên kết xác thực có hiệu lực hai mươi bốn giờ. 5. Người dùng mở email và nhấn liên kết xác thực. 6. Hệ thống kích hoạt tài khoản và chuyển hướng người dùng đến trang đăng nhập.
Luồng thay thế	<i>1a. Đăng ký qua Google:</i> Tại bước 1, người dùng chọn đăng ký bằng Google; hệ thống nhận thông tin từ Google, tạo tài khoản đã xác thực và chuyển thẳng đến bước 6.
Luồng ngoại lệ	<i>3a. Email đã tồn tại:</i> Hệ thống thông báo lỗi và gợi ý người dùng đăng nhập hoặc sử dụng chức năng quên mật khẩu. <i>5a. Liên kết xác thực hết hạn:</i> Hệ thống thông báo liên kết không còn hiệu lực và cho phép người dùng yêu cầu gửi lại email xác thực.

Bảng 2.2: Đặc tả use case UC-02 — Đăng ký tài khoản

2.3.3 Đặc tả use case: Tìm kiếm sự kiện

Bảng 2.3 đặc tả chi tiết use case tìm kiếm và khám phá sự kiện, bao gồm tìm kiếm theo từ khóa, lọc theo nhiều tiêu chí và xem gợi ý cá nhân hóa.

Tên use case	Tìm kiếm sự kiện
Mã use case	UC-03
Tác nhân	Người dùng (bao gồm cả người chưa đăng nhập)
Mô tả	Người dùng tìm kiếm và khám phá sự kiện thông qua từ khóa hoặc bộ lọc; hệ thống trả về danh sách kết quả phù hợp và đề xuất gợi ý cá nhân hóa với người dùng đã đăng nhập.
Tiền điều kiện	Không có. Chức năng khả dụng cho cả người dùng chưa đăng nhập.
Hậu điều kiện	Danh sách sự kiện phù hợp được hiển thị; địa chỉ URL phản ánh tiêu chí tìm kiếm hiện tại để người dùng có thể chia sẻ.
Luồng sự kiện chính	1. Người dùng truy cập trang danh sách sự kiện. 2. Người dùng nhập từ khóa hoặc thiết lập bộ lọc theo thể loại, thành phố, khoảng thời gian và khoảng giá. 3. Hệ thống trả về danh sách sự kiện phù hợp, sắp xếp theo ngày gần nhất và độ nổi bật, phân trang mỗi mười hai sự kiện. 4. Hệ thống cập nhật địa chỉ URL theo tiêu chí tìm kiếm hiện tại. 5. Người dùng nhấn vào một sự kiện để xem trang chi tiết.
Luồng thay thế	<i>1a. Người dùng đã đăng nhập:</i> Tại bước 1, ngoài danh sách chung, hệ thống hiển thị thêm danh sách sự kiện được gợi ý dựa trên lịch sử tương tác và sở thích cá nhân của người dùng.
Luồng ngoại lệ	<i>3a. Không có kết quả:</i> Hệ thống hiển thị thông báo không tìm thấy sự kiện phù hợp và đề xuất danh sách sự kiện nổi bật thay thế.

Bảng 2.3: Đặc tả use case UC-03 — Tìm kiếm sự kiện

2.3.4 Đặc tả use case: Đặt vé và thanh toán

Bảng 2.4 đặc tả chi tiết use case đặt vé và thanh toán. Quy trình tổng thể đã được minh họa trực quan tại Hình 2.5 và Hình 2.6; phần này tập trung vào các điều kiện, luồng thay thế và ngoại lệ chưa được thể hiện đầy đủ trong biểu đồ hoạt động.

Tên use case	Đặt vé và thanh toán
Mã use case	UC-04
Tác nhân	User (đã đăng nhập)
Mô tả	Người dùng chọn vé, giữ chỗ và hoàn tất thanh toán để nhận vé điện tử.
Tiền điều kiện	Người dùng đã đăng nhập; sự kiện đang trong thời gian mở bán; còn vé trong kho.
Hậu điều kiện	Vé điện tử được phát hành và gửi đến người dùng qua email; số lượng tồn kho được cập nhật; doanh thu được ghi nhận.
Luồng sự kiện chính	1. Người dùng xem trang chi tiết sự kiện, chọn loại vé và số lượng. 2. Hệ thống kiểm tra tồn kho, giữ chỗ trong mười phút và trả về mã QR kèm thông tin chuyển khoản. 3. Người dùng quét mã QR bằng ứng dụng ngân hàng và thực hiện chuyển khoản với nội dung chứa mã đặt vé. 4. Cổng thanh toán xác nhận giao dịch; hệ thống tạo vé điện tử. 5. Hệ thống thông báo kết quả đến người dùng và gửi vé điện tử qua email.
Luồng thay thế	<i>1a. Sự kiện có sơ đồ chỗ ngồi:</i> Tại bước 1, người dùng chọn khu vực trực tiếp trên sơ đồ chỗ ngồi; lựa chọn đó xác định loại vé tương ứng. <i>1b. Hàng chờ ảo đang hoạt động:</i> Sau bước 1, nếu lưu lượng vượt ngưỡng, người dùng được xếp vào hàng chờ và nhận số thứ tự; khi đến lượt, hệ thống tiếp tục từ bước 2.
Luồng ngoại lệ	<i>2a. Hết vé:</i> Hệ thống thông báo ngay lập tức và không tạo đơn hàng. <i>3a. Hết mười phút chưa thanh toán:</i> Hệ thống hủy giữ chỗ, trả vé về kho và thông báo cho người dùng. <i>3b. Sai nội dung chuyển khoản:</i> Giao dịch không được khớp với đơn hàng; đơn hàng hết hạn sau mười phút; người dùng liên hệ bộ phận hỗ trợ để được xử lý thủ công.

Bảng 2.4: Đặc tả use case UC-04 — Đặt vé và thanh toán

2.3.5 Đặc tả use case: Tạo và quản lý sự kiện

Bảng 2.5 đặc tả chi tiết use case tạo và quản lý sự kiện. Quy trình xét duyệt giữa Organizer và Admin đã được minh họa tại Hình 2.8.

Tên use case	Tạo và quản lý sự kiện
Mã use case	UC-05
Tác nhân	Organizer
Mô tả	Organizer tạo sự kiện, cấu hình vé và tài liệu pháp lý, gửi lên để Admin xét duyệt; sau khi được phê duyệt, sự kiện bắt đầu nhận đặt vé.
Tiền điều kiện	Tài khoản đã đăng nhập với vai trò Organizer, đã hoàn tất KYC/KYB và đã cấu hình tài khoản ngân hàng.
Hậu điều kiện	Sự kiện được lưu ở trạng thái nháp hoặc chuyển sang chờ xét duyệt; sau khi Admin phê duyệt, sự kiện ở trạng thái Upcoming và bắt đầu nhận đặt vé.
Luồng sự kiện chính	1. Organizer nhập thông tin sự kiện: tên, mô tả, thể loại, địa điểm, thời gian và hình ảnh banner. 2. Organizer cấu hình các loại vé (tên, giá, số lượng, thời gian mở bán) và đính kèm tài liệu pháp lý. 3. Organizer gửi sự kiện lên để Admin xét duyệt. 4. Admin phê duyệt (theo UC-08); sự kiện chuyển sang Upcoming và bắt đầu nhận đặt vé.
Luồng thay thế	<i>2a. Sự kiện có sơ đồ chỗ ngồi:</i> Organizer thiết lập sơ đồ chỗ ngồi bằng công cụ kéo thả và gắn mỗi khu vực với loại vé tương ứng. <i>3a. Lưu nháp:</i> Organizer lưu nháp để tiếp tục chỉnh sửa sau thay vì gửi ngay. <i>4b. Chỉnh sửa sau phê duyệt:</i> Mô tả và ảnh được cập nhật trực tiếp; thay đổi giá, ngày giờ hoặc địa điểm cần gửi Admin xét duyệt lại.
Luồng ngoại lệ	<i>3b. Dữ liệu không hợp lệ:</i> Hệ thống báo lỗi và giữ nguyên dữ liệu đã nhập. <i>4a. Admin yêu cầu chỉnh sửa:</i> Organizer nhận danh sách mục cần bổ sung; có năm ngày để phản hồi, quá hạn thì sự kiện tự động hủy.

Bảng 2.5: Đặc tả use case UC-05 — Tạo và quản lý sự kiện

2.3.6 Đặc tả use case: Check-in vé bằng mã QR

Bảng 2.6 đặc tả chi tiết use case check-in vé tại cổng sự kiện.

Tên use case	Check-in vé bằng mã QR
Mã use case	UC-06
Tác nhân	Organizer, CTV
Mô tả	Organizer hoặc CTV quét mã QR trên vé của khách tham dự để xác thực và đánh dấu vào cổng.
Tiền điều kiện	Tài khoản đã đăng nhập với vai trò Organizer hoặc CTV; sự kiện đang trong thời gian diễn ra; CTV phải được Organizer liên kết với sự kiện được chỉ định trước khi thực hiện.
Hậu điều kiện	Vé được đánh dấu đã sử dụng kèm thời điểm check-in; khách tham dự được cho vào.
Luồng sự kiện chính	1. Organizer hoặc CTV mở chức năng check-in và kích hoạt camera. 2. Người quét hướng camera vào mã QR trên vé của khách. 3. Hệ thống đọc mã QR và kiểm tra tính hợp lệ: vé tồn tại trong hệ thống, thuộc đúng sự kiện được chỉ định, đã thanh toán và chưa được sử dụng. 4. Hệ thống đánh dấu vé đã sử dụng, ghi nhận thời điểm check-in và hiển thị thông tin xác nhận gồm tên người tham dự và hạng vé.
Luồng thay thế	Không có.
Luồng ngoại lệ	<i>2a. Không đọc được mã QR:</i> Hệ thống thông báo không thể nhận diện mã; người quét điều chỉnh góc camera hoặc yêu cầu khách hiển thị lại mã. <i>3a. Vé đã được sử dụng trước đó:</i> Hệ thống hiển thị cảnh báo kèm thời điểm check-in trước đó và ghi nhận sự cố để báo cáo Organizer; khách bị từ chối vào. <i>3b. Vé không hợp lệ:</i> Hệ thống hiển thị lý do cụ thể — sai sự kiện, chưa thanh toán hoặc không tồn tại — và từ chối cho vào. <i>3c. Mất kết nối mạng:</i> Hệ thống thông báo không thể xác thực và yêu cầu kiểm tra kết nối trước khi thử lại.

Bảng 2.6: Đặc tả use case UC-06 — Check-in vé bằng mã QR

2.3.7 Đặc tả use case: Phê duyệt nhà tổ chức (KYC/KYB)

Bảng 2.7 đặc tả chi tiết use case phê duyệt hồ sơ nhà tổ chức. Quy trình xác minh danh tính đã được minh họa tại Hình 2.7.

Tên use case	Phê duyệt nhà tổ chức (KYC/KYB)
Mã use case	UC-07
Tác nhân	Admin
Mô tả	Admin xem xét hồ sơ đăng ký của nhà tổ chức, thực hiện kiểm tra pháp lý và ra quyết định cấp hoặc từ chối quyền tổ chức sự kiện.
Tiền điều kiện	Admin đã đăng nhập; tồn tại hồ sơ nhà tổ chức đang ở trạng thái chờ xác minh danh tính.
Hậu điều kiện	Hồ sơ được phê duyệt — tài khoản được cấp quyền Organizer và nhận thông báo — hoặc bị từ chối kèm lý do cụ thể; mọi thao tác được ghi vào nhật ký kiểm toán.
Luồng sự kiện chính	1. Admin truy cập danh sách hồ sơ đang chờ, được sắp xếp theo thứ tự nộp. 2. Admin mở hồ sơ, xác minh tài liệu pháp lý, thông tin cá nhân và tài khoản ngân hàng theo loại tổ chức. 3. Admin kiểm tra lịch sử vi phạm của tài khoản. 4. Admin phê duyệt hồ sơ. 5. Hệ thống cấp quyền Organizer, gửi thông báo chào mừng kèm hướng dẫn sử dụng cho nhà tổ chức và ghi nhật ký kiểm toán.
Luồng thay thế	<i>2a. Tổ chức cá nhân:</i> Tại bước 2, Admin kiểm tra CCCD rõ nét còn hiệu lực, tên khớp giấy tờ và xác minh tài khoản ngân hàng đứng tên chủ CCCD. <i>2b. Tổ chức doanh nghiệp:</i> Tại bước 2, Admin kiểm tra thêm Giấy đăng ký doanh nghiệp, mã số thuế và xác nhận người đại diện pháp luật khớp giấy tờ. <i>4a. Từ chối:</i> Thay vì phê duyệt ở bước 4, Admin từ chối kèm lý do cụ thể; hệ thống thông báo cho nhà tổ chức và cho bảy ngày để bổ sung hồ sơ, quá hạn thì hồ sơ tự động hủy.
Luồng ngoại lệ	<i>Quá thời hạn xử lý:</i> Nếu hồ sơ chưa được xử lý sau hai ngày làm việc kể từ ngày nộp, hệ thống kích hoạt cảnh báo cho nhóm Admin.

Bảng 2.7: Đặc tả use case UC-07 — Phê duyệt nhà tổ chức (KYC/KYB)

2.3.8 Đặc tả use case: Phê duyệt sự kiện

Bảng 2.8 đặc tả chi tiết use case phê duyệt sự kiện theo ba cấp độ quy mô. Quy trình tổng thể đã được minh họa tại Hình 2.8.

Tên use case	Phê duyệt sự kiện
Mã use case	UC-08
Tác nhân	Admin
Mô tả	Admin xem xét hồ sơ sự kiện theo checklist tương ứng với quy mô, xác định mức phí sàn và ra quyết định phê duyệt, yêu cầu chỉnh sửa hoặc từ chối.
Tiền điều kiện	Admin đã đăng nhập; tồn tại sự kiện ở trạng thái chờ xét duyệt do Organizer đã hoàn tất xác minh danh tính gửi lên.
Hậu điều kiện	Sự kiện chuyển sang trạng thái Upcoming và bắt đầu nhận đặt vé; mức phí sàn được xác định và ghi vào hợp đồng điện tử; Organizer nhận thông báo kết quả.
Luồng sự kiện chính	1. Admin mở hồ sơ sự kiện chờ duyệt; hệ thống xác định cấp độ theo quy mô (Cấp 1: dưới 100 khách, Cấp 2: 100–1.000 khách, Cấp 3: trên 1.000 khách). 2. Admin thực hiện checklist kiểm tra tương ứng với cấp độ. 3. Admin xác định mức phí sàn và ghi nhận vào hợp đồng điện tử. 4. Admin phê duyệt; hệ thống chuyển sự kiện sang Published và thông báo cho Organizer.
Luồng thay thế	<i>2a. Cấp 1:</i> Kiểm tra KYC nhà tổ chức, nội dung mô tả, địa điểm thực tế và cấu hình vé hợp lý. <i>2b. Cấp 2:</i> Bổ sung giấy phép biểu diễn, tư cách pháp nhân và hợp đồng địa điểm có chữ ký. <i>2c. Cấp 3:</i> Bổ sung giấy phép PCCC, bảo hiểm sự kiện, phương án an ninh và năng lực tài chính. <i>4a. Yêu cầu chỉnh sửa:</i> Admin gửi danh sách mục cần bổ sung; Organizer có năm ngày phản hồi, quá hạn sự kiện tự hủy.
Luồng ngoại lệ	<i>4b. Từ chối vĩnh viễn:</i> Áp dụng khi sự kiện vi phạm pháp luật; hệ thống ghi cảnh báo vào hồ sơ nhà tổ chức; vi phạm lần hai dẫn đến khóa tài khoản.

Bảng 2.8: Đặc tả use case UC-08 — Phê duyệt sự kiện

2.4 Yêu cầu phi chức năng

Về hiệu năng và khả năng mở rộng, hệ thống phải đảm bảo thời gian phản hồi cho các thao tác thông thường như tìm kiếm và xem chi tiết sự kiện dưới 500 ms dưới tải trọng cao. Đặc biệt, thao tác kiểm tra tồn kho và giữ chỗ vé phải được thực thi chính xác kể cả với quy mô lớn

Về độ tin cậy và tính nhất quán dữ liệu, mọi giao dịch tài chính bao gồm tạo vé và ghi nhận thanh toán phải được thực hiện trong phạm vi một giao dịch cơ sở dữ liệu duy nhất để bảo đảm tính toàn vẹn theo bốn thuộc tính nguyên tử, nhất quán, cô lập và bền vững.

Về bảo mật, hệ thống áp dụng kiểm soát truy cập theo vai trò với năm vai trò gồm User, CTV, Organizer, Admin và System, trong đó mỗi vai trò chỉ được phép truy cập đúng tập chức năng được phân công. Thông báo từ cổng thanh toán SePay được xác thực bằng API key trong tiêu đề `Authorization` trước khi xử lý nhằm ngăn chặn giả mạo webhook. Mã QR vé chỉ chứa định danh được mã hóa và chỉ có thể xác thực phía máy chủ, không thể làm giả phía người dùng.

Hệ thống triển khai giới hạn tần suất đa tầng theo định danh người dùng hoặc địa chỉ IP, với bốn mức độ riêng biệt tương ứng với toàn bộ API, các điểm cuối xác thực và đặt vé, ngăn chặn đột biến tức thời, và riêng cho chức năng đăng ký tài khoản. Để ngăn tài khoản bị dò mật khẩu, sau năm lần đăng nhập sai liên tiếp trong vòng năm phút, tài khoản bị khóa tự động mười lăm phút và phản hồi không tiết lộ trường nào sai.

Về tính khả dụng của giao diện, giao diện người dùng phải có thiết kế thích ứng và hoạt động mượt mà trên các thiết bị máy tính để bàn, máy tính bảng và điện thoại di động.

Chương 2 đã xác định đầy đủ các yêu cầu chức năng thông qua biểu đồ use case, hai quy trình nghiệp vụ trung tâm và đặc tả chi tiết bảy use case quan trọng nhất, cùng các yêu cầu phi chức năng về hiệu năng, độ tin cậy, bảo mật, giao diện và triển khai. Các yêu cầu này là cơ sở trực tiếp cho việc lựa chọn công nghệ trong Chương 3 và thiết kế kiến trúc hệ thống trong Chương 4.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Dựa trên kết quả phân tích yêu cầu từ Chương 2, chương này trình bày nền tảng lý thuyết và căn cứ lựa chọn công nghệ cho từng thành phần của hệ thống. Với mỗi công nghệ, tác giả trình bày bài toán cụ thể mà công nghệ đó giải quyết, cơ chế hoạt động cốt lõi, lý do lựa chọn so với các giải pháp thay thế và cách áp dụng trong dự án.

3.1 Tổng quan các lựa chọn công nghệ

Hệ thống đặt vé sự kiện trực tuyến đặt ra bốn nhóm thách thức kỹ thuật chính: (i) xử lý hàng nghìn yêu cầu đặt vé đồng thời mà không xảy ra tình trạng bán quá số lượng; (ii) truyền tải trạng thái chỗ ngồi và kết quả thanh toán tới người dùng theo thời gian thực; (iii) cá nhân hoá trải nghiệm khám phá sự kiện theo sở thích từng người dùng dựa trên ngữ nghĩa tiếng Việt; (iv) triển khai và vận hành ổn định trên hạ tầng tối giản chi phí. Mỗi nhóm thách thức đòi hỏi một tập công nghệ chuyên biệt: xử lý đặt vé đồng thời cần cơ sở dữ liệu tuân thủ ACID và cơ chế đồng bộ hóa nguyên tử; truyền tải thời gian thực cần giao thức kết nối hai chiều bền vững; gợi ý ngữ nghĩa tiếng Việt cần mô hình ngôn ngữ và hạ tầng tìm kiếm véc-tơ tích hợp; triển khai tối giản chi phí cần công cụ đóng gói nhất quán trên một máy chủ duy nhất. Quá trình lựa chọn cân nhắc đồng thời ba ràng buộc: tính trưởng thành của hệ sinh thái trong môi trường sản xuất, khả năng tích hợp liên thành phần mà không phát sinh chi phí chuyển đổi cao, và mức độ phù hợp với mục tiêu vận hành trên hạ tầng một máy chủ riêng ảo. Các phần tiếp theo của chương lần lượt phân tích chi tiết từng nhóm: Phần 3.2 trình bày nhóm xử lý nghiệp vụ, Phần 3.3 nhóm dữ liệu và hệ gợi ý, Phần 3.4 giao tiếp thời gian thực và xác thực, Phần 3.5 giao diện người dùng, và Phần 3.6 tích hợp dịch vụ cùng triển khai. Bảng 3.1 tổng hợp các công nghệ được lựa chọn theo từng nhóm chức năng, bài toán giải quyết, lý do lựa chọn và phương án thay thế đã xem xét.

Công nghệ	Bài toán giải quyết	Lý do lựa chọn	Thay thế xem xét
<i>Nhóm xử lý nghiệp vụ</i>			
NestJS v11	Tổ chức mã nguồn máy chủ theo miền nghiệp vụ	Module hóa, tiềm phụ thuộc, tích hợp sẵn TypeORM và Socket.IO	Express.js, Fastify
TypeScript	Kiểm tra kiểu tĩnh cho JavaScript	Phát hiện lỗi tại biên dịch, làm rõ giao kèo giữa các lớp	JavaScript thuần
<i>Nhóm dữ liệu</i>			
PostgreSQL 15	Lưu trữ toàn bộ dữ liệu nghiệp vụ	ACID đầy đủ, hỗ trợ JSONB và phần mở rộng pgvector	MySQL, MongoDB
TypeORM v0.3	Ánh xạ đối tượng-quan hệ	Migration tự động, gói thao tác vào giao dịch	Prisma, Sequelize
pgvector	Tìm kiếm láng giềng gần nhất theo véc-tơ nhúng	Tích hợp trực tiếp PostgreSQL, chỉ mục HNSW	Qdrant, Weaviate
Redis 7	Bộ nhớ đệm và đồng bộ hóa phân tán	Tập lệnh Lua nguyên tử, adapter Socket.IO	Memcached
<i>Giao tiếp thời gian thực và Xác thực</i>			
RabbitMQ	Xử lý bất đồng bộ và hẹn giờ trả vé	Plugin tin nhắn trễ, dead-letter queue, giao thức AMQP	Kafka, AWS SQS
Socket.IO	Đẩy cập nhật trạng thái tới trình duyệt	Namespace, room, Redis adapter hỗ trợ đa phiên bản	WebSocket thuần
JWT	Xác thực phi trạng thái	Chuẩn mở RFC 7519, không cần lưu phiên trên máy chủ	Session cookie
<i>Nhóm giao diện người dùng</i>			
React 18	Xây dựng giao diện dạng component	Concurrent Rendering, hệ sinh thái phong phú	Vue.js, Angular
Konva.js	Sơ đồ chỗ ngồi tương tác dạng canvas	Binding React sẵn có, hiệu năng cao với nhiều đối tượng	Fabric.js, SVG thuần
<i>Tích hợp dịch vụ và Triển khai</i>			
SePay	Nhận thanh toán chuyển khoản nội địa	Hỗ trợ VietQR, xác thực webhook bằng API key, miễn phí giao dịch	PayOS, VNPay
Cloudinary	Lưu trữ và phân phối tệp phương tiện	CDN toàn cầu, tự động tối ưu định dạng theo thiết bị	AWS S3
Docker Compose	Đóng gói và điều phối dịch vụ Backend	Môi trường nhất quán, triển khai một lệnh trên VPS	Kubernetes
Vercel	Hosting Frontend tĩnh	Tích hợp CI/CD tự động, mạng phân phối nội dung edge toàn cầu	Netlify

Bảng 3.1: Tổng quan lựa chọn công nghệ theo nhóm chức năng

3.2 Nhóm xử lý nghiệp vụ

3.2.1 NestJS và TypeScript

Bài toán cốt lõi của tầng máy chủ là tổ chức mã nguồn sao cho mỗi miền nghiệp vụ như quản lý sự kiện, đặt chỗ hay thanh toán đều có ranh giới rõ ràng, dễ kiểm

thử độc lập và dễ mở rộng khi bổ sung tính năng mới. NestJS v11 là framework Node.js [1] áp dụng kiến trúc module hóa và tiêm phụ thuộc theo phong cách Angular, cho phép tổ chức mã nguồn thành các module độc lập như EventsModule, BookingsModule và PaymentsModule [2]. Framework tích hợp sẵn TypeORM, Passport.js, Socket.IO gateway và thư viện kết nối RabbitMQ, giúp liên kết các thành phần hạ tầng mà không cần cấu hình thủ công từng thư viện riêng lẻ. So với Express.js, NestJS áp đặt cấu trúc rõ ràng hơn khi dự án mở rộng quy mô, đồng thời tích hợp sẵn bộ kiểm thử phù hợp với yêu cầu kiểm thử đơn vị từng module.

TypeScript [3] được dùng xuyên suốt toàn bộ mã nguồn để phát hiện lỗi kiểu dữ liệu ngay tại bước biên dịch thay vì lúc chạy thực. Kiểu tĩnh làm rõ giao kèo giữa các lớp: kiểu dữ liệu trả về của một phương thức dịch vụ khớp chính xác với kiểu mà lớp điều khiển kỳ vọng, giúp phát hiện sớm các lỗi không tương thích trước khi triển khai lên môi trường sản xuất.

3.3 Nhóm dữ liệu

3.3.1 PostgreSQL và TypeORM

Dữ liệu nghiệp vụ của hệ thống bao gồm nhiều thực thể có quan hệ chặt chẽ như người dùng, sự kiện, đơn hàng và vé, đòi hỏi bảo đảm toàn vẹn tham chiếu và hỗ trợ giao dịch đa bảng. PostgreSQL 15 [4] là hệ quản trị cơ sở dữ liệu quan hệ tuân thủ đầy đủ tiêu chuẩn tính nguyên tử, nhất quán, cô lập và bền vững (Atomicity, Consistency, Isolation, Durability — ACID), được lựa chọn vì ngoài khả năng quan hệ truyền thống còn hỗ trợ kiểu JSONB để lưu dữ liệu bán cấu trúc như sơ đồ chỗ ngồi theo khu vực, và tích hợp phần mở rộng pgvector để lưu trữ và truy vấn véc-tơ nhúng ngay trong cùng một cơ sở dữ liệu [5]. MySQL được cân nhắc nhưng loại bỏ vì thiếu hỗ trợ native cho pgvector; MongoDB loại bỏ vì hệ thống yêu cầu toàn vẹn tham chiếu chặt chẽ giữa đơn hàng và vé.

TypeORM v0.3 [6] đóng vai trò thư viện ánh xạ đối tượng–quan hệ (ORM), cung cấp hai chức năng chính trong dự án: quản lý phiên bản lược đồ thông qua migration tự động và gói nhiều thao tác vào một giao dịch duy nhất để đảm bảo tính nguyên tử. So với Prisma hay Sequelize, TypeORM tương thích tốt hơn với hệ thống kiểu TypeScript phức tạp và hỗ trợ decorator khai báo trực tiếp trên lớp thực thể.

3.3.2 Redis

Khi hàng nghìn yêu cầu đặt vé đến đồng thời cho một sự kiện, các thao tác đọc–kiểm tra–ghi liên tiếp trên cơ sở dữ liệu quan hệ có thể dẫn đến xung đột tương tranh khiến nhiều yêu cầu cùng đọc được số lượng vé còn lại hợp lệ trước khi bắt kỳ yêu cầu nào thực hiện ghi. Redis 7 [7] là kho dữ liệu trong bộ nhớ hỗ trợ thực

thi tập lệnh Lua nguyên tử: toàn bộ chuỗi lệnh kiểm tra số lượng còn lại và trừ kho được thực hiện trong một thao tác không thể bị gián đoạn, giải quyết xung đột tương tranh (race condition) mà không cần khóa phân tán công kênh [5].

Ngoài vai trò đồng bộ hóa, Redis đảm nhiệm ba chức năng bổ sung trong hệ thống: (i) lưu trạng thái hàng chờ ảo gồm vị trí xếp hàng và số lượng slot đang hoạt động; (ii) lưu vào bộ nhớ đệm (cache) kết quả các truy vấn tốn kém để giảm tải cho PostgreSQL; (iii) làm adapter cho Socket.IO để đồng bộ sự kiện thời gian thực khi hệ thống chạy nhiều phiên bản máy chủ song song. So với Memcached, Redis được lựa chọn vì hỗ trợ cấu trúc dữ liệu phong phú hơn gồm danh sách, tập hợp có thứ tự, hash và cơ chế xuất bản–đăng ký (publish/subscribe) cần thiết cho đồng bộ Socket.IO.

3.3.3 Hệ gợi ý ngữ nghĩa: Vietnamese-SBERT và pgvector

Hệ gợi ý sự kiện cần hiểu ngữ nghĩa tiếng Việt để đề xuất sự kiện phù hợp sở thích người dùng, thay vì chỉ lọc theo từ khóa hoặc danh mục cứng nhắc. Mô hình Vietnamese-SBERT ([keepitreal/vietnamese-sbert](https://github.com/keepitreal/vietnamese-sbert)) [8] áp dụng kiến trúc Sentence-BERT để chuyển tên và mô tả sự kiện thành véc-tơ nhúng 768 chiều nắm bắt ngữ nghĩa tiếng Việt thông qua kiến trúc mã hóa hai chiều. Véc-tơ 768 chiều này được ghép thêm 40 chiều dữ liệu có cấu trúc gồm loại sự kiện, tỉnh thành và mức giá chuẩn hóa, tạo thành véc-tơ 808 chiều đại diện đầy đủ cho mỗi sự kiện.

pgvector [9] với chỉ mục đồ thị dẫn hướng thứ bậc (HNSW) [10] cho phép tìm kiếm láng giềng gần nhất xấp xỉ theo độ tương đồng cosin trực tiếp trong PostgreSQL mà không cần triển khai thêm hệ thống vector database riêng biệt. Hệ thống áp dụng ba chiến lược gợi ý: (i) theo nội dung tương tự giữa các sự kiện dựa trên khoảng cách cosin giữa các véc-tơ nhúng; (ii) theo hồ sơ véc-tơ của người dùng được xây dựng từ trung bình có trọng số các sự kiện đã đặt vé; (iii) theo độ phổ biến làm phương án dự phòng (fallback) khi người dùng chưa có lịch sử tương tác. Việc tích hợp pgvector vào PostgreSQL hiện có loại bỏ nhu cầu vận hành thêm hệ thống như Qdrant hay Weaviate, giảm phức tạp hạ tầng và chi phí đồng bộ dữ liệu giữa các hệ thống.

3.4 Giao tiếp thời gian thực và Xác thực

3.4.1 Hàng đợi tin nhắn: RabbitMQ

Các tác vụ nền như gửi thư điện tử xác nhận, nhả vé hết hạn giữ chỗ hay cập nhật hồ sơ gợi ý cần được xử lý bất đồng bộ để không chặn luồng phục vụ yêu cầu trực tiếp từ người dùng, đồng thời cần cơ chế hẹn giờ chính xác để nhả vé đúng thời điểm mà không cần kiểm tra định kỳ liên tục. RabbitMQ [11] là trình môi giới tin

nhấn hỗ trợ giao thức hàng đợi tin nhắn nâng cao (AMQP), với plugin tin nhắn trễ cho phép lên lịch gửi tin nhắn sau khoảng thời gian tùy chỉnh tính theo mili-giây.

Hệ thống định nghĩa năm exchange chuyên biệt: (i) `booking-exchange` giải phóng vé sau mười phút giữ chỗ; (ii) `waiting-room-exchange` xử lý toàn bộ vòng đời hàng chờ ảo; (iii) `recommendation-exchange` kích hoạt tính toán véc-tơ nhúng và cập nhật hồ sơ gợi ý bất đồng bộ; (iv) `delayed-exchange` kiểu `x-delayed-message` định tuyến tin nhắn theo thời gian trễ tùy chỉnh; (v) `dlq-exchange` tiếp nhận tin nhắn thất bại sau nhiều lần thử lại để quản trị viên xử lý thủ công. So với Apache Kafka, RabbitMQ phù hợp hơn cho bài toán này vì cơ chế xử lý phù hợp hơn với hệ thống, không quá phức tạp nhưng vẫn đáp ứng được yêu cầu cần thiết

3.4.2 Giao tiếp hai chiều: Socket.IO

Luồng đặt vé yêu cầu máy chủ đẩy cập nhật trạng thái thanh toán và chỗ ngồi tới trình duyệt của người dùng ngay khi có thay đổi, thay vì chờ trình duyệt chủ động gửi yêu cầu mới. Socket.IO [12] là thư viện cung cấp kết nối hai chiều bền vững dựa trên giao thức WebSocket [13], với cơ chế tự động chuyển về thăm dò định kỳ khi WebSocket không khả dụng trong một số môi trường mạng. Thư viện tổ chức kết nối theo namespace như `/payment-events` và `/waiting-room`, và theo room dựa trên mã đơn hàng hoặc mã người dùng, cho phép phát sự kiện có chủ đích đến đúng máy khách thay vì phát rộng toàn bộ kết nối.

Redis adapter của Socket.IO cho phép mọi phiên bản máy chủ trong cụm đều có thể phát sự kiện và đảm bảo sự kiện đến đúng máy khách đích bất kể máy khách đang kết nối đến phiên bản nào, hỗ trợ mở rộng hệ thống theo chiều ngang mà không làm mất tính nhất quán trong phân phối sự kiện.

3.4.3 Xác thực và phân quyền: JWT và Passport.js

Hệ thống có năm vai trò người dùng là Khách hàng, Cộng tác viên, Nhà tổ chức, Quản trị viên và Hệ thống, đòi hỏi cơ chế kiểm soát truy cập theo vai trò (Role-Based Access Control — RBAC) [14] có thể mở rộng ngang và kiểm tra quyền tại từng điểm cuối API theo kiến trúc REST [15]. Thẻ xác thực JSON (JWT) [16] cho phép nhúng mã người dùng và vai trò vào token và xác thực bằng chữ ký số mà không cần lưu phiên làm việc trên máy chủ, phù hợp với mô hình triển khai nhiều phiên bản. Token truy cập có thời hạn mười lăm phút; token làm mới có thời hạn ba mươi ngày được lưu trong cookie `httpOnly` để ngăn mã chèn trang web (XSS) đọc được.

Passport.js tích hợp đồng nhất ba phương thức xác thực: (i) đăng nhập bằng email và mật khẩu; (ii) xác thực JWT; (iii) đăng nhập qua Google OAuth2. Cloud-

flare Turnstile được tích hợp tại các điểm nhạy cảm như đăng ký tài khoản và đặt vé để ngăn thao tác hàng loạt bằng bot tự động. So với cơ chế phiên làm việc truyền thống, JWT phù hợp hơn cho hệ thống có nhiều phiên bản máy chủ vì không cần chia sẻ bộ nhớ phiên giữa các phiên bản.

3.5 Nhóm giao diện người dùng

3.5.1 React 18 và Vite

Giao diện hệ thống bao gồm nhiều luồng tương tác phức tạp như sơ đồ chỗ ngồi thời gian thực, hàng chờ ảo và biểu mẫu đa bước, đặt ra yêu cầu cao về hiệu năng kết xuất và khả năng tái sử dụng thành phần. React 18 [17] là thư viện giao diện dựa trên component cho phép xây dựng giao diện phức tạp từ các thành phần độc lập và tái sử dụng được. Tính năng Concurrent Rendering của React 18 giúp giao diện không bị chặn khi xử lý các tác vụ nặng như cập nhật đồng thời trạng thái hàng nghìn chỗ ngồi. Vite đóng vai trò công cụ build, sử dụng ES Modules gốc trong môi trường phát triển để đạt tốc độ khởi động tức thì và cập nhật module nóng phản hồi dưới 50 ms, rút ngắn đáng kể vòng lặp phát triển so với webpack.

3.5.2 Quản lý trạng thái và dữ liệu

TanStack Query v4 quản lý dữ liệu từ máy chủ với bộ nhớ đệm tự động, tự động làm mới khi dữ liệu hết hạn và cập nhật lạc quan, giúp giao diện luôn phản ánh trạng thái mới nhất mà không cần viết logic đồng bộ thủ công. Zustand quản lý trạng thái cục bộ phía máy khách với giao diện lập trình tối giản và hỗ trợ lưu tự động vào bộ nhớ cục bộ của trình duyệt, phù hợp để duy trì trạng thái giỏ hàng và thông tin phiên giữa các lần điều hướng trang. React Hook Form kết hợp Zod xử lý kiểm tra biểu mẫu với hiệu năng cao nhờ không kiểm soát trực tiếp từng trường nhập liệu, giúp giảm số lần kết xuất lại giao diện khi người dùng nhập liệu.

3.5.3 Sơ đồ chỗ ngồi tương tác: Konva.js

Nhà tổ chức sự kiện cần một công cụ thiết kế sơ đồ chỗ ngồi trực quan, cho phép vẽ và chỉnh sửa khu vực bằng thao tác kéo thả (drag-and-drop) trực tiếp trên trình duyệt với hiệu năng đủ mượt khi số lượng đối tượng lên tới hàng nghìn. Konva.js là thư viện vẽ canvas hai chiều cung cấp binding React sẵn có thông qua react-konva, cho phép quản lý từng đối tượng đồ họa như một node độc lập trên cây cảnh tượng tự mô hình thao tác DOM nhưng hoạt động trên canvas thay vì SVG, nhờ đó đạt hiệu năng cao hơn khi số lượng đối tượng lớn. Nhà tổ chức có thể tạo, kéo thả, thay đổi kích thước và xoay các khu vực trực tiếp; tọa độ được lưu theo tỷ lệ phần trăm thay vì pixel tuyệt đối để sơ đồ thích ứng đúng tỷ lệ trên mọi kích thước màn hình. So với Fabric.js, Konva.js có hệ sinh thái React tốt hơn và tài liệu phong phú hơn; so với SVG thuần, Konva.js xử lý nhiều đối tượng đồng thời tốt hơn đáng kể.

3.6 Tích hợp dịch vụ và Triển khai

SePay là cổng thanh toán chuyên theo dõi giao dịch chuyển khoản ngân hàng nội địa qua mã QR chuẩn VietQR. Khi người dùng hoàn tất chuyển khoản, SePay gửi webhook tới máy chủ kèm API key trong tiêu đề `Authorization`; máy chủ đối chiếu giá trị này với khoá bí mật được cấu hình để xác thực tính hợp lệ của thông báo trước khi cấp vé. Mô hình này đơn giản và đủ an toàn khi kết hợp với HTTPS và khoá bí mật có độ dài đủ lớn. So với PayOS hay VNPay, SePay không thu phí giao dịch và hỗ trợ nhận webhook theo thời gian thực phù hợp với yêu cầu cấp vé tức thời sau khi xác nhận thanh toán.

Cloudinary là dịch vụ lưu trữ và xử lý tệp phương tiện tích hợp mạng phân phối nội dung (CDN) toàn cầu. Ảnh banner sự kiện và tài liệu pháp lý của nhà tổ chức được tải lên Cloudinary và tự động chuyển đổi định dạng theo thiết bị của người xem, giúp giảm băng thông và tăng tốc độ tải trang mà không cần xử lý ảnh thủ công trên máy chủ.

Nodemailer kết hợp Gmail qua SMTP đảm nhiệm gửi thư điện tử tự động cho các thông báo nghiệp vụ như xác nhận đặt vé, cập nhật trạng thái phê duyệt sự kiện và xác nhận quyết toán. Nội dung thư được xây dựng bằng template Handlebars để tách biệt phần trình bày khỏi logic gửi, cho phép thay đổi mẫu thư mà không cần sửa mã nguồn.

Toàn bộ dịch vụ Backend bao gồm NestJS, PostgreSQL, Redis và RabbitMQ được đóng gói trong các container riêng biệt và điều phối bằng Docker Compose [18] trên một máy chủ riêng ảo. Cách tiếp cận này đảm bảo môi trường chạy nhất quán giữa máy phát triển và môi trường sản xuất, đồng thời cho phép khởi động lại từng dịch vụ độc lập mà không ảnh hưởng đến các dịch vụ còn lại. Phần giao diện người dùng được triển khai trên Vercel, tận dụng mạng edge toàn cầu và tích hợp CI/CD tự động để rút ngắn thời gian đưa bản cập nhật lên môi trường sản xuất. So với Kubernetes, Docker Compose phù hợp hơn cho quy mô một máy chủ vì không phát sinh chi phí vận hành của một cụm điều phối đầy đủ.

Chương 3 đã trình bày căn cứ lý thuyết và lý do lựa chọn từng công nghệ trong hệ thống. Chương 4 sẽ trình bày cách các công nghệ này kết hợp trong thiết kế, triển khai và kết quả đánh giá thực nghiệm.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày quá trình thiết kế, xây dựng, kiểm thử và triển khai hệ thống đặt vé sự kiện trực tuyến. Nội dung được tổ chức theo bốn phần chính: thiết kế kiến trúc và chi tiết, xây dựng ứng dụng, kiểm thử tải, và triển khai thực tế.

4.1 Thiết kế kiến trúc

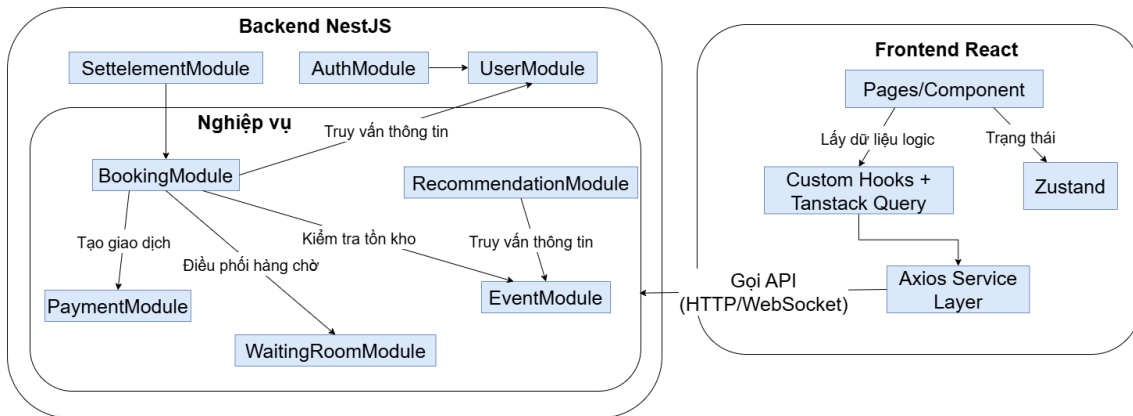
4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống áp dụng kiến trúc phân tầng (Layered Architecture) ở cả hai cấp độ: cấp vĩ mô (Client-Server) và cấp vi mô (bên trong từng thành phần). Kiến trúc phân tầng tổ chức phần mềm thành các tầng chồng lên nhau, mỗi tầng chỉ giao tiếp với tầng liền kề, đảm bảo nguyên tắc tách biệt trách nhiệm và giảm phụ thuộc giữa các thành phần.

Phía backend, NestJS được tổ chức theo ba tầng. Tầng Controller tiếp nhận HTTP request và WebSocket event, xác thực đầu vào, rồi ủy thác xử lý cho tầng dưới. Tầng Service chứa toàn bộ logic nghiệp vụ, phối hợp giữa các domain và các service hạ tầng (Redis, RabbitMQ, Cloudinary). Tầng Repository truy xuất và ghi nhận dữ liệu thông qua TypeORM, hoàn toàn tách biệt khỏi logic nghiệp vụ. Mỗi domain nghiệp vụ được đóng gói thành một Module độc lập: AuthModule, UsersModule, EventsModule, BookingsModule, PaymentsModule, WaitingRoomModule, RecommendationModule, và SettlementModule. Các module giao tiếp với nhau qua Dependency Injection, không truy cập trực tiếp vào repository của module khác.

Phía frontend, React được tổ chức theo bốn tầng. Tầng giao diện (Pages và Components) chịu trách nhiệm duy nhất là hiển thị dữ liệu và nhận tương tác người dùng. Tầng logic (Custom Hooks và TanStack Query) đóng gói toàn bộ trạng thái máy chủ (server state), lưu đệm (caching) và hiệu ứng phụ (side effect). Tầng trạng thái (Zustand) quản lý trạng thái máy khách (client state) cần lưu bền (persist) giữa các trang. Tầng API (axios service layer) tập trung hóa mọi lời gọi đến backend. Luồng dữ liệu theo một chiều từ tầng API lên đến giao diện, đảm bảo tính nhất quán và khả năng kiểm thử.

4.1.2 Thiết kế tổng quan



Hình 4.1: Biểu đồ phụ thuộc module của hệ thống

Hình 4.1 thể hiện cấu trúc gói của hệ thống. Phía backend, tám module được phân thành hai nhóm: nhóm nền tảng (AuthModule, UsersModule) cung cấp dịch vụ dùng chung cho toàn hệ thống, và nhóm nghiệp vụ triển khai các tính năng cốt lõi. Trong nhóm nghiệp vụ, BookingsModule phụ thuộc vào EventsModule (kiểm tra tồn kho), PaymentsModule (tạo giao dịch), và WaitingRoomModule (điều phối hàng chờ). RecommendationModule phụ thuộc vào EventsModule để truy vấn véc-tơ nhúng qua pgvector. SettlementModule phụ thuộc vào BookingsModule để tổng hợp doanh thu sau sự kiện. Phía frontend, các tầng tuân theo quy tắc phụ thuộc một chiều: Pages phụ thuộc Hooks, Hooks phụ thuộc Services, tầng dưới không bao giờ phụ thuộc tầng trên.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện hệ thống hướng đến màn hình máy tính để bàn với độ phân giải tối thiểu 1280×720, đồng thời đảm bảo thích nghi (responsive) xuống đến 375px trên thiết bị di động nhờ hệ thống điểm ngắt (breakpoint) của Tailwind CSS. Bảng 4.1 trình bày năm mức điểm ngắt được áp dụng xuyên suốt ứng dụng.

Tên điểm ngắt	Chiều rộng tối thiểu
sm	640 px
md	768 px
lg	1024 px
xl	1280 px
2xl	1400 px

Bảng 4.1: Hệ thống điểm ngắt giao diện

Font chữ chủ đạo là Quicksand (Google Fonts), được tải với năm mức độ đậm: 300, 400, 500, 600 và 700. Quicksand là font sans-serif hình học với nét bo tròn,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

phù hợp với phong cách hiện đại của nền tảng sự kiện.

Bảng màu xoay quanh màu vàng gold (#FFD700) làm màu chủ đạo (primary color), với bộ mười sắc độ từ #FFF9E6 (rất nhạt) đến #332C00 (rất đậm) phục vụ các trạng thái hover, focus và disabled. Hệ thống hỗ trợ chế độ tối (dark mode) thông qua biến CSS (CSS variable) — giao diện tự chuyển bảng màu theo lựa chọn của người dùng mà không cần tải lại trang. Bảng 4.2 liệt kê các màu ngữ nghĩa chính ở hai chế độ hiển thị.

Vai trò ngữ nghĩa	Chế độ sáng	Chế độ tối
Primary (chủ đạo)	#FFD700	#FFD700
Primary nhạt	#FFF9E6	#332C00
Nền trang	#FFFFFF	hsl(20, 14%, 4%)
Văn bản chính	hsl(240, 10%, 4%)	hsl(0, 0%, 95%)
Lỗi (error)	hsl(0, 84%, 60%)	hsl(0, 84%, 60%)

Bảng 4.2: Bảng màu ngữ nghĩa của hệ thống

Một số chuẩn hóa giao diện được áp dụng nhất quán xuyên suốt ứng dụng. Nút hành động chính dùng màu primary với bo góc 8px; nút phụ dùng kiểu viền (outlined) cùng màu primary. Thông điệp phản hồi (thành công, lỗi, cảnh báo) hiển thị dưới dạng Toast notification ở vị trí trên cùng giữa màn hình, tự đóng sau một khoảng thời gian ngắn, được xử lý bởi thư viện react-hot-toast. Kiểm tra hợp lệ biểu mẫu (form validation) tận dụng cơ chế kiểm tra gốc của HTML5 thông qua các thuộc tính `required`, `minLength` và `pattern` khai báo trực tiếp trên phần tử nhập liệu, lỗi hiển thị ngay bên dưới trường tương ứng.

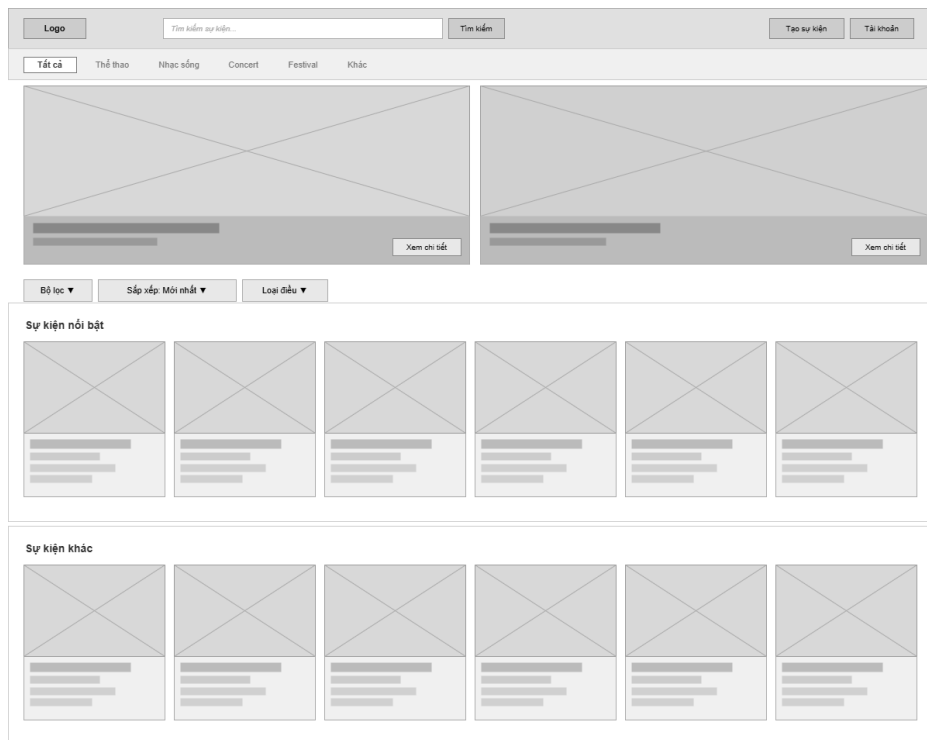
Ứng dụng frontend gồm 21 trang phân theo bốn nhóm vai trò, được trình bày trong Bảng 4.3.

Vai trò	Số trang	Các trang chính
Người dùng	7	Home, EventDetail, WaitingRoom, Check-Out, BookingSuccess, MyTickets, Profile
Nhà tổ chức	7	RegisterOrganizer, CreateEvent, EventEdit, MyEvents, EventStats, SeatMapEditor, CollaboratorMgmt
Quản trị viên	5	AdminDashboard, AdminEvents, AdminOrganizers, AdminUsers, AdminSettlements
Cộng tác viên	1	CheckIn
Chung	1	Login / Register

Bảng 4.3: Danh sách trang của ứng dụng frontend phân theo vai trò

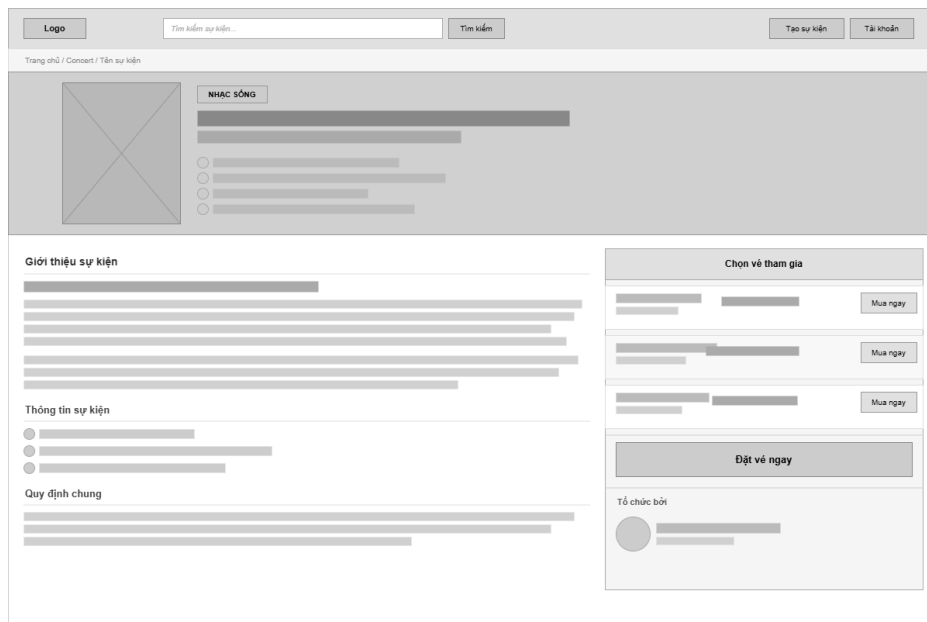
Hình 4.2 thể hiện giao diện trang chủ, nơi người dùng có thể tìm kiếm, lọc theo thể loại và duyệt danh sách sự kiện được phân thành hai nhóm: sự kiện nổi bật và

sự kiện khác.



Hình 4.2: Giao diện trang chủ — danh sách và phân loại sự kiện

Hình 4.3 trình bày giao diện trang chi tiết sự kiện, hiển thị thông tin mô tả bên trái và bảng chọn hạng vé kèm nút đặt vé bên phải.



Hình 4.3: Giao diện trang chi tiết sự kiện — thông tin và lựa chọn vé

Hình 4.4 mô tả giao diện tạo sự kiện dành cho nhà tổ chức, áp dụng quy trình nhiều bước gồm bốn giai đoạn: Thông tin cơ bản, Thời gian & Địa điểm, Cấu hình Vé và Tài liệu pháp lý.

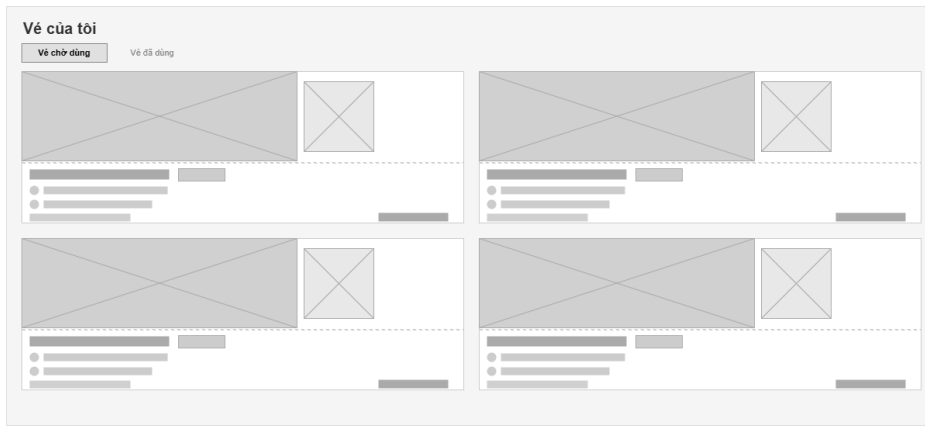
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Hình 4.4: Giao diện tạo sự kiện mới — bước nhập thông tin cơ bản

Hình 4.5 trình bày giao diện thanh toán, hiển thị đồng hồ đếm ngược thời gian giữ vé, thông tin đơn hàng và mã QR chuyển khoản ngân hàng qua SePay.

Hình 4.5: Giao diện thanh toán — đếm ngược thời gian giữ vé và mã QR chuyển khoản

Hình 4.6 thể hiện giao diện trang vé của người dùng, phân thành hai tab "Vé chờ dùng" và "Vé đã dùng", mỗi thẻ vé hiển thị ảnh sự kiện và mã QR dùng cho check-in.



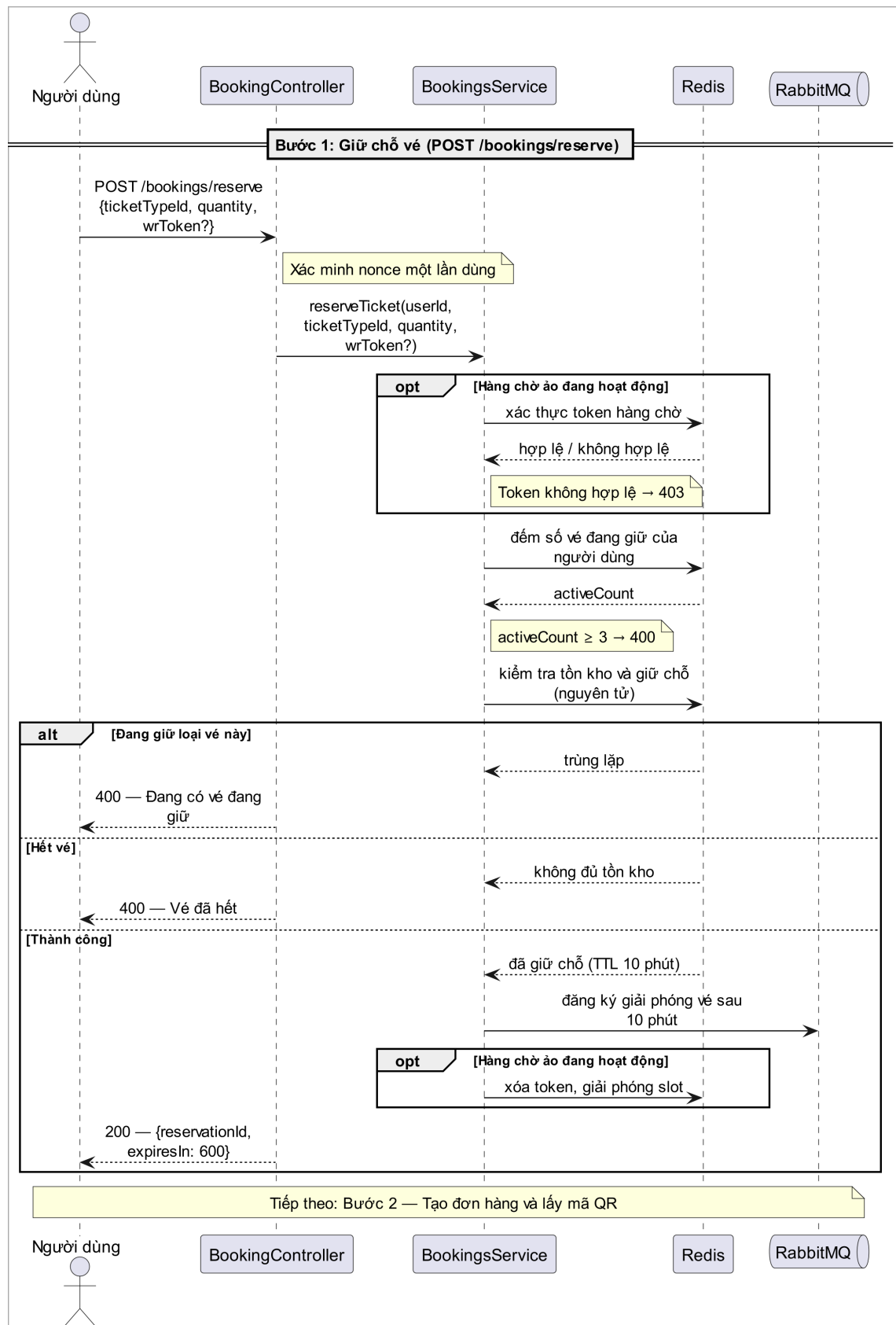
Hình 4.6: Giao diện trang vé của tôi — quản lý vé chờ dùng và vé đã dùng

4.2.2 Thiết kế hành vi

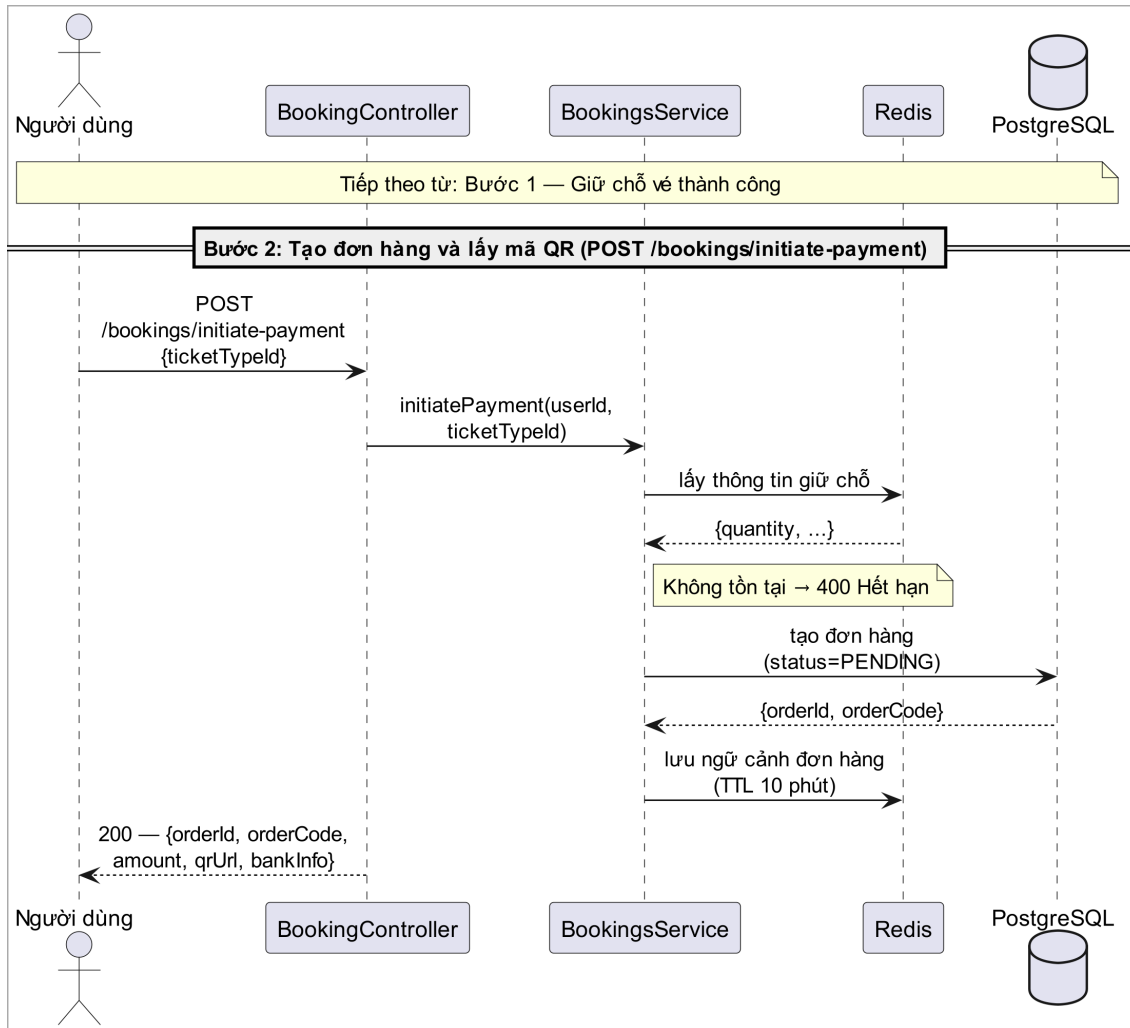
Phần này trình bày biểu đồ trình tự (sequence diagram) cho hai luồng nghiệp vụ cốt lõi của hệ thống: luồng đặt vé và luồng xác nhận thanh toán qua webhook. Hai luồng này được chọn vì chúng bao trùm toàn bộ vòng đời của một giao dịch mua vé và tích hợp đồng thời nhiều cơ chế kỹ thuật quan trọng nhất của hệ thống mà các biểu đồ hoạt động ở Chương 2 chưa thể hiện ở cấp độ tương tác giữa các thành phần.

Hình 4.7 và Hình 4.8 trình bày biểu đồ trình tự của luồng đặt vé, gồm hai bước tương ứng với hai lời gọi API độc lập. Hình 4.7 minh họa Bước 1 — giữ chỗ vé (POST /bookings/reserve): BookingController xác thực mã nonce dùng một lần, sau đó BookingsService kiểm tra token hàng chờ ảo (nếu đang hoạt động), đếm số vé đang giữ của người dùng, rồi thực thi tập lệnh Lua trên Redis để kiểm tra tồn kho và giữ chỗ trong một thao tác nguyên tử — đảm bảo không xảy ra tình trạng bán vượt số lượng kể cả khi có nhiều yêu cầu đồng thời. Tập lệnh Lua trả về ba kết quả có thể: -2 nếu người dùng đã giữ loại vé này, 0 nếu hết vé, và 1 nếu thành công; khi thành công, hệ thống đăng ký một tin nhắn có độ trễ lên RabbitMQ để tự động hoàn trả tồn kho sau mười phút nếu người dùng không hoàn tất thanh toán. Hình 4.8 minh họa Bước 2 — tạo đơn hàng và lấy mã QR (POST /bookings/initiate-payment): hệ thống đọc thông tin giữ chỗ từ Redis, tạo bản ghi đơn hàng trong PostgreSQL với trạng thái PENDING, lưu ngữ cảnh đơn hàng vào Redis, rồi trả về mã QR chuyển khoản cho người dùng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

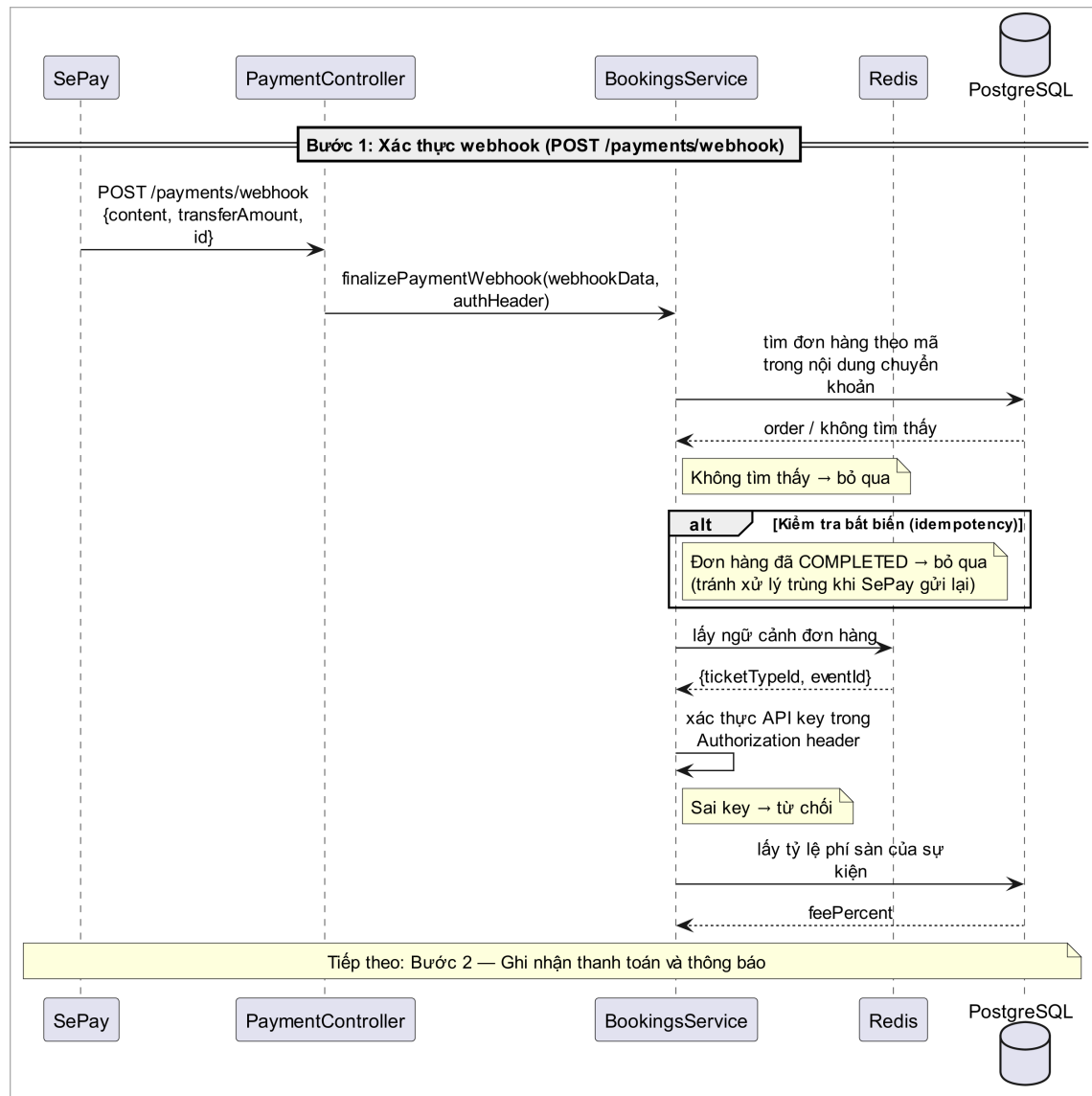


Hình 4.7: Biểu đồ trình tự luồng đặt vé — Bước 1: Giữ chỗ vé (POST /bookings/reserve)



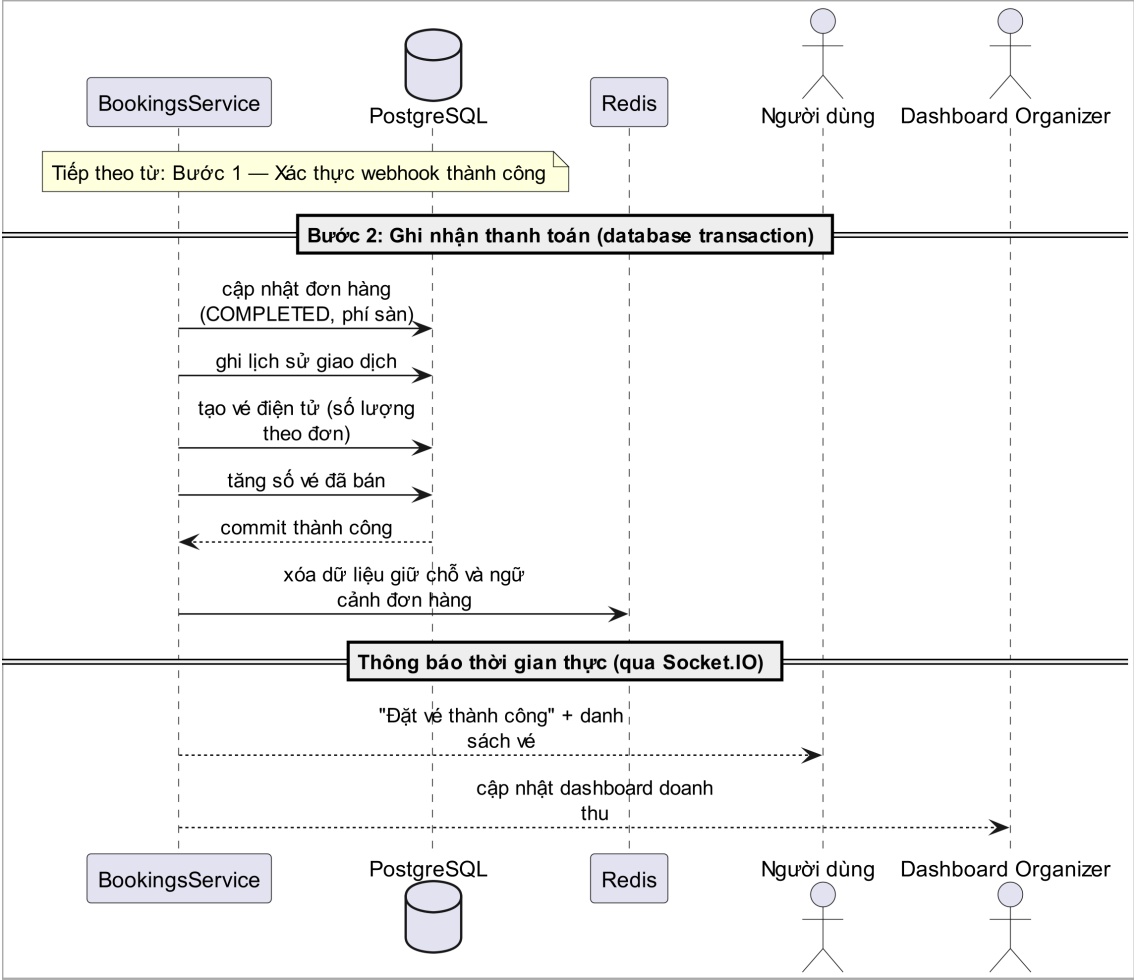
Hình 4.8: Biểu đồ trình tự luồng đặt vé — Bước 2: Tạo đơn hàng và lấy mã QR (POST /bookings/initiate-payment)

Hình 4.9 và Hình 4.10 trình bày biểu đồ trình tự của luồng xác nhận thanh toán qua webhook, khởi nguồn từ phía SePay sau khi phát hiện giao dịch ngân hàng có nội dung khớp mã đơn hàng. Luồng được chia thành hai giai đoạn. Hình 4.9 mô tả giai đoạn xác thực: BookingsService thực hiện tuần tự bốn lớp kiểm tra gồm (i) xác nhận đơn hàng tồn tại trong cơ sở dữ liệu, (ii) kiểm tra tính bất biến (idempotency) — nếu đơn hàng đã ở trạng thái COMPLETED thì bỏ qua ngay để tránh xử lý trùng lặp khi SePay gửi lại webhook, (iii) đối chiếu API key trong tiêu đề Authorization để ngăn chặn webhook giả mạo từ bên ngoài, và (iv) kiểm tra số tiền chuyển không thấp hơn giá trị đơn hàng.



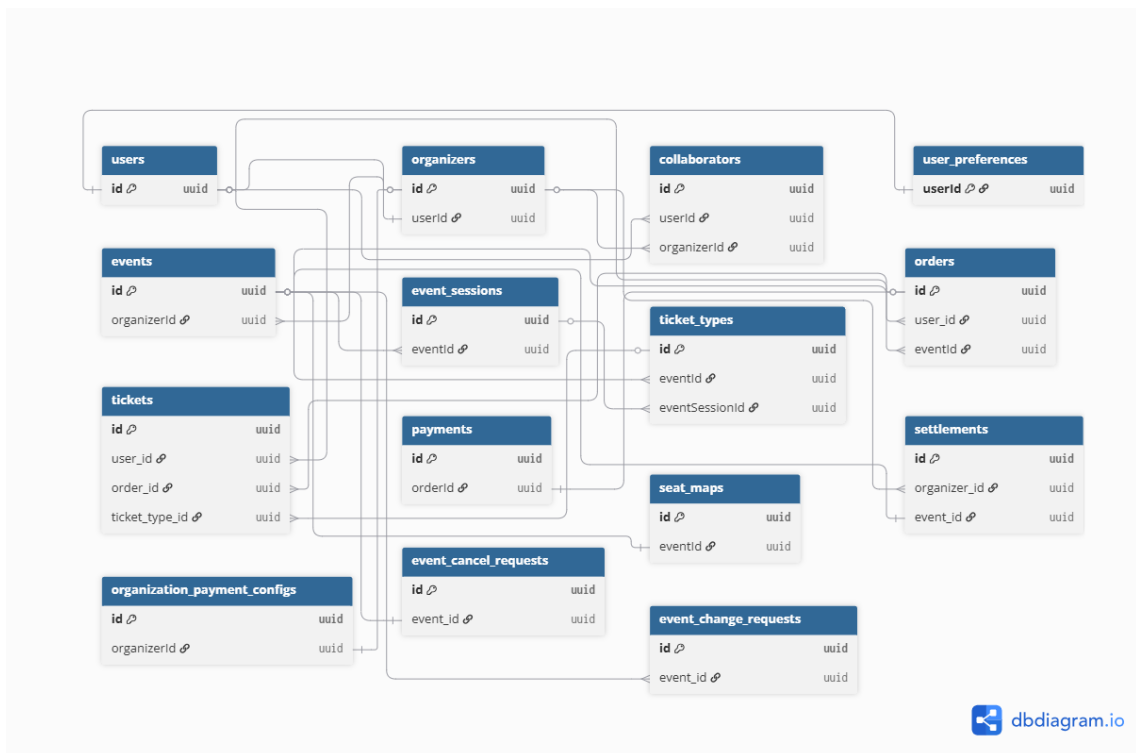
Hình 4.9: Biểu đồ trình tự luồng xác nhận thanh toán qua webhook SePay — Bước 1: Xác thực webhook

Hình 4.10 mô tả giai đoạn ghi nhận: khi toàn bộ điều kiện thỏa mãn, hệ thống mở một giao dịch cơ sở dữ liệu để thực hiện nguyên tử bốn thao tác gồm cập nhật trạng thái đơn hàng kèm thông tin phí sản, ghi lịch sử giao dịch vào bảng `payments`, tạo hàng loạt vé điện tử với mã truy cập, và tăng số lượng vé đã bán. Sau khi commit thành công, hệ thống xóa toàn bộ khóa Redis liên quan rồi phát hai sự kiện `Socket.IO` song song: `payment_success` đến người dùng để hiển thị màn hình thành công, và `ticket-sold` đến dashboard nhà tổ chức để cập nhật thống kê doanh thu theo thời gian thực.



Hình 4.10: Biểu đồ trình tự luồng xác nhận thanh toán qua webhook SePay — Bước 2: Ghi nhận thanh toán và thông báo thời gian thực

4.2.3 Thiết kế cơ sở dữ liệu



Hình 4.11: Sơ đồ cơ sở dữ liệu vật lý của hệ thống

a, Danh sách bảng theo nhóm chức năng

Hình 4.11 thể hiện sơ đồ cơ sở dữ liệu vật lý của hệ thống. Cơ sở dữ liệu gồm mười lăm bảng được tổ chức thành ba nhóm chức năng, được mô tả chi tiết trong Bảng 4.4, Bảng 4.5 và Bảng 4.6.

Bảng	Chức năng	Trường / ràng buộc đáng chú ý
users	Tài khoản hệ thống	role (enum: admin, organizer, user, collaborator); isActive kiểm soát trạng thái hoạt động
organizers	Hồ sơ KYC/KYB của nhà tổ chức	organizerType (INDIVIDUAL / BUSINESS / HOUSEHOLD); kycStatus theo quy trình duyệt; quan hệ một-một với users
collaborators	Liên kết cộng tác viên với nhà tổ chức	assignedEventIds (mảng uuid) lưu danh sách sự kiện được phân quyền check-in
user_preferences	Dữ liệu cá nhân hóa phục vụ gợi ý	profileVector (vector) và interactionCount; quan hệ một-một với users

Bảng 4.4: Nhóm bảng tài khoản và người dùng

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng	Chức năng	Trường / ràng buộc đáng chú ý
orders	Đơn hàng	platformFeePercent, platformFeeAmount, organizerAmount cố định tại thời điểm đặt để tránh tranh chấp phí
payments	Lịch sử giao dịch từ webhook SePay	rawPayload dạng JSONB; ràng buộc một-một với orders chống ghi nhận trùng lặp
tickets	Vé điện tử	accessCode dùng cho check-in; status (unchecked / checked_in / cancelled / refunded)
settlements	Phiếu quyết toán tài chính sau sự kiện	status (PENDING / PROCESSING / COMPLETED); thông tin ngân hàng được sao chép tại thời điểm tạo phiếu; quan hệ một-một với events

Bảng 4.5: Nhóm bảng giao dịch tài chính

Bảng	Chức năng	Trường / ràng buộc đáng chú ý
events	Thông tin sự kiện	status (8 giá trị theo quy trình duyệt); embedding (vector) cho tìm kiếm ngữ nghĩa; deletedAt (xóa mềm)
event_sessions	Suất diễn thuộc sự kiện	Có thời gian, địa điểm và sức chứa riêng; quan hệ nhiều-một với events
seat_maps	Sơ đồ chỗ ngồi	stageElements và zones dạng JSONB; quan hệ một-một với events
ticket_types	Hạng vé	Ràng buộc CHECK đảm bảo thuộc đúng một trong hai: eventId hoặc eventSessionId; maxPerTransaction giới hạn số vé mỗi giao dịch
organization_payment_configs	Tài khoản ngân hàng nhận tiền của nhà tổ chức	Quan hệ một-một với organizers
event_cancel_requests	Yêu cầu hủy sự kiện	refundProposal; status (pending / approved / rejected); quan hệ một-một với events
event_change_requests	Yêu cầu chỉnh sửa thông tin sự kiện	requestedChanges dạng JSONB; quan hệ nhiều-một với events

Bảng 4.6: Nhóm bảng sự kiện và cấu hình

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Các quan hệ một-một trong lược đồ gồm: `users—organizers`, `users—user_preferences`, `organizers—organization_payment_configs`, `events—seat_maps`, `events—event_cancel_requests`, `orders—payments` và `events—settlements`. Các quan hệ một-nhiều gồm: `organizers—events`, `events—event_sessions`, `events—event_change_requests`, `users—orders` và `orders—tickets`. Toàn bộ ràng buộc khóa ngoại được PostgreSQL thực thi ở cấp độ cơ sở dữ liệu, đảm bảo tính nhất quán dữ liệu (data integrity) trong các luồng giao dịch đồng thời.

b, Đặc tả chi tiết các bảng cơ sở dữ liệu

Phần này trình bày đặc tả chi tiết cấu trúc của chín bảng cốt lõi trong lược đồ cơ sở dữ liệu, bao gồm tên trường, kiểu dữ liệu, ràng buộc và ý nghĩa ngữ nghĩa của từng trường.

Bảng 4.7 đặc tả bảng `users` — bảng trung tâm lưu trữ thông tin tài khoản cho tất cả vai trò trong hệ thống.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>id</code>	<code>uuid</code>	PK, NOT NULL	Định danh tài khoản, sinh tự động
<code>name</code>	<code>varchar(200)</code>	NOT NULL	Họ và tên người dùng
<code>email</code>	<code>varchar</code>	NOT NULL, UNIQUE	Địa chỉ email đăng nhập
<code>phone</code>	<code>varchar(11)</code>	UNIQUE, NULL	Số điện thoại; NULL với tài khoản đăng nhập OAuth
<code>password</code>	<code>varchar(255)</code>	NULL	Mật khẩu đã băm bcrypt; NULL với đăng nhập OAuth
<code>role</code>	<code>enum</code>	NOT NULL, DEFAULT 'user'	Vai trò: <code>admin</code> , <code>organizer</code> , <code>user</code> , <code>collaborator</code>
<code>isActive</code>	<code>boolean</code>	NOT NULL, DEFAULT true	Trạng thái hoạt động; false khi tài khoản bị khóa
<code>isEmailVerified</code>	<code>boolean</code>	NOT NULL, DEFAULT false	Trạng thái xác minh địa chỉ email
<code>refreshToken</code>	<code>varchar(500)</code>	NULL	Mã làm mới phiên đăng nhập JWT
<code>createdAt</code>	<code>timestampz</code>	NOT NULL	Thời điểm tạo tài khoản
<code>updatedAt</code>	<code>timestampz</code>	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.7: Đặc tả bảng `users`

Bảng 4.8 đặc tả bảng `organizers` — lưu hồ sơ KYC/KYB của nhà tổ chức, liên kết một-một với `users`.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh nhà tổ chức
userId	uuid	FK→users, UNIQUE	Tài khoản người dùng tương ứng (quan hệ 1-1)
name	varchar	NOT NULL	Tên tổ chức hoặc cá nhân
description	varchar	NULL	Mô tả giới thiệu nhà tổ chức
organizerType	enum	NOT NULL	Loại hình: INDIVIDUAL, BUSINESS, HOUSEHOLD
taxCode	varchar	NULL	Mã số thuế (bắt buộc với BUSINESS và HOUSEHOLD)
phone	varchar	NULL	Số điện thoại liên hệ công khai
website	varchar	NULL	Địa chỉ website của tổ chức
documents	jsonb	NULL	Danh sách URL tài liệu pháp lý đính kèm
kycStatus	enum	NOT NULL, DEFAULT 'PENDING'	Trạng thái duyệt: PENDING, APPROVED, REJECTED, NEEDS_REVISION
kycRejectedReason	varchar	NULL	Lý do từ chối, điền khi kycStatus = REJECTED
kycSubmittedAt	timestamp	NULL	Thời điểm nộp hồ sơ KYC
kycReviewedAt	timestamp	NULL	Thời điểm admin hoàn thành duyệt
createdAt	timestamp	NOT NULL	Thời điểm tạo hồ sơ
updatedAt	timestamp	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.8: Đặc tả bảng organizers

Bảng 4.9 đặc tả bảng event_sessions — lưu thông tin các suất diễn thuộc một sự kiện đa suất; mỗi suất có thể có địa điểm và sức chứa riêng.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh suất diễn
eventId	uuid	FK→events, ON DELETE CASCADE	Sự kiện cha; xóa suất khi sự kiện bị xóa
name	varchar(200)	NOT NULL	Tên suất diễn
startTime	timestampz	NOT NULL	Thời gian bắt đầu suất diễn
endTime	timestampz	NOT NULL	Thời gian kết thúc suất diễn
sessionLocation	varchar(255)	NULL	Địa điểm riêng của suất (nếu khác với sự kiện cha)
capacity	int4	NULL	Sức chứa tối đa; NULL là không giới hạn
status	enum	NOT NULL, DEFAULT 'upcoming'	Trạng thái: upcoming, ongoing, ended, cancelled
createdAt	timestampz	NOT NULL	Thời điểm tạo
updatedAt	timestampz	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.9: Đặc tả bảng event_sessions

Bảng 4.10 đặc tả bảng seat_maps — lưu cấu trúc sơ đồ chỗ ngồi dưới dạng JSONB, liên kết một-một với events.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh sơ đồ chỗ ngồi
eventId	uuid	FK→events, UNIQUE, ON DELETE CASCADE	Sự kiện tương ứng (quan hệ 1-1)
bgUrl	varchar(500)	NULL	URL ảnh nền của canvas
canvasWidth	int4	NOT NULL, DEFAULT 900	Chiều rộng canvas (đơn vị: px)
canvasHeight	int4	NOT NULL, DEFAULT 600	Chiều cao canvas (đơn vị: px)
stageElements	jsonb	NOT NULL, DEFAULT '[]'	Mảng JSON chứa các phần tử sân khấu (hình dạng, vị trí, kích thước)
zones	jsonb	NOT NULL, DEFAULT '[]'	Mảng JSON chứa các vùng chỗ ngồi và hạng vé gán cho từng vùng
createdAt	timestampz	NOT NULL	Thời điểm tạo
updatedAt	timestampz	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.10: Đặc tả bảng `seat_maps`

Bảng 4.11 đặc tả bảng `orders` — lưu thông tin đơn hàng với các trường tài chính được cố định tại thời điểm tạo để tránh tranh chấp phí sau này.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh đơn hàng
user_id	uuid	FK→users	Người dùng đặt vé
eventId	uuid	NULL	Sự kiện liên quan (lưu để tra cứu không cần join thêm bảng)
status	enum	NOT NULL, DEFAULT 'pending'	Trạng thái: pending, completed, cancelled, refunded
totalPrice	int4	NOT NULL	Tổng giá trị đơn hàng (đồng VND)
totalQuantity	int4	NOT NULL, DEFAULT 0	Tổng số vé trong đơn
paymentMethod	varchar	NULL	Phương thức thanh toán
transactionId	varchar	NULL	Mã giao dịch từ SePay
platformFeePercent	numeric(5,2)	NULL	Phí sàn (%) cố định tại thời điểm đặt
platformFeeAmount	int4	NULL	Số tiền phí sàn (đồng), cố định tại thời điểm đặt
organizerAmount	int4	NULL	Số tiền nhà tổ chức nhận (đồng), cố định tại thời điểm đặt
createdAt	timestampz	NOT NULL	Thời điểm tạo đơn hàng
updatedAt	timestampz	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.11: Đặc tả bảng `orders`

Bảng 4.12 đặc tả bảng `tickets` — lưu từng vé điện tử được tạo sau khi thanh toán thành công, với mã truy cập dùng cho check-in tại sự kiện.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh vé điện tử
user_id	uuid	FK→users	Chủ sở hữu vé
order_id	uuid	FK→orders	Đơn hàng phát sinh vé
ticket_type_id	uuid	FK→ticket_types	Hạng vé tương ứng
accessCode	varchar(20)	NOT NULL, UNIQUE	Mã truy cập ngẫu nhiên dùng để sinh mã QR check-in
status	enum	NOT NULL, DEFAULT 'unchecked'	Trạng thái: unchecked, checked_in, cancelled, refunded
checkedInAt	timestampz	NULL	Thời điểm check-in; NULL nếu vé chưa được sử dụng
createdAt	timestampz	NOT NULL	Thời điểm tạo vé
updatedAt	timestampz	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.12: Đặc tả bảng `tickets`

Bảng 4.13 đặc tả bảng `ticket_types` — định nghĩa các hạng vé với ràng buộc CHECK đảm bảo mỗi hạng vé thuộc đúng một trong hai: sự kiện hoặc suất diễn.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	uuid	PK, NOT NULL	Định danh hạng vé
eventId	uuid	FK→events, NULL	Thuộc sự kiện không phân suất; CHECK đảm bảo XOR với eventSessionId
eventSessionId	uuid	FK→event_sessions, NULL	Thuộc suất diễn; CHECK đảm bảo XOR với eventId
name	varchar(200)	NOT NULL	Tên hạng vé
price	int4	NOT NULL	Giá vé (đơn vị: đồng VND)
quantity	int4	NOT NULL	Tổng số vé phát hành
sold	int4	NOT NULL	Số vé đã bán thành công
rank	int4	NOT NULL	Thứ tự hiển thị trong danh sách hạng vé
description	text	NOT NULL	Mô tả quyền lợi và điều kiện của hạng vé
status	enum	NOT NULL, DEFAULT 'available'	Trạng thái: available, sold_out
saleStartTime	timestampz	NULL	Thời điểm bắt đầu bán riêng của hạng vé
saleEndTime	timestampz	NULL	Thời điểm kết thúc bán riêng của hạng vé
maxPer Transaction	int4	NOT NULL, DEFAULT 4	Số vé tối đa được mua trong một giao dịch
createdAt	timestampz	NOT NULL	Thời điểm tạo
updatedAt	timestampz	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.13: Đặc tả bảng `ticket_types`

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.14 đặc tả bảng `events` — bảng trung tâm quản lý toàn bộ thông tin sự kiện và điều phối vòng đời duyệt.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>id</code>	<code>uuid</code>	PK, NOT NULL	Định danh sự kiện
<code>organizerId</code>	<code>uuid</code>	FK→ <code>organizers</code>	Nhà tổ chức sự kiện
<code>name</code>	<code>varchar(200)</code>	NOT NULL, UNIQUE (name, organizerId)	Tên sự kiện, duy nhất trong phạm vi một nhà tổ chức
<code>description</code>	<code>text</code>	NOT NULL	Mô tả nội dung sự kiện
<code>eventType</code>	<code>enum</code>	NOT NULL, DEFAULT 'other'	Thể loại: <code>sport</code> , <code>live music</code> , <code>concert</code> , <code>festival</code> , <code>other</code>
<code>status</code>	<code>enum</code>	NOT NULL, DEFAULT 'draft'	Vòng đời 8 trạng thái
<code>location</code>	<code>varchar(255)</code>	NOT NULL	Địa chỉ chi tiết nơi tổ chức
<code>province</code>	<code>varchar(255)</code>	NOT NULL	Tỉnh/Thành phố tổ chức
<code>ward</code>	<code>varchar(255)</code>	NOT NULL	Phường/Xã tổ chức
<code>startSellDate</code>	<code>timestampz</code>	NOT NULL	Thời điểm bắt đầu bán vé
<code>endSellDate</code>	<code>timestampz</code>	NOT NULL	Thời điểm kết thúc bán vé
<code>startTime</code>	<code>timestampz</code>	NULL	Thời gian bắt đầu (sự kiện không phân suất)
<code>endTime</code>	<code>timestampz</code>	NULL	Thời gian kết thúc (sự kiện không phân suất)
<code>bannerUrl</code>	<code>varchar(500)</code>	NULL	URL ảnh banner quảng bá
<code>imageUrl</code>	<code>varchar(500)</code>	NULL	URL ảnh đại diện
<code>feePercentage</code>	<code>numeric(5,2)</code>	NULL	Phí sàn (%) do admin thiết lập khi duyệt
<code>adminNotes</code>	<code>text</code>	NULL	Ghi chú của admin trong quá trình duyệt
<code>documents</code>	<code>jsonb</code>	NULL	Tài liệu pháp lý đính kèm
<code>embedding</code>	<code>vector</code>	NULL	Véc-tơ nhúng 808 chiều phục vụ gợi ý cá nhân hóa
<code>submittedAt</code>	<code>timestampz</code>	NULL	Thời điểm nộp duyệt
<code>reviewedAt</code>	<code>timestampz</code>	NULL	Thời điểm admin hoàn thành duyệt
<code>deletedAt</code>	<code>timestampz</code>	NULL	Thời điểm xóa mềm; NULL nghĩa là chưa xóa
<code>createdAt</code>	<code>timestampz</code>	NOT NULL	Thời điểm tạo
<code>updatedAt</code>	<code>timestampz</code>	NOT NULL	Thời điểm cập nhật cuối

Bảng 4.14: Đặc tả bảng `events`

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.15 liệt kê các công cụ và thư viện chính sử dụng trong quá trình phát triển hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mục đích	Công cụ / Thư viện	Phiên bản
IDE	Visual Studio Code	1.89
Quản lý mã nguồn	Git / GitHub	2.44
Backend framework	NestJS	11.x
Frontend framework	React	18.x
Ngôn ngữ lập trình	TypeScript	5.x
Build tool	Vite	5.x
ORM	TypeORM	0.3.26
UI components	MUI (Material UI)	5.x
CSS utility	Tailwind CSS	3.x
Canvas / Seat Map	Konva.js / react-konva	9.x
Server state	TanStack Query	4.x
Client state	Zustand	4.x
Form validation	React Hook Form + Zod	7.x / 3.x
Cache / Queue state	Redis	7.x
Message queue	RabbitMQ	3.13
Cơ sở dữ liệu	PostgreSQL	15
Vector search	pgvector	0.7
Thời gian thực	Socket.IO	4.x
Thanh toán	SePay Webhook API	–
Lưu trữ media	Cloudinary SDK	2.x
Kiểm thử tải	k6	0.51
API testing	Postman	11.x
Container hóa	Docker + Docker Compose	26.x
Triển khai frontend	Vercel CLI	–

Bảng 4.15: Danh sách công cụ và thư viện sử dụng

4.3.2 Kết quả đạt được

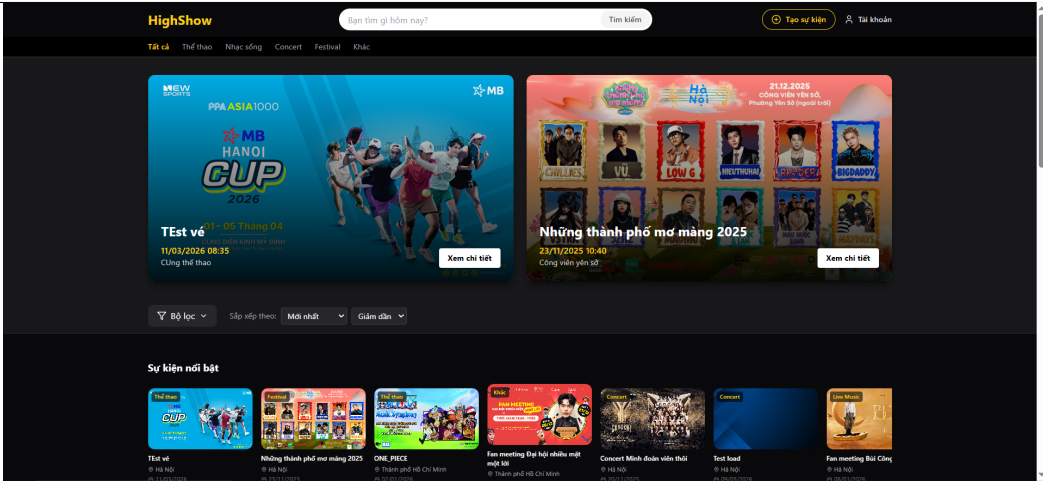
Hệ thống được đóng gói thành hai sản phẩm độc lập. Sản phẩm thứ nhất là ứng dụng backend NestJS triển khai dưới dạng Docker container trên VPS, bao gồm REST API server, Socket.IO gateway, và các RabbitMQ consumer (booking release, waiting room advance, recommendation embedding, batch refund). Sản phẩm thứ hai là ứng dụng frontend React build thành tập file tĩnh (HTML/CSS/JS) triển khai trên Vercel.

4.3.3 Minh họa các chức năng chính

Phần này minh họa giao diện thực tế của bảy chức năng cốt lõi trong hệ thống thông qua ảnh chụp màn hình từ ứng dụng đã triển khai.

Hình 4.12 thể hiện trang chủ thực tế của hệ thống, nơi người dùng có thể tìm kiếm sự kiện theo từ khóa, lọc theo thể loại và xem danh sách sự kiện được gợi ý dựa trên hồ sơ sở thích cá nhân hóa.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.12: Giao diện trang chủ thực tế — tìm kiếm và gợi ý sự kiện cá nhân hóa

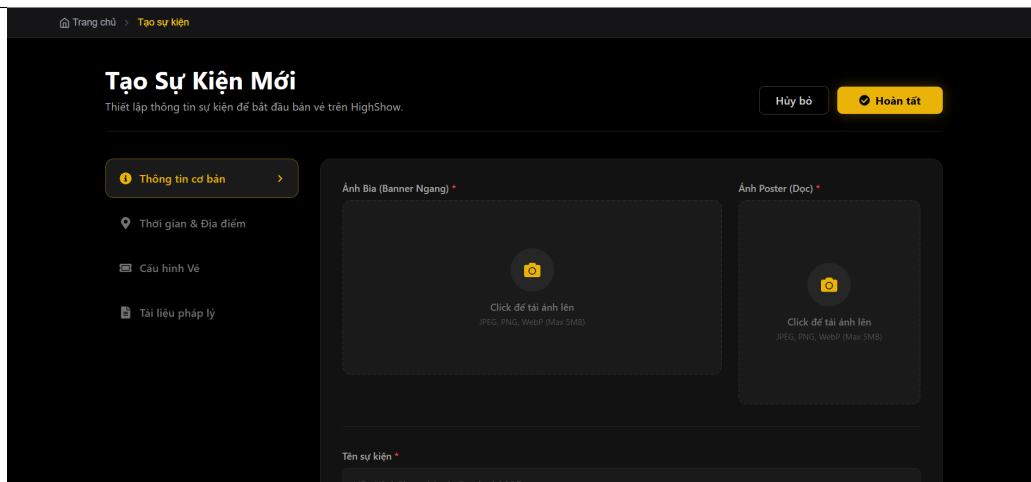
Hình 4.13 trình bày trang chi tiết sự kiện, hiển thị đầy đủ thông tin mô tả, thời gian, địa điểm và bảng chọn hạng vé kèm số lượng còn lại, cho phép người dùng lựa chọn và tiến hành đặt vé.



Hình 4.13: Giao diện trang chi tiết sự kiện thực tế — thông tin và lựa chọn hạng vé

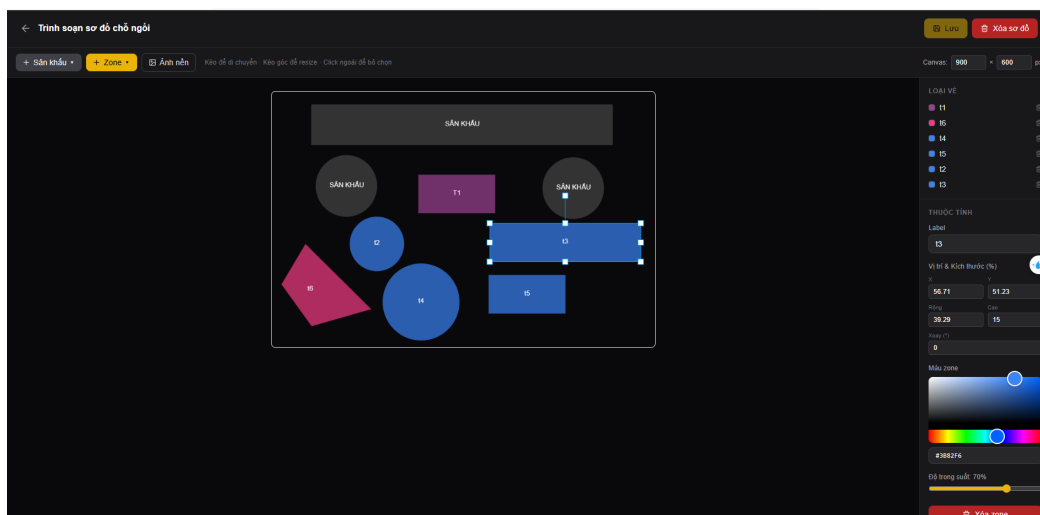
Hình 4.14 mô tả giao diện tạo sự kiện mới dành cho nhà tổ chức, áp dụng quy trình nhiều bước gồm bốn giai đoạn: Thông tin cơ bản, Thời gian & Địa điểm, Cấu hình vé và Tài liệu pháp lý; ảnh được thu nhỏ để chụp toàn bộ biểu mẫu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



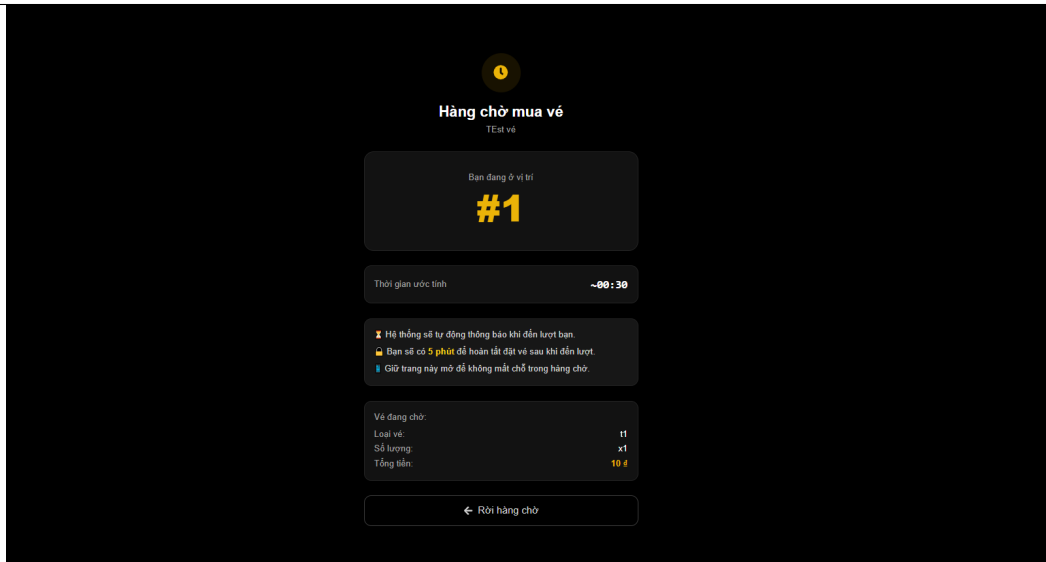
Hình 4.14: Giao diện tạo sự kiện thực tế — quy trình nhiều bước nhập thông tin và cấu hình vẽ

Hình 4.15 thể hiện trình soạn thảo sơ đồ chỗ ngồi kéo thả được xây dựng bằng Konva.js, cho phép nhà tổ chức vẽ sân khấu, phân vùng khu vực và gán hạng vé tương ứng cho từng vùng trực tiếp trên canvas.



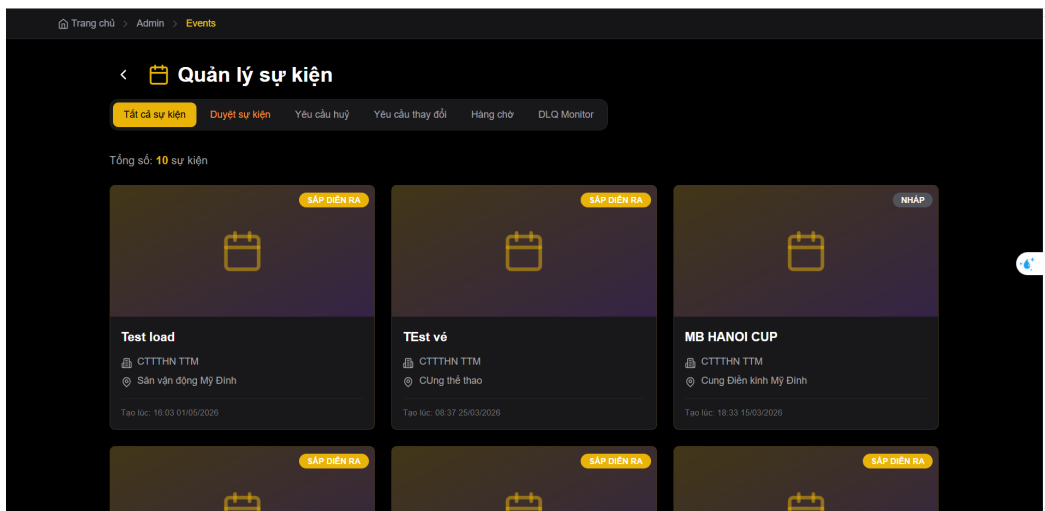
Hình 4.15: Giao diện thiết kế sơ đồ chỗ ngồi thực tế — kéo thả phân vùng trên canvas

Hình 4.16 trình bày màn hình hàng chờ ảo hiển thị cho người dùng vị trí xếp hàng và thời gian ước tính còn lại theo thời gian thực, giúp điều phối lưu lượng truy cập trong giai đoạn mở bán sự kiện có nhu cầu cao.



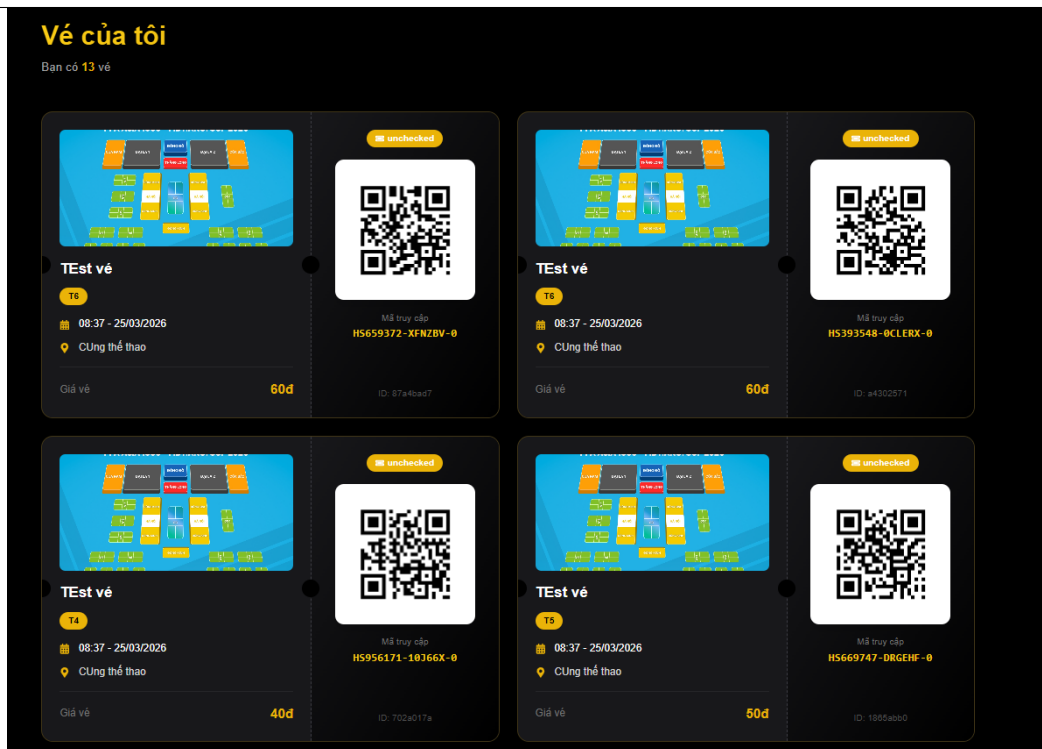
Hình 4.16: Giao diện hàng chờ ảo thực tế — hiển thị vị trí và thời gian chờ theo thời gian thực

Hình 4.17 mô tả trang quản lý sự kiện dành cho nhà tổ chức, tổng hợp danh sách các sự kiện đang hoạt động kèm trạng thái duyệt, số vé đã bán và doanh thu, đồng thời cung cấp các thao tác chỉnh sửa, hủy và xem thống kê chi tiết.



Hình 4.17: Giao diện quản lý sự kiện thực tế — danh sách, trạng thái và thao tác của nhà tổ chức

Hình 4.18 thể hiện trang vé điện tử của người dùng, phân thành hai tab "Vé chờ dùng" và "Vé đã dùng", mỗi thẻ vé hiển thị ảnh sự kiện, thông tin suất diễn và mã QR dùng cho check-in tại sự kiện.



Hình 4.18: Giao diện trang vé thực tế — quản lý vé điện tử và mã QR check-in

4.4 Kiểm thử

Hệ thống áp dụng hai kỹ thuật kiểm thử chính. Thứ nhất là kiểm thử chức năng bằng Postman — mỗi điểm cuối REST và sự kiện WebSocket được kiểm tra thủ công với bộ ca kiểm thử bao phủ luồng hợp lệ và các trường hợp biên (đầu vào không hợp lệ, token hết hạn, quyền không đủ). Thứ hai là kiểm thử tải bằng k6 với ba kịch bản tăng dần độ phức tạp, nhằm xác minh lần lượt: tính nguyên tử của thao tác gia nhập hàng chờ ảo, tính đúng đắn của cơ chế đẩy hàng theo nguyên tắc FIFO, và tính chính xác về tồn kho vé khi toàn bộ luồng hoạt động dưới tải cao.

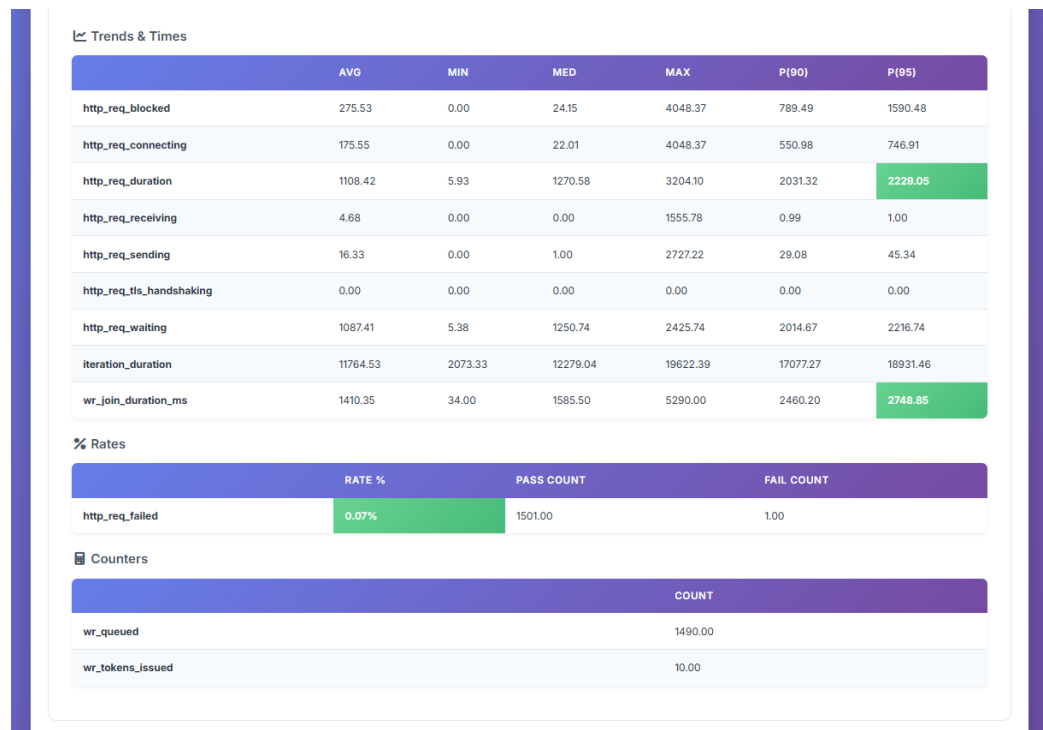
4.4.1 Kịch bản 1: Tính nguyên tử của thao tác gia nhập hàng chờ

Kịch bản này mô phỏng 1500 VU đồng thời gửi yêu cầu `POST /waiting-room/join` tới một sự kiện có sức chứa đồng thời tối đa (`maxConcurrent`) là 10. Mỗi VU thực hiện đúng một lần và không có bất kỳ độ trễ nhân tạo nào — toàn bộ 1500 yêu cầu tranh nhau cùng một tập slot giới hạn trong cùng một thời điểm. Mục tiêu là xác minh tập lệnh Lua `joinOrQueue` thực thi nguyên tử: số token cấp ra phải bằng đúng `maxConcurrent`, không hơn không kém, bất kể mức độ tranh chấp.

Kết quả thể hiện trong Hình 4.19 cho thấy chính xác 10 VU nhận token truy cập ngay lập tức và 1490 VU còn lại được xếp vào hàng chờ; không có trường hợp nào trả về trạng thái hàng chờ đầy hay hết vé bất thường. Điều quan trọng là nếu tập lệnh Lua không nguyên tử — tức là hai yêu cầu cùng lúc đọc được số slot còn trống

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

rồi đều cấp token — số token thực tế sẽ vượt quá 10. Kết quả đúng 10 token xác nhận Redis thực thi tập lệnh Lua theo cơ chế đơn luồng, không có xung đột tương tranh nào xảy ra dù có 1500 yêu cầu đồng thời.

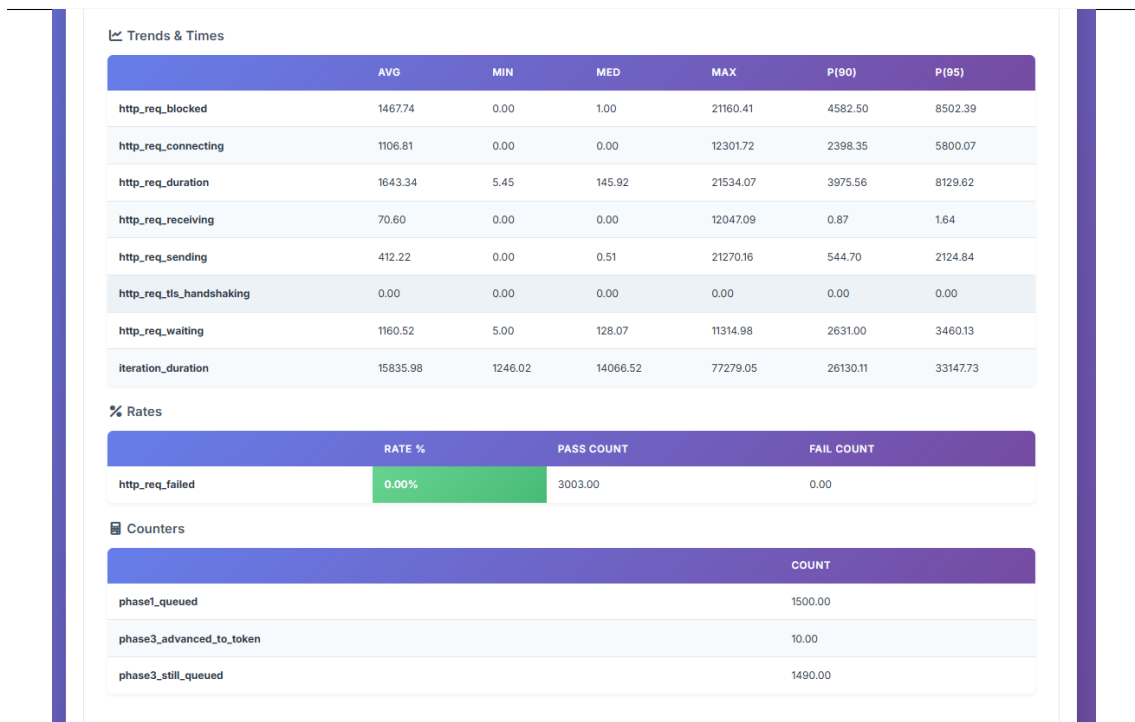


Hình 4.19: Kết quả kiểm thử tải kịch bản 1 — gia nhập hàng chờ đồng thời với 1500 VU

4.4.2 Kịch bản 2: Tính đúng đắn của cơ chế đẩy hàng FIFO

Kịch bản này xác minh cơ chế advance queue duy trì thứ tự FIFO chính xác khi có slot được giải phóng, thực hiện qua ba giai đoạn. Giai đoạn 1: lấp đầy tập active bằng `maxConcurrent` slot giả lập thông qua lời gọi `devFill`, đảm bảo hàng chờ không còn chỗ trống trước khi VU gia nhập. Giai đoạn 2: 1500 VU đồng thời gửi yêu cầu gia nhập — kỳ vọng toàn bộ vào QUEUED vì slot đã đầy, không có VU nào nhận TOKEN bất thường. Giai đoạn 3: giải phóng toàn bộ slot giả lập thông qua `devRelease`, sau đó hệ thống tự động kích hoạt advance queue và đẩy đúng `maxConcurrent` VU đứng đầu hàng lên TOKEN.

Kết quả thể hiện trong Hình 4.20 xác nhận toàn bộ 1500 VU ở giai đoạn 2 vào đúng trạng thái QUEUED, không có VU nào nhận token bất thường; sau khi giải phóng slot ở giai đoạn 3, đúng 10 VU được advance lên TOKEN — khớp chính xác với số slot được giải phóng. Kết quả xác nhận thứ tự FIFO được duy trì: VU nào vào hàng trước sẽ được advance trước, không có trường hợp nào bị bỏ qua hay xử lý sai thứ tự.



Hình 4.20: Kết quả kiểm thử tải kịch bản 2 — xác minh thứ tự FIFO khi đẩy hàng chờ

4.4.3 Kịch bản 3: Tính chính xác tồn kho vé dưới tải cao

Kịch bản này mô phỏng toàn bộ luồng từ gia nhập hàng chờ đến đặt vé với 1500 VU đồng thời. Mỗi VU thực hiện đầy đủ bốn bước: gia nhập hàng chờ, thăm dò trạng thái định kỳ nếu vào QUEUED, lấy mã đặt vé dùng một lần, và gửi yêu cầu đặt vé. Sự kiện được cấu hình với tổng tồn kho 30 vé và `maxConcurrent` là 10 — tức số yêu cầu đồng thời gấp 50 lần tồn kho.

Cần lưu ý rằng hàng chờ ảo kiểm soát số lượng người dùng được truy cập đồng thời, không kiểm soát trực tiếp tồn kho vé. Vì vậy, quá trình đặt vé diễn ra theo từng sóng: sóng 1 gồm 10 VU nhận token ngay và đặt thành công 10 vé (tồn kho: 30→20); sóng 2 và sóng 3 gồm mỗi sóng 10 VU được advance từ hàng chờ và lần lượt đặt thành công 10 vé mỗi sóng (tồn kho: 20→10→0); sóng 4 gồm 10 VU tiếp theo được advance với token hợp lệ nhưng khi gửi yêu cầu đặt vé thì tồn kho đã về 0 nên nhận phản hồi hết vé (HTTP 400) — đây là hành vi đúng, không phải lỗi. Khoảng 1460 VU còn lại trong hàng chờ không đến lượt trong thời gian thăm dò tối đa và kết thúc với trạng thái hết thời gian chờ — cũng là hành vi kỳ vọng khi tổng nhu cầu vượt xa tồn kho.

Kết quả thể hiện trong Hình 4.21 cho thấy chính xác 30 yêu cầu đặt vé thành công (HTTP 201), khớp đúng tổng tồn kho; không có phản hồi lỗi HTTP 409 hay HTTP 403 bất thường nào được ghi nhận. Kết quả xác nhận cơ chế kiểm tra và giữ chỗ nguyên tử bằng tập lệnh Lua trên Redis hoạt động chính xác — hệ thống không bán vượt số lượng ở mọi mức độ tranh chấp.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

	AVG	MIN	MED	MAX	P(90)	P(95)
http_req_blocked	272.35	0.00	0.00	17377.06	0.00	614.60
http_req_connecting	251.08	0.00	0.00	17377.06	0.00	589.21
http_req_duration	4188.83	4.40	4160.70	14713.63	6235.55	8574.40
http_req_receiving	1.11	0.00	0.00	5996.73	0.51	0.55
http_req_sending	27.26	0.00	0.00	11110.16	0.00	24.88
http_req_tls_handshaking	0.00	0.00	0.00	0.00	0.00	0.00
http_req_waiting	4160.46	4.40	4142.68	12247.85	6231.35	8573.33
iteration_duration	100176.98	20203.23	98117.60	114559.08	109411.00	110499.37
join_to_reserve_ms	37453.90	20023.00	42204.00	58315.00	50382.80	50937.50

% Rates

	RATE %	PASS COUNT	FAIL COUNT
http_req_failed	0.04%	23598.00	10.00
reserve_success_rate	2.00%	30.00	1470.00

Counters

	COUNT
poll_timeout	1460.00
reserve_fail	10.00
reserve_success	30.00
wr_queued_users	1490.00
wr_token_received	10.00

Hình 4.21: Kết quả kiểm thử tải kịch bản 3 — kiểm tra tồn kho vé với 1500 VU đồng thời

4.5 Triển khai

Frontend React được triển khai trên Vercel — mỗi lần push lên nhánh main trên GitHub tự động trigger build và deploy lên mạng phân phối nội dung (Content Delivery Network — CDN) toàn cầu, không cần cấu hình thêm.

Backend và toàn bộ service hạ tầng được triển khai trên VPS dưới dạng Docker Compose stack gồm năm container: `nestjs-app` (NestJS build từ Dockerfile đa giai đoạn, expose nội bộ cổng 3000), `nginx` (reverse proxy, xử lý HTTPS và forward đến NestJS), `postgres` (PostgreSQL 15 với pgvector, dữ liệu persist qua Docker named volume), `redis` (Redis 7, lưu bền bằng tệp chỉ thêm — Append Only File, viết tắt AOF), và `rabbitmq` (RabbitMQ 3.13 với delayed message plugin, image build tùy chỉnh). Toàn bộ container giao tiếp qua mạng nội bộ Docker bridge, chỉ expose cổng 80 và 443 của Nginx ra ngoài.

Chương 4 đã trình bày toàn bộ quá trình thiết kế, xây dựng, kiểm thử và triển khai hệ thống. Chương 5 sẽ phân tích các giải pháp và đóng góp nổi bật của đề tài.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày năm giải pháp kỹ thuật nổi bật được xây dựng trong quá trình phát triển hệ thống, bao gồm: kiểm soát tồn kho và hàng chờ ảo trong điều kiện lưu lượng đột biến, bảo vệ đa lớp chống bot, hệ gợi ý sự kiện theo ngữ nghĩa tiếng Việt, giao diện thiết kế sơ đồ chỗ ngồi tương tác, và quy trình xác minh nhà tổ chức cùng phê duyệt sự kiện. Mỗi giải pháp được phân tích từ bài toán gốc, qua thiết kế cụ thể, đến kết quả đo lường được, nhằm làm rõ giá trị kỹ thuật so với các tiếp cận thông thường.

5.1 Kiểm soát tồn kho vé và hàng chờ ảo trong điều kiện lưu lượng đột biến

5.1.1 Bài toán

Khi một sự kiện được chờ đón mở bán vé, lưu lượng truy cập thường tăng đột ngột từ vài trăm đến hàng nghìn yêu cầu trong vòng vài giây đầu tiên. Bài toán đặt ra hai thách thức đồng thời và có phần mâu thuẫn nhau: đảm bảo tính chính xác tuyệt đối về số lượng vé (không bán vượt tồn kho) trong khi vẫn duy trì thời gian phản hồi thấp cho tất cả người dùng.

Tiếp cận đơn giản nhất là kiểm tra tồn kho trong PostgreSQL trước khi ghi nhận đơn hàng. Tuy nhiên, khi nhiều luồng thực thi đồng thời, thao tác đọc–kiểm tra–ghi có thể bị xen kẽ giữa các luồng: hai yêu cầu đến đồng thời cùng đọc được tồn kho còn một vé, cả hai vượt qua kiểm tra, và kết quả là hai đơn hàng được tạo cho cùng một vé cuối — đây là lỗi bán vượt tồn kho điển hình do xung đột tương tranh. Khóa bi quan trên PostgreSQL giải quyết được xung đột tương tranh nhưng buộc các yêu cầu đồng thời phải chờ nhau tuần tự, làm tăng độ trễ đáng kể khi tải cao.

Ngoài bán vượt tồn kho, một vấn đề khác xuất hiện khi lưu lượng vượt quá khả năng xử lý: nếu hệ thống cho tất cả người dùng vào cùng lúc, đa phần sẽ thất bại sau khi vé đã hết, dẫn đến trải nghiệm tệ và cảm giác không công bằng. Người dùng không biết mình phải chờ bao lâu, không biết còn cơ hội hay không, và có xu hướng tải lại trang liên tục — càng làm nặng tải hệ thống.

5.1.2 Giải pháp

Hệ thống giải quyết thách thức thứ nhất bằng cách chuyển hoàn toàn bước kiểm tra và giữ tồn kho sang Redis với tập lệnh Lua nguyên tử. Redis thực thi mỗi tập lệnh Lua trong vòng lặp sự kiện đơn luồng — toàn bộ logic kiểm tra số lượng vé còn lại, trừ tồn kho, và ghi khóa giữ chỗ diễn ra như một thao tác bất khả phân, không thể bị gián đoạn bởi bất kỳ yêu cầu nào khác. Mỗi khóa giữ chỗ có thời gian sống (TTL) mười phút: nếu người dùng không hoàn thành thanh toán, tin nhắn trễ

của RabbitMQ tự động kích hoạt giải phóng vé mà không cần thăm dò định kỳ. PostgreSQL chỉ được ghi nhận sau khi giữ chỗ thành công, tránh hoàn toàn tranh chấp tài nguyên ở tầng cơ sở dữ liệu.

Giải thuật 1 trình bày thủ tục kiểm tra và giữ tồn kho nguyên tử. Thủ tục nhận hai khóa Redis — K_1 là khóa tồn kho và K_2 là khóa giữ chỗ của người dùng — cùng ba tham số: số lượng vé cần đặt q , dữ liệu giữ chỗ $data$, và thời gian sống tll . Thủ tục trả về bốn mã kết quả phân biệt: -2 nếu người dùng đã tồn tại khóa giữ chỗ cho loại vé này (tránh đặt trùng), -1 nếu khóa tồn kho chưa được đồng bộ vào Redis, 0 nếu tồn kho không đủ so với số lượng yêu cầu, và 1 nếu giữ chỗ thành công — tồn kho đã bị trừ và khóa giữ chỗ đã được ghi cùng TTL.

Algorithm 1: Kiểm tra và giữ tồn kho nguyên tử trên Redis

Input: K_1 : khóa tồn kho; K_2 : khóa giữ chỗ; q : số lượng cần đặt; $data$: dữ liệu giữ chỗ; tll : thời gian sống

Output: -2 nếu đặt trùng; -1 nếu chưa đồng bộ tồn kho; 0 nếu không đủ hàng; 1 nếu thành công

```

if EXISTS( $K_2$ ) = 1 then
    | return  $-2$ ;
 $stock \leftarrow$  GET( $K_1$ );
if  $stock = \emptyset$  then
    | return  $-1$ ;
if  $stock \geq q$  then
    | DECRBY( $K_1$ ,  $q$ );
    | SET( $K_2$ ,  $data$ , EX=  $tll$ );
    | return 1;
else
    | return 0;
    
```

Thách thức thứ hai được giải quyết bằng cơ chế hàng chờ ảo. Khi số người dùng đang trong luồng đặt vé vượt ngưỡng cấu hình, hệ thống chuyển sang chế độ hàng đợi: người dùng mới được xếp hàng và nhận cập nhật vị trí theo thời gian thực qua Socket.IO. Thiết kế này dựa trên kiến trúc kết hợp gồm hai thành phần bổ trợ nhau: Redis quản lý trạng thái hàng chờ với ba tập lệnh Lua nguyên tử (`joinOrQueue`, `advanceQueue`, `removeFromQueue`), trong khi RabbitMQ (`waiting-room-exchange`) xử lý quy trình bất đồng bộ để đẩy người dùng tiếp theo vào khi có vị trí trống.

Giải thuật 2 trình bày thủ tục `joinOrQueue` — thủ tục trung tâm xử lý nguyên tử toàn bộ quyết định cấp token hay xếp hàng. Thủ tục nhận bốn khóa: K_1 là bảng băm cấu hình hàng chờ theo sự kiện (chứa `maxConcurrent`, `maxQueueSize`), K_2 là tập hợp người dùng đang chiếm slot, K_3 là tập hợp có thứ tự lưu hàng đợi theo dấu thời gian FIFO, và K_4 là khóa token của người dùng. Thủ tục trả về bốn mã kết quả: $\{0\}$ nếu hàng chờ đang tắt, $\{1, token, ttl\}$ nếu cấp token thành công (bao gồm cả trường hợp người dùng tái kết nối còn token cũ), $\{2, position\}$ nếu được xếp vào hàng chờ kèm vị trí FIFO hiện tại, và $\{3, queueSize\}$ nếu hàng chờ đã đầy.

Luồng thực thi của Giải thuật 2 đi qua bảy bước kiểm tra tuần tự, mỗi bước giải quyết một trường hợp cụ thể.

Bước đầu tiên kiểm tra trường `enabled` trong bảng băm cấu hình K_1 : nếu hàng chờ đang tắt cho sự kiện đó, thủ tục trả về ngay $\{0\}$ để phía gọi biết rằng người dùng có thể tiến thẳng vào luồng đặt vé thông thường mà không cần xử lý hàng chờ.

Bước thứ hai đọc `maxConcurrent` từ cùng bảng băm; nếu giá trị này chưa được khởi tạo — tức là cấu hình hàng chờ chưa được nạp vào Redis — thủ tục cũng trả về $\{0\}$ theo hướng an toàn thay vì cấp token không kiểm soát.

Bước thứ ba xử lý trường hợp tái kết nối: nếu mã người dùng u đã có mặt trong tập slot đang chiếm K_2 , thủ tục kiểm tra xem token tương ứng tại K_4 còn hiệu lực không. Nếu còn, thủ tục trả về token hiện tại kèm thời gian sống còn lại, đảm bảo người dùng không bị đẩy ra khỏi hàng chờ chỉ vì tải lại trang hay mất kết nối tạm thời. Nếu token đã hết hạn nhưng mã người dùng vẫn còn trong K_2 do chưa được dọn dẹp, thủ tục chủ động xóa u khỏi K_2 trước khi tiếp tục xử lý, tránh rò rỉ slot.

Bước thứ tư kiểm tra hàng đợi K_3 : nếu u đã được xếp hàng từ một yêu cầu trước đó (ví dụ do mất kết nối rồi gọi lại), thủ tục truy vấn thứ hạng hiện tại bằng `ZRANK` và trả về $\{2, rank + 1\}$ mà không thêm u vào hàng lần nữa, giữ nguyên vị trí FIFO.

Bước thứ năm là nhánh cấp slot thông thường: nếu số người đang chiếm slot `activeCount` nhỏ hơn `maxConcurrent`, thủ tục thêm u vào K_2 , ghi token mới vào K_4 với TTL, và trả về $\{1, tok, ttl\}$.

Hai bước cuối xử lý trường hợp hàng đợi: nếu hàng đã đầy (số lượng phần tử trong K_3 đạt `maxQueueSize`) thủ tục trả về $\{3, queueSize\}$ để hệ thống từ chối yêu cầu; ngược lại, u được thêm vào K_3 bằng `ZADD` với điểm số là dấu thời gian ts , đảm bảo thứ tự FIFO, và thủ tục trả về vị trí hiện tại $\{2, rank + 1\}$.

Algorithm 2: Cấp token ngay hoặc xếp hàng chờ nguyên tử (joinOrQueue)

Input: K_1 : bảng băm cấu hình; K_2 : tập slot đang chiếm; K_3 : hàng đợi có thứ tự FIFO; K_4 : khóa token; u : mã người dùng; ts : dấu thời gian; tok : token mới; tll : thời gian sống token

Output: $\{0\}$ hàng chờ tắt; $\{1, token, ttl\}$ cấp token; $\{2, pos\}$ xếp hàng; $\{3, size\}$ hàng đầy

if HGET(K_1 , enabled) $\neq$ '1' **then**

return $\{0\}$;

$maxConcurrent \leftarrow$ HGET(K_1 , maxConcurrent);

if $maxConcurrent = \emptyset$ **then**

return $\{0\}$;

if $u \in K_2$ **then**

$existingToken \leftarrow$ GET(K_4);

if $existingToken \neq \emptyset$ **then**

return $\{1, existingToken, TTL(K_4)\}$;

else

 SREM(K_2 , u);

if $u \in K_3$ **then**

$rank \leftarrow$ ZRANK(K_3 , u);

return $\{2, rank + 1\}$;

$activeCount \leftarrow$ SCARD(K_2);

if $activeCount < maxConcurrent$ **then**

 SADD(K_2 , u);

 SET(K_4 , tok , EX= tll);

return $\{1, tok, tll\}$;

else

$queueSize \leftarrow$ ZCARD(K_3);

$maxQueue \leftarrow$ HGET(K_1 , maxQueueSize) hoặc giá trị dự phòng;

if $queueSize \geq maxQueue$ **then**

return $\{3, queueSize\}$;

else

 ZADD(K_3 , ts , u);

$rank \leftarrow$ ZRANK(K_3 , u);

return $\{2, rank + 1\}$;

Ngoài ra khi người dùng đang chiếm vị trí mà mất kết nối tạm thời (tải lại trang, chuyển tab, mất mạng ngắn), hệ thống giữ vị trí trong sáu mươi giây thay vì lập tức giải phóng và đẩy người tiếp theo vào. Cơ chế này ngăn người dùng mất lượt oan do sự cố kết nối không phải lỗi của họ, trong khi vẫn đảm bảo vị trí không bị chiếm vĩnh viễn nếu người dùng thực sự rời đi. Hàng chờ ảo có thể bật hoặc tắt độc lập theo từng sự kiện — khi tắt, toàn bộ luồng đặt vé hoạt động bình thường, đảm bảo tính tương thích ngược.

5.1.3 Kết quả

Giải pháp được kiểm thử bằng k6 theo ba kịch bản mô tả tại mục 4.4. Kịch bản thứ nhất với 1500 VU đồng thời tranh nhau cùng tập slot giới hạn: số token cấp ra không vượt quá `maxConcurrent` dù có 1500 yêu cầu đồng thời, phần còn lại được xếp vào hàng chờ. Kịch bản thứ hai xác minh tính đúng đắn của cơ chế đẩy hàng: 1500 VU gia nhập khi slot đã đầy đều vào đúng trạng thái `QUEUED`; sau khi giải phóng slot, số VU được `advance` khớp chính xác với số slot giải phóng — thứ tự FIFO được duy trì, không xảy ra xung đột tương tranh. Kịch bản thứ ba mô phỏng toàn bộ luồng đặt vé với 1500 VU đồng thời: số đơn hàng thành công (HTTP 201) khớp chính xác với tổng tồn kho, không xảy ra bán vượt số lượng ở bất kỳ mức độ tranh chấp nào. So với tiếp cận khóa bi quan PostgreSQL, tập lệnh Lua Redis duy trì độ trễ thấp do Redis xử lý trên bộ nhớ với thông lượng hàng triệu thao tác mỗi giây, không tạo tranh chấp tài nguyên ở tầng cơ sở dữ liệu.

5.2 Bảo vệ hệ thống chống bot và lạm dụng tự động

5.2.1 Bài toán

Bot và kịch bản tự động là mối đe dọa thường trực với hệ thống bán vé. Kịch bản phổ biến nhất là đầu cơ vé: bot đăng ký hàng loạt tài khoản, đặt vé ngay khi mở bán với tốc độ vượt xa người dùng thực, rồi bán lại với giá cao hơn. Kịch bản khác là chiếm giữ tồn kho: bot giữ vé ở trạng thái chờ thanh toán nhưng không thanh toán nhằm ngăn người dùng thực mua được. Một máy chủ bot có thể gửi hàng trăm yêu cầu mỗi giây, làm cạn kiệt tồn kho trước khi người dùng thực kịp thao tác.

Thách thức của bài toán là phân biệt bot với người dùng thực trong khi duy trì trải nghiệm mượt mà — giải pháp quá chặt sẽ chặn nhầm người dùng hợp lệ, giải pháp quá lỏng sẽ bị bot vượt qua dễ dàng. Hơn nữa, không một lớp bảo vệ đơn lẻ nào là đủ: bot hiện đại có thể vượt qua CAPTCHA bằng dịch vụ giải mã thuê ngoài, luân chuyển địa chỉ IP để qua giới hạn tốc độ, hay dùng nhiều tài khoản để qua giới hạn theo từng người dùng.

5.2.2 Giải pháp

Hệ thống triển khai bảo vệ chống bot theo mô hình đa lớp, mỗi lớp nhắm vào một hướng tấn công riêng biệt, để ngay cả khi bot vượt qua được một lớp, các lớp còn lại vẫn có hiệu lực.

Lớp thứ nhất là xác minh hành vi tại điểm đăng ký. Cloudflare Turnstile phân tích hành vi người dùng trong nền (chuyển động chuột, thời gian tương tác, dấu vân tay kỹ thuật số của trình duyệt) để phân biệt người thật với bot mà không cần người dùng giải câu đố. Lớp này ngăn bot tạo hàng loạt tài khoản từ đầu.

Lớp thứ hai là giới hạn tốc độ đa tầng trên Redis. Các điểm cuối nhạy cảm có ba chiều giới hạn độc lập: theo địa chỉ IP, theo mã người dùng, và theo cặp (mã người dùng, điểm cuối). Một tài khoản bị khóa tạm thời sau năm lần đăng nhập sai liên tiếp trong một phút; một địa chỉ IP bị chặn sau mười yêu cầu đặt vé trong ba mươi giây. Bộ đếm được lưu trên Redis với TTL tự đặt lại, không cần tác vụ định kỳ dọn dẹp.

Lớp thứ ba là booking nonce. Trước khi gửi yêu cầu đặt vé, phía máy khách bắt buộc phải gọi `GET /events/:id/nonce` để lấy một mã chỉ sử dụng được một lần duy nhất, hết hạn sau năm phút. Mã này được xác thực và vô hiệu hóa ngay khi nhận yêu cầu đặt vé, khiến bot không thể phát lại cùng một yêu cầu hay gửi song song nhiều yêu cầu từ cùng một mã. Mỗi lần đặt vé đòi hỏi ít nhất một yêu cầu HTTP riêng để lấy mã mới, tăng chi phí hoạt động của bot lên đáng kể.

Lớp thứ tư là giới hạn số đơn hàng đang chờ thanh toán đồng thời trên mỗi tài khoản cho cùng một sự kiện. Giới hạn này được kiểm tra bên trong tập lệnh Lua nguyên tử trên Redis, đảm bảo tính nhất quán ngay cả khi nhiều yêu cầu của cùng một tài khoản đến đồng thời.

5.2.3 Kết quả

Mô hình đa lớp đảm bảo mỗi lớp bảo vệ một giai đoạn riêng trong hành trình của bot: Cloudflare Turnstile chặn tại đăng ký, giới hạn tốc độ chặn tại tấn công ồ ạt, booking nonce chặn tại phát lại và gửi song song, giới hạn đơn hàng chờ chặn tại chiếm giữ tồn kho. Đặc biệt, booking nonce là lớp khó vượt nhất về chi phí: để đặt n vé đồng thời, bot cần lấy n mã riêng biệt qua n yêu cầu HTTP riêng biệt, khiến tốc độ tấn công bị giới hạn bởi độ trễ mạng và thông lượng máy chủ tương tự người dùng thực. Thời gian sống mười phút của khóa giữ chỗ từ giải pháp 5.1 cũng trực tiếp ngăn chiếm giữ tồn kho — bot không thể giữ vé mãi mãi mà không thanh toán.

5.3 Hệ gợi ý sự kiện cá nhân hóa theo ngữ nghĩa tiếng Việt

5.3.1 Bài toán

Với danh mục sự kiện ngày càng lớn, người dùng gặp khó khăn trong việc khám phá sự kiện phù hợp với sở thích. Bài toán gợi ý đặt ra hai yêu cầu: (i) gợi ý phải có chiều sâu ngữ nghĩa — không chỉ so khớp từ khóa mà phải hiểu được nội dung sự kiện, và (ii) gợi ý phải hoạt động được ngay cả với người dùng mới chưa có lịch sử (bài toán khởi động nguội — cold-start).

Thách thức kỹ thuật chính là ngôn ngữ tiếng Việt. Các mô hình véc-tơ nhúng đa ngôn ngữ tổng quát như multilingual-BERT được huấn luyện chủ yếu trên tiếng Anh và thường cho chất lượng thấp hơn trên tiếng Việt do đặc thù hình thái học và cú pháp. Nếu véc-tơ nhúng không phản ánh đúng ngữ nghĩa, toàn bộ hệ thống gợi ý sẽ cho kết quả kém chất lượng bất kể thuật toán tìm kiếm được chọn tốt đến đâu. Một thách thức khác là chi phí vận hành: các kho véc-tơ chuyên dụng như Pinecone, Weaviate hay Qdrant giải quyết tốt bài toán tìm kiếm nhưng tạo thêm một thành phần hạ tầng riêng biệt, tăng độ phức tạp triển khai và chi phí vận hành.

5.3.2 Giải pháp

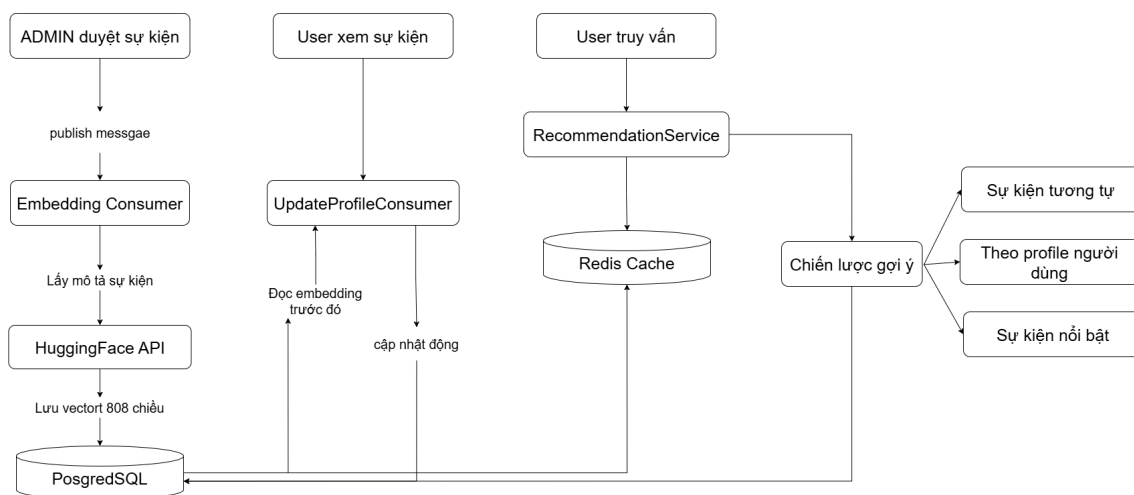
Hệ thống biểu diễn mỗi sự kiện bằng véc-tơ 808 chiều là sự kết hợp của hai nguồn thông tin bổ trợ nhau. Thành phần ngữ nghĩa 768 chiều được sinh ra bởi mô hình Vietnamese-SBERT (`keepitreal/vietnamese-sbert` trên Hugging-Face) — mô hình được tinh chỉnh trực tiếp trên kho dữ liệu huấn luyện tiếng Việt, nắm bắt quan hệ ngữ nghĩa giữa tên và mô tả sự kiện tốt hơn các mô hình đa ngôn ngữ tổng quát. Thành phần có cấu trúc 40 chiều bổ sung các đặc trưng định lượng không nằm trong văn bản: mã hóa một-nóng (one-hot) cho loại sự kiện (âm nhạc, thể thao, hội thảo, triển lãm...), mã hóa địa điểm theo tỉnh/thành phố, và mức giá chuẩn hóa về đoạn $[0, 1]$. Ghép nối hai thành phần tạo ra véc-tơ 808 chiều được lưu trong PostgreSQL với kiểu `vector(808)` của extension `pgvector`, trên chỉ mục HNSW cho phép tìm kiếm lân cận gần nhất xấp xỉ (approximate nearest neighbor) với hiệu năng cao.

Hệ thống triển khai ba chiến lược gợi ý tương ứng với ba trường hợp sử dụng. Chiến lược *item-to-item* tính độ tương đồng cosine (cosine similarity) giữa véc-tơ của sự kiện đang xem và toàn bộ danh mục, gợi ý sự kiện tương tự về nội dung lẫn địa điểm và mức giá — hoạt động cho cả người dùng chưa đăng nhập. Chiến lược *user-to-item* xây dựng véc-tơ hồ sơ người dùng (profile vector) bằng cách tính trung bình có trọng số các véc-tơ sự kiện trong lịch sử đặt vé (sự kiện gần đây có trọng số cao hơn), sau đó tìm sự kiện gần nhất với véc-tơ đó trong không gian véc-tơ — người dùng càng có nhiều lịch sử, gợi ý càng cá nhân hóa. Chiến lược theo độ phổ

biến là phương án dự phòng cho người dùng mới chưa có lịch sử, dựa trên số lượng đơn đặt vé trong khoảng thời gian cấu hình.

Toàn bộ quá trình tính véc-tơ nhúng diễn ra bất đồng bộ qua RabbitMQ: khi nhà tổ chức tạo hoặc cập nhật sự kiện, tin nhắn được phát lên `recommendation-exchange`; tiến trình tiêu thụ nhận tin nhắn, gọi HuggingFace Inference API, rồi cập nhật hoặc chèn mới kết quả vào bảng `EventEmbedding`. Kết quả gợi ý được lưu vào bộ nhớ đệm trên Redis với thời gian sống nhiều cấp (gợi ý cá nhân: mười phút; gợi ý theo độ phổ biến: ba mươi phút) để giảm tần suất truy vấn pgvector.

Hình 5.1 minh họa tổng quan ba luồng xử lý của hệ thống gợi ý: luồng tính embedding bất đồng bộ khi sự kiện được duyệt, luồng cập nhật hồ sơ người dùng khi có lượt xem, và luồng phục vụ gợi ý theo thời gian thực với bộ nhớ đệm Redis.



Hình 5.1: Kiến trúc ba luồng của hệ thống gợi ý sự kiện

5.3.3 Kết quả

Việc lưu véc-tơ nhúng trực tiếp trong PostgreSQL thay vì kho véc-tơ riêng mang lại lợi thế vận hành đáng kể: một giao dịch có thể cập nhật đồng thời thông tin sự kiện và véc-tơ nhúng tương ứng, đảm bảo nhất quán dữ liệu; việc chuyển đổi lược đồ của TypeORM áp dụng cho cả bảng véc-tơ nhúng như mọi bảng khác; không cần quản lý thêm một dịch vụ hạ tầng riêng trên môi trường thực tế. Chỉ mục HNSW cho thời gian truy vấn lân cận gần nhất dưới 20ms trên tập dữ liệu thử nghiệm. Véc-tơ lai ghép ngữ nghĩa và đặc trưng cấu trúc cho phép gợi ý phù hợp cả về nội dung lẫn địa lý và mức giá — sự kiện âm nhạc ở Hà Nội sẽ không được gợi ý cho người dùng ở thành phố Hồ Chí Minh với lịch sử chủ yếu là hội thảo công nghệ.

5.4 Giao diện thiết kế sơ đồ chỗ ngồi tương tác theo khu vực

5.4.1 Bài toán

Các sự kiện như hòa nhạc, thể thao, hay hội nghị thường có sơ đồ chỗ ngồi phức tạp với nhiều khu vực hình dạng tùy ý. Yêu cầu từ §2.3.4 đặt ra hai vế: nhà tổ chức phải tự thiết kế sơ đồ ngay trên trình duyệt mà không cần công cụ bên ngoài, và người mua vé phải thấy sơ đồ đó theo thời gian thực với trạng thái còn trống hay đã bán.

Về đầu đặt ra thách thức kỹ thuật: cần xử lý hàng chục đối tượng đồ họa có thể kéo thả, thay đổi kích thước, và phân nhóm theo khu vực, tất cả trong trình duyệt với hiệu năng đủ mượt. SVG là lựa chọn tự nhiên cho đồ họa véc-tơ trên web nhưng hiệu năng suy giảm rõ rệt khi số lượng phần tử lớn do chi phí thao tác cây DOM. Về thứ hai yêu cầu sơ đồ hiển thị nhất quán trên mọi kích thước màn hình — nhà tổ chức thiết kế trên máy tính để bàn rộng 1920 điểm ảnh, người mua vé xem trên điện thoại thông minh 375 điểm ảnh phải thấy cùng bố cục tương đối.

5.4.2 Giải pháp

Giao diện thiết kế được xây dựng trên Konva.js với liên kết `react-konva`, vẽ trên HTML5 Canvas. Canvas xử lý hàng nghìn hình dạng thông qua vẽ được tăng tốc phần cứng GPU thay vì thao tác cây DOM, cho hiệu năng đều đặn bất kể số lượng phần tử. Giao diện lập trình xử lý sự kiện của Konva (kéo thả, nhấp chuột, di chuột, thay đổi kích thước) trực tiếp trên đối tượng canvas giúp triển khai tương tác kéo thả mà không cần thư viện bổ sung.

Giải pháp cho bài toán thích nghi theo màn hình là lưu tọa độ theo tỷ lệ phần trăm thay vì điểm ảnh tuyệt đối. Mỗi khu vực được lưu với tọa độ chuẩn hóa như `{x: 0.35, y: 0.42, width: 0.15, height: 0.08}` — con số 0.35 nghĩa là khu vực bắt đầu ở 35% chiều rộng canvas, không phụ thuộc canvas rộng bao nhiêu điểm ảnh. Khi hiển thị, hệ thống nhân tọa độ với kích thước canvas thực tế tại thời điểm đó. Cách tiếp cận này đảm bảo thích nghi theo màn hình hoàn toàn chỉ với phép nhân đơn giản, không cần điểm ngắt giao diện hay truy vấn phương tiện. Toàn bộ cấu hình sơ đồ được lưu dưới dạng JSONB trong PostgreSQL, cho phép nhà tổ chức cập nhật sơ đồ mà không cần thay đổi lược đồ cơ sở dữ liệu.

Phần cập nhật theo thời gian thực trạng thái vé tích hợp Socket.IO: khi một vé được tạo sau thanh toán thành công, máy chủ phát sự kiện `ticket-sold` kèm thông tin khu vực đến tất cả máy khách đang xem trang sự kiện đó (qua phòng Socket.IO theo mã sự kiện); phía máy khách nhận sự kiện và cập nhật màu sắc khu vực trên canvas tức thì mà không cần tải lại trang.

5.4.3 Kết quả

Hình 4.15 minh họa giao diện trình soạn sơ đồ chỗ ngồi thực tế với các khu vực được phân vùng, gán màu theo loại vé và căn chỉnh bằng thao tác kéo thả trực tiếp trên canvas.

Nhà tổ chức có thể tạo sơ đồ chỗ ngồi hoàn toàn tùy chỉnh trong trình duyệt với trải nghiệm kéo thả trực quan. Việc lưu tọa độ theo tỷ lệ phần trăm giải quyết bài toán thích nghi theo màn hình mà không cần logic chuyển đổi riêng cho từng điểm ngắt giao diện — cùng một dữ liệu JSONB hiển thị chính xác trên mọi thiết bị. Cập nhật theo thời gian thực số chỗ còn trống tạo cảm giác khan hiếm trung thực, khuyến khích người dùng ra quyết định nhanh hơn và giảm tỷ lệ bỏ giỏ hàng.

5.5 Quy trình xác minh nhà tổ chức và phê duyệt sự kiện

5.5.1 Bài toán

Hệ thống đặt vé trực tuyến hoạt động như một nền tảng trung gian hai chiều: người mua tin tưởng rằng sự kiện họ đặt vé là có thật và nhà tổ chức sẽ thực hiện đúng cam kết; nhà tổ chức tin tưởng rằng doanh thu sẽ được quyết toán đầy đủ sau sự kiện. Nếu không có cơ chế xác minh, bất kỳ ai cũng có thể tạo sự kiện giả mạo để thu tiền vé — đây là rủi ro nghiêm trọng với người mua lẫn uy tín nền tảng.

Bài toán đặt ra yêu cầu đối nghịch: xác minh đủ chặt để ngăn gian lận, nhưng không tạo rào cản quá cao khiến các nhà tổ chức uy tín bỏ qua nền tảng. Ngoài ra, cần bảo vệ người mua ngay cả sau khi sự kiện được phê duyệt — nhà tổ chức có thể hành động không trung thực sau đó như hủy sự kiện đột ngột hay thay đổi điều khoản.

5.5.2 Giải pháp

Hệ thống thiết kế quy trình xác minh hai cấp độc lập: cấp nhà tổ chức và cấp sự kiện.

Ở cấp nhà tổ chức, người dùng đăng ký vai trò nhà tổ chức phải cung cấp thông tin định danh bổ sung so với tài khoản thông thường. Hồ sơ này được quản trị viên xét duyệt thủ công; tài khoản chỉ được nâng lên vai trò ORGANIZER sau khi phê duyệt. Bước này đảm bảo mỗi nhà tổ chức trên nền tảng là một thực thể có thể truy vết và chịu trách nhiệm.

Ở cấp sự kiện, mỗi sự kiện do nhà tổ chức tạo ra phải trải qua quy trình phê duyệt trước khi được công khai và nhận đặt vé. Sự kiện chuyển qua các trạng thái: nháp → chờ xét duyệt → công khai, hoặc cần chỉnh sửa hay hủy kèm lý do từ quản trị viên. Cơ chế này ngăn nhà tổ chức — dù đã được xác minh — đăng sự kiện có thông tin sai lệch hay vi phạm quy định nền tảng mà không qua kiểm soát.

Tầng bảo vệ thứ ba là cơ chế quyết toán tài chính theo mô hình ký quỹ. Doanh thu từ đặt vé không chuyển ngay cho nhà tổ chức mà được nền tảng giữ lại; chỉ sau khi sự kiện kết thúc và hết thời gian khiếu nại, doanh thu mới được giải ngân.

Nếu sự kiện bị hủy, hệ thống tự động khởi động quy trình hoàn tiền hàng loạt cho toàn bộ người mua qua RabbitMQ. Tại thời điểm đặt vé, hệ thống ghi nhận tức thời các thông số tài chính (`platformFeePercent`, `organizerAmount`) vào bảng `Settlement`, tránh tranh chấp do thay đổi chính sách phí về sau.

5.5.3 Kết quả

Quy trình hai cấp tạo ra hai điểm kiểm soát độc lập: ngay cả khi tài khoản nhà tổ chức bị chiếm đoạt, kẻ tấn công vẫn phải vượt qua cấp phê duyệt sự kiện — trong khi đó quản trị viên có thể đình chỉ tài khoản — trước khi có thể thu tiền từ người mua. Mô hình ký quỹ đảm bảo người mua không mất tiền ngay cả trong trường hợp nhà tổ chức hành động không trung thực sau phê duyệt; việc ghi nhận tức thời thông số tài chính loại bỏ khả năng tranh chấp về tỷ lệ phí. Kết hợp với cơ chế hoàn tiền hàng loạt tự động qua RabbitMQ, hệ thống xử lý được kịch bản hủy sự kiện mà không cần quản trị viên can thiệp thủ công vào từng đơn hàng.

Chương 5 đã phân tích năm giải pháp kỹ thuật nổi bật, từ kiểm soát tồn kho theo thời gian thực không bán vượt số lượng, bảo vệ đa lớp chống bot, gợi ý ngữ nghĩa tiếng Việt với pgvector, giao diện sơ đồ chỗ ngồi thích nghi theo màn hình, đến quy trình xác minh và ký quỹ bảo vệ người mua. Chương 6 sẽ tổng kết kết quả đạt được và định hướng phát triển tiếp theo của hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết toàn bộ quá trình thực hiện đồ án theo hai phần chính. Phần đầu đánh giá mức độ hoàn thành các mục tiêu đề ra, bao gồm kết quả kỹ thuật đã đạt được, kết quả kiểm thử tải thực tế, và những hạn chế còn tồn tại cần thừa nhận. Phần sau đề xuất các hướng phát triển tiếp theo nhằm hoàn thiện hệ thống về chiều rộng lẫn chiều sâu, từ tích hợp thêm cổng thanh toán và ứng dụng di động đến chuyển đổi kiến trúc và ứng dụng mô hình ngôn ngữ lớn hỗ trợ ban tổ chức.

6.1 Kết luận

Đồ án đã xây dựng một hệ thống đặt vé trực tuyến hướng đến bài toán thực tế của thị trường Việt Nam, nơi các sự kiện quy mô lớn thường xuyên xảy ra tình trạng bán vượt số lượng, hệ thống sập do tải đột biến và thiếu cơ chế bảo vệ người mua lẫn ban tổ chức.

Trong suốt quá trình thực hiện đồ án, em đã hoàn thành được những nội dung cốt lõi sau đây.

Thứ nhất, luồng đặt vé áp dụng cơ chế kiểm tra và giữ tồn kho nguyên tử bằng Lua Script trên Redis, đảm bảo tuyệt đối không xảy ra bán vượt số lượng ngay cả khi có hàng trăm yêu cầu đồng thời.

Thứ hai, hàng chờ ảo được xây dựng theo kiến trúc kết hợp giữa Redis xử lý trạng thái thời gian thực và RabbitMQ điều phối luồng bất đồng bộ, hỗ trợ cơ chế ăn hạn 60 giây giúp người dùng không bị mất chỗ khi tải lại trang.

Thứ ba, sơ đồ ghế tương tác được xây dựng bằng Konva.js với giao tiếp thời gian thực qua Socket.IO, cho phép người dùng chọn ghế trực quan và thấy ngay trạng thái ghế thay đổi của những người dùng khác.

Thứ tư, hệ thống gợi ý sự kiện áp dụng mô hình ngôn ngữ Vietnamese-SBERT kết hợp với tìm kiếm véc-tơ trên pgvector theo ba chiến lược: gợi ý theo sự kiện tương tự, theo hồ sơ người dùng và theo độ phổ biến.

Thứ năm, cơ chế chống bot được triển khai theo nhiều lớp gồm xác thực CAPTCHA qua Cloudflare Turnstile, booking nonce và giới hạn tần suất nhiều tầng.

Thứ sáu, mô-đun thanh toán và ký quỹ xử lý vòng đời tài chính từ lúc người mua thanh toán đến khi ban tổ chức nhận tiền sau thời gian ký quỹ tương ứng với quy mô sự kiện. Kết quả kiểm thử tải bằng k6 với 1500 VU đồng thời trên ba kịch bản cho thấy hệ thống không xảy ra bán vượt số lượng và hàng chờ ảo điều phối đúng

thứ tự FIFO — số đơn hàng thành công khớp chính xác với tổng tồn kho, số VU được advance từ hàng chờ khớp đúng số slot được giải phóng.

Bên cạnh những kết quả đạt được, đề án vẫn còn một số hạn chế cần thừa nhận. Về tích hợp thanh toán, hệ thống hiện chỉ hỗ trợ cổng thanh toán SePay thông qua mã QR, chưa tích hợp các cổng phổ biến khác tại Việt Nam như MoMo hay VNPay. Về hệ thống gợi ý, mô hình Vietnamese-SBERT được gọi qua HuggingFace API, tạo ra sự phụ thuộc vào dịch vụ bên ngoài và tăng độ trễ so với tự triển khai. Ngoài ra, hệ thống hiện chỉ có giao diện web, chưa có ứng dụng di động dành cho người dùng cuối.

Qua quá trình thực hiện đề án, em rút ra một số bài học kinh nghiệm quan trọng. Bài học đầu tiên là cần thiết phân tích thiết kế hệ thống thật kỹ trước khi triển khai đặc biệt đối với các luồng quan trọng tránh việc phải sửa đổi nhiều lần trong quá trình phát triển. Bài học thứ hai là cần thiết lập các công cụ giám sát và kiểm thử tải từ sớm để phát hiện các vấn đề về hiệu năng và khả năng mở rộng trước khi triển khai thực tế. Bài học thứ ba là cần thiết nghiên cứu kỹ các công nghệ mới, đặc biệt là các mô hình ngôn ngữ lớn, để tận dụng tối đa khả năng của chúng trong việc nâng cao trải nghiệm người dùng và hỗ trợ ra quyết định cho ban tổ chức.

6.2 Hướng phát triển

Dựa trên nền tảng đã xây dựng, có nhiều hướng có thể tiếp tục để hoàn thiện và nâng cấp hệ thống trong tương lai.

Đối với hệ thống gợi ý, hướng ưu tiên là tự triển khai mô hình Vietnamese-SBERT trên cơ sở hạ tầng riêng nhằm loại bỏ phụ thuộc vào HuggingFace API, giảm độ trễ và kiểm soát tốt hơn tính khả dụng của dịch vụ.

Đối với thanh toán, cần tích hợp thêm các cổng phổ biến tại Việt Nam như MoMo và VNPay để mở rộng tệp người dùng; lớp trừu tượng hóa cổng thanh toán hiện tại cho phép bổ sung cổng mới mà không cần thay đổi luồng nghiệp vụ cốt lõi.

Đối với mô-đun ký quỹ, cần kết nối với API ngân hàng hoặc ví điện tử để tự động hóa bước giải ngân sau khi hết thời gian ký quỹ, thay vì xử lý thủ công như hiện tại.

Đối với tính năng hủy sự kiện và hoàn tiền hàng loạt, cần hoàn thiện luồng quản trị đầu cuối: nhà tổ chức gửi yêu cầu hủy kèm đề xuất phương án hoàn tiền, quản trị viên phê duyệt hoặc từ chối, sau đó hệ thống tự động phát lệnh hàng loạt lên RabbitMQ để xử lý song song hoàn tiền từng đơn hàng, cập nhật trạng thái vé thành đã hủy và gửi thư điện tử xác nhận hoàn tiền đến từng người mua.

Tiếp theo, một số hướng mới cho phép nâng cấp hệ thống lên một tầm cao hơn. Hướng thứ nhất là phát triển ứng dụng di động bằng React Native, tái sử dụng toàn bộ REST API và Socket.IO hiện có, nhằm phục vụ phân khúc người dùng mua vé chủ yếu qua điện thoại. Hướng thứ hai là chuyển dần sang kiến trúc vi dịch vụ: tách dịch vụ gợi ý và dịch vụ thông báo thành các đơn vị triển khai độc lập, giúp mỗi thành phần có thể mở rộng theo chiều ngang tương ứng với tải thực tế. Hướng thứ ba là ứng dụng mô hình ngôn ngữ lớn hỗ trợ ban tổ chức tự động sinh mô tả sự kiện, gợi ý giá vé phù hợp dựa trên dữ liệu lịch sử và phân tích phản hồi người mua. Hướng thứ tư là xây dựng bảng điều khiển phân tích cho ban tổ chức, trực quan hóa doanh thu theo thời gian, bản đồ nhiệt tỷ lệ lấp đầy ghế và phân tích hành vi người dùng, từ đó hỗ trợ ra quyết định định giá và marketing hiệu quả hơn.

TÀI LIỆU THAM KHẢO

- [1] OpenJS Foundation, *Node.js Documentation*, Truy cập tháng 3 năm 2026. [Online]. Available: <https://nodejs.org/en/docs/>
- [2] NestJS contributors, *NestJS Documentation*, Truy cập tháng 3 năm 2026. [Online]. Available: <https://docs.nestjs.com/>
- [3] Microsoft Corporation, *TypeScript Documentation*, Truy cập tháng 3 năm 2026. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [4] The PostgreSQL Global Development Group, *PostgreSQL 15 Documentation*, Truy cập tháng 4 năm 2026. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [5] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [6] TypeORM contributors, *TypeORM Documentation*, Truy cập tháng 4 năm 2026. [Online]. Available: <https://typeorm.io/>
- [7] Redis Ltd., *Redis Documentation*, Truy cập tháng 4 năm 2026. [Online]. Available: <https://redis.io/docs/>
- [8] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [9] A. Kane, *pgvector: Open-source vector similarity search for Postgres*, Truy cập tháng 5 năm 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [10] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [11] Broadcom Inc., *RabbitMQ Documentation*, Truy cập tháng 5 năm 2026. [Online]. Available: <https://www.rabbitmq.com/documentation.html>
- [12] Socket.IO contributors, *Socket.IO Documentation*, Truy cập tháng 5 năm 2026. [Online]. Available: <https://socket.io/docs/>
- [13] I. Fette and A. Melnikov, “The WebSocket Protocol,” Internet Engineering Task Force, RFC 6455, 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>

- [14] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [15] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [16] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [17] Meta Platforms, Inc., *React Documentation*, Truy cập tháng 3 năm 2026. [Online]. Available: <https://react.dev/>
- [18] Docker Inc., *Docker Documentation*, Truy cập tháng 5 năm 2026. [Online]. Available: <https://docs.docker.com/>